

Complete Interview Prep Master Guide

Generated automatically on: 2026-08-06

Topic: array

Notes & Concept Guide

Array Problems - Interview Prep Guide

Overview

This folder contains **25 problems** focused on array manipulation, covering fundamental to advanced techniques.

Problem Categories

1. Basic Operations (5 problems)

- `MergeSortedArrays.java` - Merge two sorted arrays
- `RemoveElement.java` - Remove specific element in-place
- `RemoveDuplicates.java` - Remove duplicates from sorted array
- `SearchInsertPosition.java` - Binary search variant
- `ContainsDuplicate.java` - Check for duplicates

Time Complexity: $O(n)$ to $O(n \log n)$ **Space Complexity:** $O(1)$ to $O(n)$

2. Zero/Element Manipulation (4 problems)

- `MoveZeroes.java` - Move all zeros to end maintaining order
- `SeparateEvenOdd.java` - Separate even and odd numbers
- `NumbersStartingWithDigit.java` - Filter numbers by starting digit
- `ReverseArray.java` - Reverse array elements

Technique: In-place manipulation, two-pointers **Best For:** Space-optimized solutions

3. Range & Subarray Problems (7 problems)

- `RangeSumQuery.java` - Range sum queries using prefix sums
- `MaxSumSubArray.java` - Maximum sum subarray (Kadane's algorithm)
- `ZeroSumSubarrays.java` - Find subarrays with zero sum
- `TrappedRainWater.java` - Classic rain water trapping problem
- `MaxAverageSubarray.java` - Maximum average subarray
- `PairSumOfSortedArray.java` - Find pair with given sum
- `StockBuySell.java` - Best time to buy and sell stock

Techniques:

- Prefix sums
- Sliding window
- Dynamic programming
- Two-pointers

4. Sorting & Searching (4 problems)

- `SortedSquares.java` - Squares of sorted array
- `MissingNumber.java` - Find missing number
- `ProductArray.java` - Product of array except self
- `TwoSum.java` - Two sum problem (HashMap and Two-pointer approaches)

Challenge: Solve without division, in $O(n)$ time

5. Comparison & Analysis (3 problems)

- `LargestElement.java` - Find maximum element
- `LargestUniqueNumber.java` - Largest unique number
- `SecondSmallestLargest.java` - Both second largest and smallest
- `MinMaxArray.java` - Find min and max

Optimization: Find both in single pass with $n+1$ comparisons

6. Data Structure Collections (3 problems)

- `RemoveDupeCharFromList.java` - Remove duplicate characters from string array
-

Key Techniques to Master

1. Two-Pointer Technique

Used in: `MoveZeroes.java`, `SeparateEvenOdd.java`, `PairSumOfSortedArray.java`

- Useful for sorted arrays
- Space-efficient ($O(1)$ extra space)
- Common pattern: slow/fast pointers or left/right pointers

2. Sliding Window

Used in: `MaxAverageSubarray.java`, `PairSumOfSortedArray.java`

- Optimal for subarray problems
- Time Complexity: $O(n)$
- Commonly asked in interviews

3. Prefix Sums

Used in: `RangeSumQuery.java`, `ZeroSumSubarrays.java`

- Precompute to answer range queries efficiently
- Trade space ($O(n)$) for time ($O(1)$ per query)

4. Kadane's Algorithm

Used in: [MaxSumSubArray.java](#)

- Classic for maximum subarray problem
 - Time: $O(n)$, Space: $O(1)$
 - Can be extended for other variants
-

Interview Preparation Checklist

Easy Level (Start Here)

- ☐ MergeSortedArrays.java
- ☐ RemoveElement.java
- ☐ MoveZeroes.java
- ☐ LargestElement.java
- ☐ RemoveDuplicates.java
- ☐ ReverseArray.java

Medium Level

- ☐ MaxSumSubArray.java
- ☐ TrappedRainWater.java
- ☐ ProductArray.java
- ☐ SeparateEvenOdd.java
- ☐ PairSumOfSortedArray.java
- ☐ TwoSum.java
- ☐ MaxAverageSubarray.java

Hard Level

- ☐ ZeroSumSubarrays.java
 - ☐ StockBuySell.java
 - ☐ MissingNumber.java
 - ☐ SortedSquares.java
-

Common Interview Questions

1. How to find duplicates with $O(1)$ space?

- See: [ContainsDuplicate.java](#)
- Technique: Modify array in-place

2. Find maximum sum of subarray

- See: [MaxSumSubArray.java](#)

- Algorithm: Kadane's Algorithm

3. Rearrange array with $O(1)$ space

- See: `MoveZeroes.java`, `SeparateEvenOdd.java`
- Technique: In-place two-pointer

4. Range sum queries

- See: `RangeSumQuery.java`
- Optimization: Prefix sum array

5. Stock trading problems

- See: `StockBuySell.java`
- Similar to max subarray

Time Complexity Summary

Problem	Time	Space	Technique
Merge Sorted	$O(n)$	$O(n)$	Two pointers
Max Sum	$O(n)$	$O(1)$	Kadane's
Trapped Water	$O(n)$	$O(1)$	Two pointers
Product Array	$O(n)$	$O(n)$	Prefix/Suffix
Find Missing	$O(n)$	$O(1)$	XOR/Math
Subarray Sum	$O(n)$	$O(n)$	HashMap
Duplicate Check	$O(n)$	$O(1)^*$	In-place mark

Review Notes

Most Important:

1. Two-pointer technique - used in 5+ problems
2. In-place modifications - space optimization is key
3. Prefix sums - powerful for range queries

Practice Priority:

1. Start with "Easy" problems to understand patterns
 2. Compare different approaches to same problem
 3. Focus on space optimization in interviews
-

Generated: Interview Prep Package - Array Folder

Code Solutions

ContainsDuplicate.java

```
package programs.array;

import org.testng.Assert;

import java.util.Arrays;
import java.util.HashSet;
import java.util.Set;

/**
 * LeetCode 217: Contains Duplicate
 * Problem Statement:
 * Given an integer array nums, return true if any value appears at least twice
 * in the array, and return false if every element is distinct.
 * Constraints:
 * - 1 <= nums.length <= 10^5
 * - -10^9 <= nums[i] <= 10^9
 */
public class ContainsDuplicate {

    public static void main(String[] args) {
        Assert.assertTrue(containsDuplicateWithSort(new int[]{1, 2, 3, 1}));
        Assert.assertFalse(containsDuplicate(new int[]{1, 2, 3, 4}));
        Assert.assertFalse(containsDuplicateBruteForce(new int[]{1, 2, 3, 4}));
    }

    /**
     * APPROACH 1: Sorting (In-Place)
     * Time Complexity: O(N log N)
     * - Arrays.sort(int[]) uses Dual-Pivot Quicksort.
     * - Sorting takes O(N log N) on average.
     * - Scanning adjacent elements takes O(N).
     * Space Complexity: O(log N)
     * - Auxiliary space is O(log N) due to the recursion stack used by Dual-Pivot
     Quicksort.
     * - Note: The array is sorted in-place (the input array is mutated).
     */
    public static boolean containsDuplicateWithSort(int[] nums) {
        if (nums == null || nums.length <= 1) {
            return false;
        }
        Arrays.sort(nums);
        for (int i = 0; i < nums.length - 1; i++) {
            if (nums[i] == nums[i + 1]) {
                return true;
            }
        }
        return false;
    }

    /**
```

```

    * APPROACH 2: HashSet (Optimal Time)
    * Time Complexity: O(N)
    * - Inserting into and querying a HashSet takes O(1) time on average.
    * - Iterating through the array takes at most N steps.
    * Space Complexity: O(N)
    * - In the worst case (when all elements are distinct), the HashSet stores N
integers.
    */
    public static boolean containsDuplicate(int[] nums) {
        if (nums == null || nums.length <= 1) {
            return false;
        }
        Set<Integer> dupeFilter = new HashSet<>();
        for (int i : nums) {
            if (!dupeFilter.add(i)) {
                return true;
            }
        }
        return false;
    }

    /**
    * APPROACH 3: Brute Force (Nested Loops)
    * Time Complexity: O(N^2)
    * - Checks every unique pair (N * (N - 1) / 2 comparisons in the worst case).
    * Space Complexity: O(1)
    * - Uses auxiliary constant space only; does not mutate the original array.
    */
    public static boolean containsDuplicateBruteForce(int[] nums) {
        if (nums == null || nums.length <= 1) {
            return false;
        }
        for (int i = 0; i < nums.length; i++) {
            for (int j = i + 1; j < nums.length; j++) {
                if (nums[i] == nums[j]) {
                    return true;
                }
            }
        }
        return false;
    }
}

```

LargestElement.java

```

package programs.array;

import org.testng.Assert;

/**
 * LeetCode (Various): Find Maximum Element in Array

```

```

* <p>
* Problem Statement:
* Given an array of integers, find the largest element in the array.
* <p>
* Constraints:
* - 1 <= nums.length <= 10^5
* - -10^9 <= nums[i] <= 10^9
*/
public class LargestElement {

    public static void main(String[] args) {
        int[] arr = new int[]{1, 2, 3, 1, 2, 3, 4, 5};
        Assert.assertEquals(findLargestLinear(arr), 5);
    }

    /**
     * APPROACH 1: Linear Scan (Optimal)
     * <p>
     * Time Complexity: O(N)
     * - Single pass through the array.
     * <p>
     * Space Complexity: O(1)
     * - Uses constant extra space.
     */
    public static int findLargestLinear(int[] nums) {
        if (nums == null || nums.length == 0) {
            throw new IllegalArgumentException("Array is empty");
        }
        int max = Integer.MIN_VALUE;
        for (int i : nums) {
            if (i > max)
                max = i;
        }
        return max;
    }
}

```

LargestUniqueNumber.java

```

package programs.array;

import org.testng.Assert;

import java.util.HashMap;
import java.util.Map;

/**
 * LeetCode 1133: Largest Unique Number
 * Given an array of integers `nums`, return the largest integer that occurs only

```

```

once.
* If no such integer exists, return -1.
* Constraints:
* - 1 <= nums.length <= 2000
* - 0 <= nums[i] <= 1000
*/
public class LargestUniqueNumber {

    public static void main(String[] args) {
        // Test Approach 1
        Assert.assertEquals(findLargestUniqueNumber(new int[]{2, 2, 2, 2}), -1);
        Assert.assertEquals(findLargestUniqueNumber(new int[]{1, 2, 2, 3, 3, 4, 4,
4}), 1);
        Assert.assertEquals(findLargestUniqueNumber(new int[]{1, 2, 3, 4, 5}), 5);
        Assert.assertEquals(findLargestUniqueNumber(new int[]{1, 1, 2, 2, 3, 4}),
4);
        Assert.assertEquals(findLargestUniqueNumber(new int[]{4, 3, 3, 2, 2, 1}),
4);
        Assert.assertEquals(findLargestUniqueNumber(new int[]{3, 1, 4, 1, 5, 5}),
4);
        Assert.assertEquals(findLargestUniqueNumber(new int[]{}), -1);
        Assert.assertEquals(findLargestUniqueNumber(new int[]{99}), 99);

        // Test Approach 2 (General)
        Assert.assertEquals(findLargestUniqueNumberHashMap(new int[]{3, 1, 4, 1,
5, 5}), 4);
    }

    /**
     * Approach 1: Counting Array (Optimal for LeetCode Constraints)
     * Time Complexity: O(N + K) where N = nums.length and K = max value (1000) ->
O(N)
     * Space Complexity: O(K) where K = max value range -> O(1) auxiliary space
     */
    public static int findLargestUniqueNumber(int[] nums) {
        if (nums == null || nums.length == 0) {
            return -1;
        }

        int[] count = new int[1001];

        for (int num : nums)
            count[num]++;

        for (int num = count.length - 1; num >= 0; num--)
            if (count[num] == 1)
                return num;

        return -1;
    }

    /**
     * Approach 2: HashMap (General Solution for arbitrary/negative numbers)
     * Time Complexity: O(N)

```



```

    * Space Complexity: O(N)
    */
    public static int findLargestUniqueNumberHashMap(int[] nums) {
        if (nums == null || nums.length == 0) {
            return -1;
        }

        Map<Integer, Integer> counts = new HashMap<>();
        for (int num : nums) {
            counts.put(num, counts.getDefault(num, 0) + 1);
        }

        int maxUnique = -1;
        for (Map.Entry<Integer, Integer> entry : counts.entrySet()) {
            if (entry.getValue() == 1) {
                maxUnique = Math.max(maxUnique, entry.getKey());
            }
        }
        return maxUnique;
    }
}

```

MaxAverageSubarray.java

```

package programs.array;

import org.testng.Assert;

/**
 * LeetCode 643: Maximum Average Subarray I
 * Problem Statement:
 * Given an array of integers and a number k, find the contiguous subarray of
 * length k
 * that has the maximum average value and return that average.
 * Constraints:
 * - 1 <= k <= nums.length <= 10^5
 * - -10^4 <= nums[i] <= 10^4
 */
public class MaxAverageSubarray {

    public static void main(String[] args) {
        int[] arr = {1, 12, -5, -6, 50, 3};
        Assert.assertEquals(getMaxAverageSlidingWindow(arr, 4), 12.75);
    }

    /**
     * APPROACH : Sliding Window (Optimal)
     * Time Complexity: O(N)
     * - Single pass through the array after computing the initial window sum.
     * Space Complexity: O(1)
     */
}

```

```

    * - Uses constant extra space.
    */
    public static double getMaxAverageSlidingWindow(int[] arr, int k) {
        if (arr == null || arr.length < k) {
            throw new IllegalArgumentException("Invalid input");
        }
        double currentSum = 0;
        for (int i = 0; i < k; i++) {
            currentSum += arr[i];
        }
        double maxWindowSum = currentSum;
        for (int i = k; i < arr.length; i++) {
            currentSum = currentSum + arr[i] - arr[i - k];
            maxWindowSum = Math.max(maxWindowSum, currentSum);
        }
        return maxWindowSum / k;
    }
}

```

MaxSumSubArray.java

```

package programs.array;

import org.testng.Assert;

/**
 * LeetCode 53: Maximum Subarray
 * Problem Statement:
 * Given an array of integers, find the maximum sum of any contiguous subarray.
 * Constraints:
 * - 1 <= nums.length <= 10^5
 * - -10^4 <= nums[i] <= 10^4
 */
public class MaxSumSubArray {

    public static void main(String[] args) {
        int[] arr = {1, 12, -5, -6, 50, 3};
        Assert.assertEquals(maxSubArrayKadane(arr), 55);
    }

    /**
     * APPROACH : Kadane's Algorithm (Optimal)
     * Time Complexity: O(N)
     * - Single pass through the array.
     * Space Complexity: O(1)
     * - Uses constant extra space.
     */
    public static int maxSubArrayKadane(int[] nums) {

```

```

        if (nums == null || nums.length == 0) {
            throw new IllegalArgumentException("Array is empty");
        }
        int maxSum = Integer.MIN_VALUE;
        int currentSum = 0;
        for (int i : nums) {
            currentSum += i;
            maxSum = Math.max(maxSum, currentSum);
            if (currentSum < 0) {
                currentSum = 0;
            }
        }
        return maxSum;
    }
}

```

MergeSortedArrays.java

```

package programs.array;

import org.testng.Assert;

/**
 * LeetCode 88: Merge Sorted Array
 * Problem Statement:
 * You are given two integer arrays nums1 and nums2, sorted in non-decreasing
 * order,
 * and two integers m and n, representing the number of elements in nums1 and
 * nums2 respectively.
 * Merge nums1 and nums2 into a single array sorted in non-decreasing order.
 * Constraints:
 * - nums1.length == m + n
 * - nums2.length == n
 * - 0 <= m, n <= 200
 * - 1 <= m + n <= 200
 * - -10^9 <= nums1[i], nums2[j] <= 10^9
 */
public class MergeSortedArrays {

    public static void main(String[] args) {
        int[] nums1_1 = {1, 2, 3, 0, 0, 0};
        merge(nums1_1, 3, new int[]{2, 5, 6}, 3);
        Assert.assertEquals(nums1_1, new int[]{1, 2, 2, 3, 5, 6});

        int[] nums1_2 = {1};
        merge(nums1_2, 1, new int[] {}, 0);
        Assert.assertEquals(nums1_2, new int[]{1});

        int[] nums1_3 = {0};
        merge(nums1_3, 0, new int[]{1}, 1);
    }
}

```

```

        Assert.assertEquals(nums1_3, new int[]{1});
    }

    /**
     * APPROACH: Two Pointers (Backward Merging)
     * Time Complexity: O(M + N)
     * - We iterate backwards through both arrays at most M + N times.
     * Space Complexity: O(1)
     * - Merging is performed completely in-place within the extra space provided
     at the end of nums1.
     */
    public static void merge(int[] nums1, int m, int[] nums2, int n) {
        if (nums1 == null || nums2 == null) {
            return;
        }

        int i = m - 1;
        int j = n - 1;
        int k = m + n - 1;

        // Merge from back to front to avoid overwriting elements in nums1
        while (i >= 0 && j >= 0) {
            if (nums1[i] > nums2[j]) {
                nums1[k] = nums1[i];
                i--;
            } else {
                nums1[k] = nums2[j];
                j--;
            }
            k--;
        }

        // If any elements remain in nums2, copy them over as Remaining elements
        in nums1 are already in place
        while (j >= 0) {
            nums1[k] = nums2[j];
            j--;
            k--;
        }
    }
}

```

MinMaxArray.java

```

package programs.array;

import org.testng.Assert;

/**
 * LeetCode (Various): Find Minimum and Maximum in Array
 * <p>

```

```
* Problem Statement:
* Given an array of integers, find both the minimum and maximum elements in the
array.
* <p>
* Constraints:
* - 1 <= nums.length <= 10^5
* - -10^9 <= nums[i] <= 10^9
*/
public class MinMaxArray {

    public static void main(String[] args) {
        int[] arr = {12, 15, 7, 19, 3, 8};
        int[] result = findMinMaxOptimal(arr);
        Assert.assertEquals(result[0], 3);
        Assert.assertEquals(result[1], 19);

        result = findMinMaxLinear(arr);
        Assert.assertEquals(result[0], 3);
        Assert.assertEquals(result[1], 19);
    }

    /**
     * APPROACH 1: Optimal Comparison (Pair Comparison)
     * <p>
     * Time Complexity: O(N)
     * - Processes elements in pairs, reducing total comparisons from 2N to 1.5N.
     * <p>
     * Space Complexity: O(1)
     * - Uses constant extra space.
     */
    public static int[] findMinMaxOptimal(int[] arr) {
        if (arr == null || arr.length == 0) {
            throw new IllegalArgumentException("Array is empty");
        }
        int min, max;
        int i;
        int n = arr.length;

        if (n % 2 == 0) {
            if (arr[0] < arr[1]) {
                min = arr[0];
                max = arr[1];
            } else {
                min = arr[1];
                max = arr[0];
            }
            i = 2;
        } else {
            min = max = arr[0];
            i = 1;
        }

        while (i < n - 1) {
            int num1 = arr[i];
```

```

        int num2 = arr[i + 1];

        if (num1 < num2) {
            min = Math.min(min, num1);
            max = Math.max(max, num2);
        } else {
            min = Math.min(min, num2);
            max = Math.max(max, num1);
        }
        i += 2;
    }
    return new int[]{min, max};
}

/**
 * APPROACH 2: Linear Scan
 * <p>
 * Time Complexity: O(N)
 * - Single pass through the array.
 * <p>
 * Space Complexity: O(1)
 * - Uses constant extra space.
 */
public static int[] findMinMaxLinear(int[] arr) {
    if (arr == null || arr.length == 0) {
        throw new IllegalArgumentException("Array is empty");
    }
    int min = arr[0];
    int max = arr[0];
    for (int i = 1; i < arr.length; i++) {
        if (arr[i] < min) {
            min = arr[i];
        }
        if (arr[i] > max) {
            max = arr[i];
        }
    }
    return new int[]{min, max};
}
}

```

MissingNumber.java

```

package programs.array;

import org.testng.Assert;

/**
 * LeetCode 268: Missing Number
 */

```

```

* Problem Statement:
* Given an array nums containing n distinct numbers in the range [0, n],
* return the only number in the range that is missing from the array.
*
* Constraints:
* - n == nums.length
* - 1 <= n <= 10^4
* - 0 <= nums[i] <= n
* - All the numbers of nums are unique.
*/
public class MissingNumber {

    public static void main(String[] args) {
        Assert.assertEquals(missingNumberSum(new int[]{3, 0, 1}), 2);
        Assert.assertEquals(missingNumberXOR(new int[]{3, 0, 1}), 2);
        Assert.assertEquals(missingNumberBruteForce(new int[]{3, 0, 1}), 2);
    }

    /**
     * APPROACH 1: Mathematical Sum (Optimal)
     *
     * Time Complexity: O(N)
     * - Single pass to calculate array sum.
     *
     * Space Complexity: O(1)
     * - Uses constant extra space.
     */
    public static int missingNumberSum(int[] nums) {
        int len = nums.length;
        int sum = (len * (len + 1)) / 2;
        int arraySum = 0;
        for (int i : nums) {
            arraySum += i;
        }
        return sum - arraySum;
    }

    /**
     * APPROACH 2: XOR (Optimal, avoids overflow)
     *
     * Time Complexity: O(N)
     * - Single pass through the array.
     *
     * Space Complexity: O(1)
     * - Uses constant extra space.
     */
    public static int missingNumberXOR(int[] nums) {
        int n = nums.length;
        int xor = 0;
        for (int i = 0; i < n; i++) {
            xor ^= i ^ nums[i];
        }
        xor ^= n;
        return xor;
    }
}

```

```

    }

    /**
     * APPROACH 3: Brute Force
     *
     * Time Complexity: O(N^2)
     * - For each number in range, search in array.
     *
     * Space Complexity: O(1)
     * - Uses constant extra space.
     */
    public static int missingNumberBruteForce(int[] nums) {
        int n = nums.length;
        for (int i = 0; i <= n; i++) {
            boolean found = false;
            for (int num : nums) {
                if (num == i) {
                    found = true;
                    break;
                }
            }
            if (!found) {
                return i;
            }
        }
        return -1;
    }
}

```

MoveZeroes.java

```

package programs.array;

import org.testng.Assert;

/**
 * LeetCode 283: Move Zeroes
 * Problem Statement:
 * Given an integer array nums, move all 0's to the end of it while maintaining
 * the relative order of the non-zero elements. Do this in-place.
 * Constraints:
 * - 1 <= nums.length <= 10^4
 * - -2^31 <= nums[i] <= 2^31 - 1
 */
public class MoveZeroes {

    public static void main(String[] args) {
        int[] nums1 = {0, 1, 0, 3, 12};
        moveZeroesOptimal(nums1);
    }
}

```



```

        Assert.assertEquals(nums1, new int[]{1, 3, 12, 0, 0});
    }

    /**
     * APPROACH: Two Pointers (Optimal)
     * Time Complexity: O(N)
     * - Single pass through the array.
     * Space Complexity: O(1)
     * - Uses constant extra space.
     */
    public static void moveZeroesOptimal(int[] nums) {
        if (nums == null || nums.length == 0) {
            return;
        }
        int slow = 0;

        for (int fast = 0; fast < nums.length; fast++) {
            if (nums[fast] != 0) {
                if (fast != slow) {
                    int temp = nums[fast];
                    nums[fast] = nums[slow];
                    nums[slow] = temp;
                }
                slow++;
            }
        }
    }
}

```

NumbersStartingWithDigit.java

```

package programs.array;

import java.util.Arrays;

public class NumbersStartingWithDigit {
    // QUESTION: Given an array of integers, print all numbers that start with the
    // digit 1.
    // Example: Input: [1, 23, 123, 45, 134] -> Output: 1, 23, 123, 134
    public static void main(String[] args) {
        int[] arr = {1, 23, 123, 45, 134};
        Arrays.stream(arr).filter(num ->
String.valueOf(num).startsWith("1")).forEach(System.out::println);
    }
}

```

PairSumOfSortedArray.java

```
package programs.array;

import org.testng.Assert;

/**
 * LeetCode 167: Two Sum II - Input Array Is Sorted
 * Problem Statement:
 * Given a sorted array of integers and a target sum, find if there exists a pair
 of elements
 * in the array that sum to the target. Return the pair if found.
 * Constraints:
 * - 2 <= numbers.length <= 3 * 10^4
 * - -1000 <= numbers[i] <= 1000
 * - numbers is sorted in non-decreasing order.
 * - -1000 <= target <= 1000
 */
public class PairSumOfSortedArray {

    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 5, 6};
        Assert.assertEquals(targetSumTwoPointer(arr, 8), new int[]{2, 6});
        Assert.assertEquals(targetSumTwoPointer(arr, 5), new int[]{2, 3});
        Assert.assertEquals(targetSumBruteForce(arr, 8), new int[]{2, 6});
    }

    /**
     * APPROACH 1: Two Pointers (Optimal for sorted arrays)
     * Time Complexity: O(N)
     * - Single pass from both ends.
     * Space Complexity: O(1)
     * - Uses constant extra space.
     */
    public static int[] targetSumTwoPointer(int[] arr, int targetSum) {
        if (arr == null || arr.length < 2) {
            return new int[]{-1, -1};
        }
        int i = 0, j = arr.length - 1;
        while (i < j) {
            int sum = arr[i] + arr[j];
            if (sum == targetSum) {
                return new int[]{arr[i], arr[j]};
            } else if (sum < targetSum) {
                i++;
            } else {
                j--;
            }
        }
        return new int[]{-1, -1};
    }
}
```

```

/**
 * APPROACH 2: Brute Force
 * Time Complexity: O(N^2)
 * - Checks every pair of elements.
 * Space Complexity: O(1)
 * - Uses constant extra space.
 */
public static int[] targetSumBruteForce(int[] arr, int targetSum) {
    if (arr == null || arr.length < 2) {
        return new int[]{-1, -1};
    }
    for (int i = 0; i < arr.length; i++) {
        for (int j = i + 1; j < arr.length; j++) {
            if (arr[i] + arr[j] == targetSum) {
                return new int[]{arr[i], arr[j]};
            }
        }
    }
    return new int[]{-1, -1};
}

```

ProductArray.java

```

package programs.array;

import org.testng.Assert;

import java.util.Arrays;

/**
 * LeetCode 238: Product of Array Except Self
 *
 * Problem Statement:
 * Given an integer array nums, return an array answer such that answer[i] is
 * equal to the
 * product of all the elements of nums except nums[i]. You must write an algorithm
 * that runs in O(n)
 * time and without using the division operation.
 *
 * Constraints:
 * - 2 <= nums.length <= 10^5
 * - -30 <= nums[i] <= 30
 * - The product of any prefix or suffix of nums is guaranteed to fit in a 32-bit
 * integer.
 */
public class ProductArray {

    public static void main(String[] args) {

```

```
        int[] nums = {1, 2, 3, 4};
        Assert.assertEquals(productExceptSelfOptimal(nums), new int[]{24, 12, 8,
6});
        Assert.assertEquals(productExceptSelfBruteForce(nums), new int[]{24, 12,
8, 6});
    }

    /**
     * APPROACH 1: Prefix and Suffix Products (Optimal)
     *
     * Time Complexity: O(N)
     * - Two passes through the array.
     *
     * Space Complexity: O(1) (excluding output array)
     * - Uses constant extra space (output array doesn't count).
     */
    public static int[] productExceptSelfOptimal(int[] nums) {
        int n = nums.length;
        int[] result = new int[n];
        result[0] = 1;
        for (int i = 1; i < n; i++) {
            result[i] = result[i - 1] * nums[i - 1];
        }
        int rightProduct = 1;
        for (int i = n - 1; i >= 0; i--) {
            result[i] *= rightProduct;
            rightProduct *= nums[i];
        }
        return result;
    }

    /**
     * APPROACH 2: Brute Force
     *
     * Time Complexity: O(N^2)
     * - For each element, calculate product of all other elements.
     *
     * Space Complexity: O(1) (excluding output array)
     * - Uses constant extra space.
     */
    public static int[] productExceptSelfBruteForce(int[] nums) {
        int n = nums.length;
        int[] result = new int[n];
        for (int i = 0; i < n; i++) {
            int product = 1;
            for (int j = 0; j < n; j++) {
                if (i != j) {
                    product *= nums[j];
                }
            }
            result[i] = product;
        }
        return result;
    }
}
```

```
}
```

RangeSumQuery.java

```
package programs.array;

import org.testng.Assert;
import org.testng.annotations.Test;

public class RangeSumQuery {
    //Given an integer array nums, handle multiple queries of the following type:
    //Calculate the sum of the elements of nums between indices left and right
    inclusive where left <= right.

    /*
    Time Complexity: O(1)
    Space Complexity: O(1)
    */

    static class NumArray {
        private final int[] prefixSums;

        public NumArray(int[] nums) {
            prefixSums = new int[nums.length + 1];
            for (int i = 0; i < nums.length; i++) {
                prefixSums[i + 1] = prefixSums[i] + nums[i];
            }
        }

        public int sumRange(int left, int right) {
            return prefixSums[right + 1] - prefixSums[left];
        }
    }

    @Test
    public void testStandardRange() {
        int[] nums = {-2, 0, 3, -5, 2, -1};
        RangeSumQuery.NumArray numArray = new NumArray(nums);

        // Sum of indices 0 to 2: (-2 + 0 + 3) = 1
        Assert.assertEquals(numArray.sumRange(0, 2), 1, "Sum of range [0, 2]
failed");

        // Sum of indices 2 to 5: (3 + -5 + 2 + -1) = -1
        Assert.assertEquals(numArray.sumRange(2, 5), -1, "Sum of range [2, 5]
failed");

        // Sum of indices 0 to 5 (Full Array): -3
        Assert.assertEquals(numArray.sumRange(0, 5), -3, "Sum of full range
```

```

failed");
    }

    @Test
    public void testSingleElementRange() {
        int[] nums = {1, 10, 100};
        RangeSumQuery.NumArray numArray = new NumArray(nums);

        // If left == right, it should return just that element
        Assert.assertEquals(numArray.sumRange(1, 1), 10, "Single element range [1, 1] failed");
        Assert.assertEquals(numArray.sumRange(0, 0), 1, "Single element range [0, 0] failed");
    }

    @Test
    public void testLargeValues() {
        int[] nums = {1000, -1000, 1000};
        RangeSumQuery.NumArray numArray = new NumArray(nums);

        // Sum of [0, 1] should be 0
        Assert.assertEquals(numArray.sumRange(0, 1), 0, "Sum of cancelling values failed");
    }
}

```

RemoveDupeCharFromList.java

```

package programs.array;
import java.util.Arrays;

// QUESTION: Given an array of strings, remove duplicate characters from each string while
// preserving the order of first occurrence.
// Example: Input: ["hello", "helloishi", "mastercard"] -> Output: ["helo", "helois", "mstercd"]
public class RemoveDupeCharFromList {
    public static void main(String[] args) {
        String[] words = {"hello", "helloishi", "mastercard"};
        String[] result = new String[words.length];

        for (int i = 0; i < words.length; i++) {
            result[i] = removeDuplicates(words[i]);
        }

        System.out.println(Arrays.toString(result));
    }

    private static String removeDuplicates(String word) {

```

```

        StringBuilder sb = new StringBuilder();
        for (char c : word.toCharArray()) {
            if (sb.indexOf(String.valueOf(c)) == -1) {
                sb.append(c);
            }
        }
        return sb.toString();
    }
}

```

RemoveDuplicates.java

```

package programs.array;

import org.testng.Assert;

import java.util.Arrays;

/**
 * LeetCode 26: Remove Duplicates from Sorted Array
 * Problem Statement:
 * Given an integer array nums sorted in non-decreasing order, remove the
 * duplicates in-place
 * such that each unique element appears only once. The relative order of the
 * elements should be kept the same.
 * Then return the number of unique elements in nums. (Modified here to return the
 * sliced array of unique elements).
 * Constraints:
 * - 1 <= nums.length <= 3 * 10^4
 * - -100 <= nums[i] <= 100
 * - nums is sorted in non-decreasing order.
 */
public class RemoveDuplicates {

    public static void main(String[] args) {
        int[] arr = {1, 1, 2, 3, 4, 4, 4, 5};
        Assert.assertEquals(removeDuplicates(arr), new int[]{1, 2, 3, 4, 5});
    }

    /**
     * APPROACH: Two Pointers (In-Place Deduplication)
     * Time Complexity: O(N)
     * - Iterating through the array takes linear time with respect to the number
     of elements N.
     * Space Complexity: O(K) where K is the number of unique elements
     * - Auxiliary space is O(1) for the in-place modification, though
     Arrays.copyOfRange
     *   allocates a new array of size K for the return value.
     */
}

```

```

    public static int[] removeDuplicates(int[] arr) {
        if (arr == null || arr.length == 0) {
            return new int[0];
        }

        // Note: The problem states the array is already sorted, but keeping
        Arrays.sort()
        // ensures safety if unsorted inputs are passed.
        Arrays.sort(arr);

        int j = 0; // Pointer for the last unique element found

        for (int i = 1; i < arr.length; i++) {
            if (arr[i] != arr[j]) { // Shifts unique elements forward
                j++;
                arr[j] = arr[i];
            }
        }

        return Arrays.copyOfRange(arr, 0, j + 1);
    }
}

```

RemoveElement.java

```

package programs.array;

import org.testng.Assert;

import java.util.Arrays;

/**
 * LeetCode 27: Remove Element
 *
 * Problem Statement:
 * Given an integer array nums and an integer val, remove all occurrences of val
 * in nums in-place.
 * The order of the elements may be changed. Then return the number of elements in
 * nums which are not equal to val.
 *
 * Constraints:
 * - 0 <= nums.length <= 100
 * - 0 <= nums[i] <= 50
 * - 0 <= val <= 100
 */
public class RemoveElement {

    public static void main(String[] args) {
        int[] nums1 = {3, 2, 2, 3};
        Assert.assertEquals(removeElementOptimal(nums1, 3), 2);
    }
}

```



```
        int[] nums2 = {3, 2, 2, 3};
        Assert.assertEquals(removeElementBruteForce(nums2, 3), 2);
    }

    /**
     * APPROACH 1: Two Pointers (Optimal)
     *
     * Time Complexity: O(N)
     * - Single pass through the array.
     *
     * Space Complexity: O(1)
     * - Uses constant extra space.
     */
    public static int removeElementOptimal(int[] nums, int val) {
        if (nums == null || nums.length == 0) {
            return 0;
        }
        int j = 0;
        for (int i = 0; i < nums.length; i++) {
            if (nums[i] != val) {
                nums[j] = nums[i];
                j++;
            }
        }
        return j;
    }

    /**
     * APPROACH 2: Brute Force (Shift elements)
     *
     * Time Complexity: O(N^2)
     * - For each occurrence of val, shift all subsequent elements.
     *
     * Space Complexity: O(1)
     * - Uses constant extra space.
     */
    public static int removeElementBruteForce(int[] nums, int val) {
        if (nums == null || nums.length == 0) {
            return 0;
        }
        int n = nums.length;
        int i = 0;
        while (i < n) {
            if (nums[i] == val) {
                for (int j = i; j < n - 1; j++) {
                    nums[j] = nums[j + 1];
                }
                n--;
            } else {
                i++;
            }
        }
        return n;
    }
}
```

```
}
```

ReverseArray.java

```
package programs.array;

import org.testng.Assert;

import java.util.Arrays;

/**
 * LeetCode (Various): Reverse Array
 *
 * Problem Statement:
 * Given an array of integers, reverse the array in-place.
 *
 * Constraints:
 * - 1 <= nums.length <= 10^5
 * - -10^9 <= nums[i] <= 10^9
 */
public class ReverseArray {

    public static void main(String[] args) {
        int[] array1 = {1, 2, 3, 4, 5, 6};
        reverseInPlace(array1);
        Assert.assertEquals(array1, new int[]{6, 5, 4, 3, 2, 1});

        int[] array2 = {1, 2, 3, 4, 5, 6};
        reverseBruteForce(array2);
        Assert.assertEquals(array2, new int[]{6, 5, 4, 3, 2, 1});
    }

    /**
     * APPROACH 1: Two Pointers (Optimal)
     *
     * Time Complexity: O(N)
     * - Single pass from both ends.
     *
     * Space Complexity: O(1)
     * - Uses constant extra space.
     */
    public static void reverseInPlace(int[] array) {
        if (array == null || array.length <= 1) {
            return;
        }
        int start = 0, end = array.length - 1;
        while (start < end) {
            int temp = array[start];
            array[start] = array[end];
            array[end] = temp;
            start++;
            end--;
        }
    }
}
```

```

        array[end] = temp;
        start++;
        end--;
    }
}

/**
 * APPROACH 2: Brute Force (Using extra array)
 *
 * Time Complexity: O(N)
 * - Single pass to copy elements.
 *
 * Space Complexity: O(N)
 * - Uses extra array for storage.
 */
public static void reverseBruteForce(int[] array) {
    if (array == null || array.length <= 1) {
        return;
    }
    int n = array.length;
    int[] reversed = new int[n];
    for (int i = 0; i < n; i++) {
        reversed[i] = array[n - 1 - i];
    }
    for (int i = 0; i < n; i++) {
        array[i] = reversed[i];
    }
}
}

```

SearchInsertPositionTest.java

```

package programs.array;

import org.testng.annotations.Test;

import static org.testng.Assert.assertEquals;

/**
 * LeetCode 35: Search Insert Position
 *
 * Problem Statement:
 * Given a sorted array of distinct integers and a target value, return the index
 * if the target is found.
 * If not, return the index where it would be if it were inserted in order.
 *
 * Constraints:
 * - 1 <= nums.length <= 10^4
 * - -10^4 <= nums[i] <= 10^4

```

```
* - nums contains distinct values sorted in ascending order.
* -  $-10^4 \leq \text{target} \leq 10^4$ 
*/
public class SearchInsertPositionTest {

    /**
     * APPROACH 1: Linear Search
     *
     * Time Complexity:  $O(N)$ 
     * - Iterates through the array elements sequentially.
     *
     * Space Complexity:  $O(1)$ 
     * - Uses constant extra space.
     */
    public int searchInsert(int[] nums, int target) {
        if (nums == null) {
            return 0;
        }
        int len = nums.length;
        for (int i = 0; i < len; i++) {
            if (nums[i] >= target) {
                return i;
            }
        }
        return len;
    }

    /**
     * APPROACH 2: Binary Search (Optimal Time)
     *
     * Time Complexity:  $O(\log N)$ 
     * - Halves the search space in each iteration.
     *
     * Space Complexity:  $O(1)$ 
     * - Uses constant extra space.
     */
    public int searchInsertBinary(int[] nums, int target) {
        if (nums == null) {
            return 0;
        }
        int left = 0;
        int right = nums.length;
        while (left < right) {
            int mid = left + (right - left) / 2;
            if (nums[mid] >= target) {
                right = mid;
            } else {
                left = mid + 1;
            }
        }
        return left;
    }
}
```

@Test

```

public void testSearchInsertAllCases() {
    // Example 1
    assertEquals(searchInsert(new int[]{1, 3, 5, 6}, 5), 2);
    assertEquals(searchInsertBinary(new int[]{1, 3, 5, 6}, 5), 2);

    // Example 2
    assertEquals(searchInsert(new int[]{1, 3, 5, 6}, 2), 1);
    assertEquals(searchInsertBinary(new int[]{1, 3, 5, 6}, 2), 1);

    // Example 3
    assertEquals(searchInsert(new int[]{1, 3, 5, 6}, 7), 4);
    assertEquals(searchInsertBinary(new int[]{1, 3, 5, 6}, 7), 4);

    // Example 4
    assertEquals(searchInsert(new int[]{1, 3, 5, 6}, 0), 0);
    assertEquals(searchInsertBinary(new int[]{1, 3, 5, 6}, 0), 0);

    // Example 5
    assertEquals(searchInsert(new int[]{1}, 0), 0);
    assertEquals(searchInsertBinary(new int[]{1}, 0), 0);

    // Example 6
    assertEquals(searchInsert(new int[]{1}, 2), 1);
    assertEquals(searchInsertBinary(new int[]{1}, 2), 1);

    // Example 7
    assertEquals(searchInsert(new int[]{1, 2, 3, 4, 5}, 3), 2);
    assertEquals(searchInsertBinary(new int[]{1, 2, 3, 4, 5}, 3), 2);

    // Example 8
    assertEquals(searchInsert(new int[]{1, 2, 3, 4, 5}, 6), 5);
    assertEquals(searchInsertBinary(new int[]{1, 2, 3, 4, 5}, 6), 5);

    // Example 9
    assertEquals(searchInsert(new int[]{1, 2, 3, 4, 5}, 0), 0);
    assertEquals(searchInsertBinary(new int[]{1, 2, 3, 4, 5}, 0), 0);
}
}

```

SecondSmallestLargest.java

```

package programs.array;

import org.testng.Assert;

import java.util.Arrays;

/**
 * LeetCode (Various): Find Second Smallest and Second Largest
 *
 * Problem Statement:

```

```

    * Given an array of integers, find the second smallest and/or second largest
    elements.
    *
    * Constraints:
    * - 1 <= nums.length <= 10^5
    * - -10^9 <= nums[i] <= 10^9
    */
    public class SecondSmallestLargest {

        public static void main(String[] args) {
            int[] arr = {1, 8, 5, 3, 20, 4, 5, 8, 6, 10};

            Assert.assertEquals(findSecondLargestOptimal(arr), 10);
            Assert.assertEquals(findSecondSmallestOptimal(arr), 3);

            Assert.assertEquals(findSecondLargestSort(arr), 10);
            Assert.assertEquals(findSecondSmallestSort(arr), 3);

            Assert.assertEquals(findSecondLargestBruteForce(arr), 10);
            Assert.assertEquals(findSecondSmallestBruteForce(arr), 3);

            // Test with array where all elements are same
            int[] sameArr = {5, 5, 5, 5};
            Assert.assertEquals(findSecondLargestOptimal(sameArr), -1);
        }

        /**
         * APPROACH 1: Single Pass (Optimal)
         *
         * Time Complexity: O(N)
         * - Single pass through the array.
         *
         * Space Complexity: O(1)
         * - Uses constant extra space.
         */
        public static int findSecondLargestOptimal(int[] array) {
            if (array == null || array.length < 2) {
                return -1;
            }

            int firstLargest = Integer.MIN_VALUE;
            int secondLargest = Integer.MIN_VALUE;

            for (int element : array) {
                if (element > firstLargest) {
                    secondLargest = firstLargest;
                    firstLargest = element;
                } else if (element > secondLargest && element != firstLargest) {
                    secondLargest = element;
                }
            }

            return (secondLargest == Integer.MIN_VALUE) ? -1 : secondLargest;
        }
    }

```

```
public static int findSecondSmallestOptimal(int[] arr) {
    if (arr == null || arr.length < 2) {
        return Integer.MAX_VALUE;
    }

    int smallest = Integer.MAX_VALUE;
    int secondSmallest = Integer.MAX_VALUE;

    for (int j : arr) {
        if (j < smallest) {
            secondSmallest = smallest;
            smallest = j;
        } else if (j < secondSmallest && j != smallest) {
            secondSmallest = j;
        }
    }

    return secondSmallest;
}

/**
 * APPROACH 2: Sorting
 *
 * Time Complexity: O(N log N)
 * - Sorting takes O(N log N) time.
 *
 * Space Complexity: O(log N)
 * - Arrays.sort() uses O(log N) auxiliary space.
 */
public static int findSecondLargestSort(int[] array) {
    if (array == null || array.length < 2) {
        return -1;
    }
    int[] sorted = array.clone();
    Arrays.sort(sorted);
    int n = sorted.length;
    for (int i = n - 2; i >= 0; i--) {
        if (sorted[i] != sorted[n - 1]) {
            return sorted[i];
        }
    }
    return -1;
}

public static int findSecondSmallestSort(int[] arr) {
    if (arr == null || arr.length < 2) {
        return Integer.MAX_VALUE;
    }
    int[] sorted = arr.clone();
    Arrays.sort(sorted);
    for (int i = 1; i < sorted.length; i++) {
        if (sorted[i] != sorted[0]) {
            return sorted[i];
        }
    }
}
```

```
    }
    }
    return Integer.MAX_VALUE;
}

/**
 * APPROACH 3: Brute Force
 *
 * Time Complexity: O(N^2)
 * - For each element, compare with all others.
 *
 * Space Complexity: O(1)
 * - Uses constant extra space.
 */
public static int findSecondLargestBruteForce(int[] array) {
    if (array == null || array.length < 2) {
        return -1;
    }
    int firstLargest = Integer.MIN_VALUE;
    int secondLargest = Integer.MIN_VALUE;

    for (int i = 0; i < array.length; i++) {
        boolean isLargest = true;
        for (int j = 0; j < array.length; j++) {
            if (array[j] > array[i]) {
                isLargest = false;
                break;
            }
        }
        if (isLargest) {
            firstLargest = array[i];
        }
    }

    for (int i = 0; i < array.length; i++) {
        if (array[i] != firstLargest) {
            boolean isSecondLargest = true;
            for (int j = 0; j < array.length; j++) {
                if (array[j] > array[i] && array[j] != firstLargest) {
                    isSecondLargest = false;
                    break;
                }
            }
            if (isSecondLargest) {
                secondLargest = array[i];
            }
        }
    }

    return (secondLargest == Integer.MIN_VALUE) ? -1 : secondLargest;
}

public static int findSecondSmallestBruteForce(int[] arr) {
    if (arr == null || arr.length < 2) {
```



```

        return Integer.MAX_VALUE;
    }
    int smallest = Integer.MAX_VALUE;
    int secondSmallest = Integer.MAX_VALUE;

    for (int i = 0; i < arr.length; i++) {
        boolean isSmallest = true;
        for (int j = 0; j < arr.length; j++) {
            if (arr[j] < arr[i]) {
                isSmallest = false;
                break;
            }
        }
        if (isSmallest) {
            smallest = arr[i];
        }
    }

    for (int i = 0; i < arr.length; i++) {
        if (arr[i] != smallest) {
            boolean isSecondSmallest = true;
            for (int j = 0; j < arr.length; j++) {
                if (arr[j] < arr[i] && arr[j] != smallest) {
                    isSecondSmallest = false;
                    break;
                }
            }
            if (isSecondSmallest) {
                secondSmallest = arr[i];
            }
        }
    }

    return secondSmallest;
}

```

SeparateEvenOdd.java

```

package programs.array;

import org.testng.Assert;

import java.util.Arrays;

/**
 * LeetCode (Various): Separate Even and Odd Numbers
 *
 * Problem Statement:

```

```
* Given an array of integers, move all even numbers to the left side and odd
numbers to the right side.
*
* Constraints:
* - 1 <= nums.length <= 10^5
* - -10^9 <= nums[i] <= 10^9
*/
public class SeparateEvenOdd {

    public static void main(String[] args) {
        int[] arr1 = {1, 2, 3, 4, 5, 6, 7, 8, 9};
        separateEvenOddOptimal(arr1);
        Assert.assertEquals(arr1, new int[]{2, 4, 6, 8, 1, 3, 5, 7, 9});

        int[] arr2 = {1, 2, 3, 4, 5, 6, 7, 8, 9};
        separateEvenOddBruteForce(arr2);
    }

    /**
     * APPROACH 1: Two Pointers (Optimal)
     *
     * Time Complexity: O(N)
     * - Single pass through the array.
     *
     * Space Complexity: O(1)
     * - Uses constant extra space.
     */
    public static void separateEvenOddOptimal(int[] nums) {
        if (nums == null || nums.length <= 1) {
            return;
        }
        int j = 0;
        for (int i = 0; i < nums.length; i++) {
            if (nums[i] % 2 == 0) {
                int temp = nums[i];
                nums[i] = nums[j];
                nums[j] = temp;
                j++;
            }
        }
    }

    /**
     * APPROACH 2: Brute Force (Using extra array)
     *
     * Time Complexity: O(N)
     * - Two passes through the array.
     *
     * Space Complexity: O(N)
     * - Uses extra array for storage.
     */
    public static void separateEvenOddBruteForce(int[] nums) {
        if (nums == null || nums.length <= 1) {
            return;
        }
    }
}
```

```

    }
    int[] result = new int[nums.length];
    int index = 0;
    for (int num : nums) {
        if (num % 2 == 0) {
            result[index++] = num;
        }
    }
    for (int num : nums) {
        if (num % 2 != 0) {
            result[index++] = num;
        }
    }
    for (int i = 0; i < nums.length; i++) {
        nums[i] = result[i];
    }
}
}

```

SortedSquares.java

```

package programs.array;

import org.testng.Assert;

import java.util.Arrays;

/**
 * LeetCode 977: Squares of a Sorted Array
 *
 * Problem Statement:
 * Given a sorted array of integers (may contain negative numbers), return an
 * array
 * of the squares of each number sorted in non-decreasing order.
 *
 * Constraints:
 * - 1 <= nums.length <= 10^4
 * - -10^4 <= nums[i] <= 10^4
 * - nums is sorted in non-decreasing order.
 */
public class SortedSquares {

    public static void main(String[] args) {
        int[] nums = {-7, -3, 2, 3, 11};
        Assert.assertEquals(sortedSquaresTwoPointer(nums), new int[]{4, 9, 9, 49,
121});
        Assert.assertEquals(sortedSquaresBruteForce(nums), new int[]{4, 9, 9, 49,
121});
    }
}

```

```
/**
 * APPROACH 1: Two Pointers (Optimal)
 *
 * Time Complexity: O(N)
 * - Single pass from both ends.
 *
 * Space Complexity: O(N)
 * - Uses extra array for result.
 */
public static int[] sortedSquaresTwoPointer(int[] nums) {
    int n = nums.length;
    int[] result = new int[n];
    int left = 0;
    int right = n - 1;
    int index = n - 1;

    while (left <= right) {
        int leftSquare = nums[left] * nums[left];
        int rightSquare = nums[right] * nums[right];
        if (leftSquare > rightSquare) {
            result[index--] = leftSquare;
            left++;
        } else {
            result[index--] = rightSquare;
            right--;
        }
    }
    return result;
}

/**
 * APPROACH 2: Brute Force (Square and Sort)
 *
 * Time Complexity: O(N log N)
 * - Squaring takes O(N), sorting takes O(N log N).
 *
 * Space Complexity: O(N)
 * - Uses extra array for result.
 */
public static int[] sortedSquaresBruteForce(int[] nums) {
    int n = nums.length;
    int[] result = new int[n];
    for (int i = 0; i < n; i++) {
        result[i] = nums[i] * nums[i];
    }
    Arrays.sort(result);
    return result;
}
}
```

StockBuySell.java

```
package programs.array;

import org.testng.Assert;

/**
 * LeetCode 121: Best Time to Buy and Sell Stock
 *
 * Problem Statement:
 * Given an array of stock prices where prices[i] is the price on day i, find the
 * maximum profit you can achieve by buying on one day and selling on a different
 * day.
 *
 * Constraints:
 * - 1 <= prices.length <= 10^5
 * - 0 <= prices[i] <= 10^4
 */
public class StockBuySell {

    public static void main(String[] args) {
        int[] prices = {7, 1, 5, 3, 6, 4};
        Assert.assertEquals(maxProfitOptimal(prices), 5);
        Assert.assertEquals(maxProfitBruteForce(prices), 5);
    }

    /**
     * APPROACH 1: Single Pass (Optimal)
     *
     * Time Complexity: O(N)
     * - Single pass through the array.
     *
     * Space Complexity: O(1)
     * - Uses constant extra space.
     */
    public static int maxProfitOptimal(int[] prices) {
        if (prices == null || prices.length < 2) {
            return 0;
        }
        int buyPrice = Integer.MAX_VALUE;
        int maxProfit = 0;
        for (int price : prices) {
            if (price > buyPrice) {
                int profit = price - buyPrice;
                maxProfit = Math.max(maxProfit, profit);
            } else {
                buyPrice = price;
            }
        }
        return maxProfit;
    }
}
```

```

/**
 * APPROACH 2: Brute Force
 *
 * Time Complexity: O(N^2)
 * - For each day, check all future days for selling.
 *
 * Space Complexity: O(1)
 * - Uses constant extra space.
 */
public static int maxProfitBruteForce(int[] prices) {
    if (prices == null || prices.length < 2) {
        return 0;
    }
    int maxProfit = 0;
    for (int i = 0; i < prices.length; i++) {
        for (int j = i + 1; j < prices.length; j++) {
            int profit = prices[j] - prices[i];
            maxProfit = Math.max(maxProfit, profit);
        }
    }
    return maxProfit;
}

```

TrappedRainWater.java

```

package programs.array;

import org.testng.Assert;

/**
 * LeetCode 42: Trapping Rain Water
 *
 * Problem Statement:
 * Given an array of non-negative integers representing the height of bars,
 * compute how
 * much water it can trap after raining.
 *
 * Constraints:
 * - n == height.length
 * - 1 <= n <= 2 * 10^4
 * - 0 <= height[i] <= 10^5
 */
public class TrappedRainWater {

    public static void main(String[] args) {
        int[] height = new int[]{4, 2, 0, 6, 3, 2, 5};
        Assert.assertEquals(trapRainWaterDP(height), 9);
        Assert.assertEquals(trapRainWaterTwoPointer(height), 9);
    }
}

```

```
        Assert.assertEquals(trapRainWaterBruteForce(height), 9);
    }

    /**
     * APPROACH 1: Dynamic Programming (Prefix/Suffix Arrays)
     *
     * Time Complexity: O(N)
     * - Three passes through the array.
     *
     * Space Complexity: O(N)
     * - Uses two auxiliary arrays for left and right max boundaries.
     */
    public static int trapRainWaterDP(int[] height) {
        if (height == null || height.length < 3) {
            return 0;
        }
        int length = height.length;
        int[] leftMaxBoundary = new int[length];
        int[] rightMaxBoundary = new int[length];

        leftMaxBoundary[0] = height[0];
        for (int i = 1; i < length; i++) {
            leftMaxBoundary[i] = Math.max(leftMaxBoundary[i - 1], height[i]);
        }

        rightMaxBoundary[length - 1] = height[length - 1];
        for (int i = length - 2; i >= 0; i--) {
            rightMaxBoundary[i] = Math.max(height[i], rightMaxBoundary[i + 1]);
        }

        int result = 0;
        for (int i = 0; i < length; i++) {
            int waterLevel = Math.min(leftMaxBoundary[i], rightMaxBoundary[i]);
            result += waterLevel - height[i];
        }

        return result;
    }

    /**
     * APPROACH 2: Two Pointers (Optimal Space)
     *
     * Time Complexity: O(N)
     * - Single pass through the array.
     *
     * Space Complexity: O(1)
     * - Uses constant extra space.
     */
    public static int trapRainWaterTwoPointer(int[] height) {
        if (height == null || height.length < 3) {
            return 0;
        }
        int left = 0, right = height.length - 1;
        int leftMax = 0, rightMax = 0;
        int water = 0;
```

```

        while (left < right) {
            if (height[left] < height[right]) {
                if (height[left] >= leftMax) {
                    leftMax = height[left];
                } else {
                    water += leftMax - height[left];
                }
                left++;
            } else {
                if (height[right] >= rightMax) {
                    rightMax = height[right];
                } else {
                    water += rightMax - height[right];
                }
                right--;
            }
        }
        return water;
    }

/**
 * APPROACH 3: Brute Force
 *
 * Time Complexity: O(N^2)
 * - For each element, find max on left and right.
 *
 * Space Complexity: O(1)
 * - Uses constant extra space.
 */
public static int trapRainWaterBruteForce(int[] height) {
    if (height == null || height.length < 3) {
        return 0;
    }
    int water = 0;
    for (int i = 0; i < height.length; i++) {
        int leftMax = 0, rightMax = 0;
        for (int j = 0; j <= i; j++) {
            leftMax = Math.max(leftMax, height[j]);
        }
        for (int j = i; j < height.length; j++) {
            rightMax = Math.max(rightMax, height[j]);
        }
        water += Math.min(leftMax, rightMax) - height[i];
    }
    return water;
}
}

```



```
package programs.array;

import java.util.Arrays;
import java.util.HashMap;
import java.util.Map;

// QUESTION: Given an array of integers, find two numbers such that they add up to
// a specific target.
// Example: Input: [2, 7, 11, 15], target = 9 -> Output: [0, 1] (indices)
public class TwoSum {

    public static void main(String[] args) {
        int[] nums = {2, 7, 11, 15};
        int target = 9;

        // Approach 1: HashMap (for unsorted arrays)
        System.out.println("HashMap Approach: " +
            Arrays.toString(twoSumHashMap(nums, target)));

        // Approach 2: Two pointers (for sorted arrays)
        int[] sortedNums = {2, 7, 11, 15};
        System.out.println("Two Pointer Approach: " +
            Arrays.toString(twoSumTwoPointer(sortedNums, target)));
    }

    /**
     * Approach 1: Using HashMap
     * Time Complexity: O(n)
     * Space Complexity: O(n)
     * Use when array is NOT sorted
     */
    public static int[] twoSumHashMap(int[] nums, int target) {
        Map<Integer, Integer> hs = new HashMap<>();
        int[] ans = new int[2];
        for (int i = 0; i < nums.length; i++) {
            int counter = target - nums[i];
            if (hs.containsKey(counter)) {
                ans[0] = i;
                ans[1] = hs.get(counter);
                break;
            } else {
                hs.put(nums[i], i);
            }
        }
        return ans;
    }

    /**
     * Approach 2: Two Pointers
     * Time Complexity: O(n)
     * Space Complexity: O(1)
     * Use when array is ALREADY sorted
     */
}
```

```
    * Returns 1-indexed positions as per LeetCode convention
    */
    public static int[] twoSumTwoPointer(int[] numbers, int target) {
        int left = 0, right = numbers.length - 1;

        while (left < right) {
            int sum = numbers[left] + numbers[right];

            if (sum == target) {
                return new int[]{left + 1, right + 1};
            } else if (sum > target) {
                right--;
            } else {
                left++;
            }
        }
        return new int[]{-1, -1};
    }
}
```

ZeroSumSubarrays.java

```
package programs.array;

import java.util.*;

/**
 * Problem Statement:
 * Given an array of integers, find all subarrays that sum to zero. Print each
 * subarray.
 *
 * Constraints:
 * - Array elements can be positive, negative, or zero.
 */
public class ZeroSumSubarrays {

    /**
     * APPROACH: Prefix Sum with Hash Map
     *
     * Time Complexity:  $O(N^2)$  in the worst case (if there are many overlapping
     * zero-sum subarrays),
     * or  $O(N + K)$  on average where K is the total number of zero-sum subarrays
     * found.
     *
     * Space Complexity:  $O(N)$ 
     * - The hash map stores prefix sum keys and lists of indices, taking linear
     * space.
     */
    public static void printZeroSumSubarrays(int[] arr) {
```

```
if (arr == null || arr.length == 0) {
    return;
}

Map<Integer, List<Integer>> map = new HashMap<>();
// To handle cases where a subarray starts from index 0
map.put(0, new ArrayList<>(List.of(-1)));

int sum = 0;
for (int i = 0; i < arr.length; i++) {
    sum += arr[i];

    // If the prefix sum has been seen before, it means the elements
    // between the previous index and current index 'i' sum to zero.
    if (map.containsKey(sum)) {
        List<Integer> indices = map.get(sum);
        for (int startIdx : indices) {
            System.out.println("Subarray found: " +
                Arrays.toString(Arrays.copyOfRange(arr, startIdx + 1,
i + 1)));
        }
    }

    map.putIfAbsent(sum, new ArrayList<>());
    map.get(sum).add(i);
}

public static void main(String[] args) {
    int[] arr = {6, 3, -1, -3, 4, -2, 2, 4, 6, -12, -7};
    System.out.println("Zero-sum subarrays:");
    printZeroSumSubarrays(arr);
}
```

Topic: dp

Notes & Concept Guide

Dynamic Programming Problems - Interview Prep Guide

Overview

This folder contains **8 problems** focused on dynamic programming techniques, covering classic DP patterns and optimizations.

Problem Categories

1. Classic DP Patterns (3 problems)

- `ClimbingStairs.java` - Fibonacci-like pattern
- `HouseRobber.java` - Rob houses without robbing adjacent ones
- `HouseRobberII.java` - Circular arrangement variant

Technique: 1D DP with state variables **Time Complexity:** $O(n)$, **Space Complexity:** $O(1)$ optimized

2. Knapsack Variants (2 problems)

- `CoinChange.java` - Minimum coins for amount (unbounded knapsack)
- `UniquePaths.java` - Count paths in grid (combinatorics)

Applications: Resource allocation, path counting **Interview Frequency:** Very Common

3. Sequence Problems (2 problems)

- `LongestIncreasingSubsequence.java` - Find length of LIS
- `LongestCommonSubsequence.java` - LCS of two strings

Techniques: Binary search optimization, 2D DP

4. String DP (1 problem)

- `EditDistance.java` - Minimum edits to transform one string to another

Interview Frequency: Very Important

5. Array DP (1 problem)

- `MaximumProductSubarray.java` - Subarray with maximum product

Technique: Track both min and max products

Key Techniques to Master

1. 1D DP Pattern

```
int[] dp = new int[n + 1];
dp[0] = 0; // Base case
for (int i = 1; i <= n; i++) {
    dp[i] = dp[i - 1] + dp[i - 2]; // Recurrence
}
return dp[n];
```

Use Case: Fibonacci, climbing stairs, house robber **Time:** $O(n)$, **Space:** $O(n)$ or $O(1)$ optimized

2. 2D DP Pattern

```
int[][] dp = new int[m + 1][n + 1];
for (int i = 1; i <= m; i++) {
    for (int j = 1; j <= n; j++) {
        if (s1[i-1] == s2[j-1]) {
            dp[i][j] = dp[i-1][j-1] + 1;
        } else {
            dp[i][j] = Math.max(dp[i-1][j], dp[i][j-1]);
        }
    }
}
```

Use Case: LCS, edit distance, grid problems **Time:** $O(mn)$, **Space:** $O(mn)$ or $O(n)$ optimized

3. Space Optimization

```
int prev2 = 0, prev1 = 0;
for (int i = 0; i < n; i++) {
    int current = Math.max(prev1, prev2 + nums[i]);
    prev2 = prev1;
    prev1 = current;
}
return prev1;
```

Use Case: When only previous states are needed **Time:** $O(n)$, **Space:** $O(1)$

4. Binary Search Optimization (LIS)

```
int[] tails = new int[n];
int size = 0;
for (int num : nums) {
    int left = 0, right = size;
    while (left < right) {
        int mid = left + (right - left) / 2;
        if (tails[mid] < num) left = mid + 1;
        else right = mid;
    }
    tails[left] = num;
    if (left == size) size++;
}
return size;
```

Use Case: Longest increasing subsequence **Time:** $O(n \log n)$, **Space:** $O(n)$

Interview Preparation Checklist

Easy Level (Start Here)

- ☐ [ClimbingStairs.java](#) - Basic DP pattern
- ☐ [HouseRobber.java](#) - 1D DP with optimization
- ☐ [UniquePaths.java](#) - Grid DP

Medium Level

- ☐ [CoinChange.java](#) - **MOST IMPORTANT** - Unbounded knapsack
- ☐ [LongestIncreasingSubsequence.java](#) - Binary search optimization
- ☐ [HouseRobberII.java](#) - Circular variant
- ☐ [MaximumProductSubarray.java](#) - Track min/max

Hard Level

- ☐ [LongestCommonSubsequence.java](#) - 2D DP
- ☐ [EditDistance.java](#) - **MOST IMPORTANT** - String DP

Common Interview Questions

1. Climbing Stairs

Problem: Count ways to climb n stairs taking 1 or 2 steps **Solution:** Fibonacci pattern, DP or formula **See:** [ClimbingStairs.java](#)

2. Coin Change

Problem: Minimum coins to make amount **Key Insight:** Unbounded knapsack, DP with amount dimension **See:** [CoinChange.java](#)

3. Longest Common Subsequence

Problem: Find longest common subsequence of two strings **Strategy:** 2D DP comparing characters **See:** [LongestCommonSubsequence.java](#)

4. Edit Distance

Problem: Minimum edits to transform one string to another **Technique:** 2D DP with three operations (insert, delete, replace) **See:** [EditDistance.java](#)

Time Complexity Summary

Problem	Time	Space	Technique
Climbing Stairs	O(n)	O(1)	1D DP
House Robber	O(n)	O(1)	1D DP
House Robber II	O(n)	O(1)	Two DP calls
Coin Change	O(n*m)	O(n)	1D DP (knapsack)

Problem	Time	Space	Technique
Unique Paths	$O(m \cdot n)$	$O(n)$	2D DP optimized
LIS	$O(n \log n)$	$O(n)$	Binary search
LCS	$O(m \cdot n)$	$O(m \cdot n)$	2D DP
Edit Distance	$O(m \cdot n)$	$O(m \cdot n)$	2D DP
Max Product	$O(n)$	$O(1)$	Track min/max

$n = \text{length/amount}$, $m = \text{other dimension}$

Related Files

Array folder:

- Similar subarray problems

String folder:

- String manipulation techniques
-

Code Patterns to Remember

Pattern 1: 1D DP with Optimization

```
int prev2 = 0, prev1 = 0;
for (int i = 0; i < n; i++) {
    int current = calculate(prev1, prev2, nums[i]);
    prev2 = prev1;
    prev1 = current;
}
return prev1;
```

Pattern 2: Knapsack Pattern

```
int[] dp = new int[amount + 1];
Arrays.fill(dp, amount + 1);
dp[0] = 0;
for (int coin : coins) {
    for (int i = coin; i <= amount; i++) {
        dp[i] = Math.min(dp[i], dp[i - coin] + 1);
    }
}
return dp[amount];
```

Pattern 2: 2D DP for Strings

```
int[][] dp = new int[m + 1][n + 1];
for (int i = 1; i <= m; i++) {
    for (int j = 1; j <= n; j++) {
        if (s1[i-1] == s2[j-1]) {
            dp[i][j] = dp[i-1][j-1] + 1;
        } else {
            dp[i][j] = Math.max(dp[i-1][j], dp[i][j-1]);
        }
    }
}
```

Practice Tips

DP Problem Identification:

- **Optimal substructure:** Can problem be broken into subproblems?
- **Overlapping subproblems:** Are subproblems repeated?
- **State definition:** What parameters define the state?

Common DP Patterns:

- **Linear DP:** Climbing stairs, house robber
- **Knapsack:** Coin change, subset problems
- **LCS/Edit Distance:** String comparison
- **Grid DP:** Unique paths, minimum path sum

Space Optimization:

- Always ask: Can we reduce space?
- Use rolling arrays for 2D DP
- Use variables for 1D DP when possible

Common Mistakes:

- Not defining base cases correctly
- Off-by-one errors in DP array
- Not considering space optimization
- Incorrect recurrence relation

Generated: Interview Prep Package - Dynamic Programming Folder

Code Solutions

ClimbingStairs.java


```
package programs.dp;

import org.testng.Assert;

/**
 * LeetCode 70: Climbing Stairs
 *
 * Problem Statement:
 * You are climbing a staircase. It takes n steps to reach the top.
 * Each time you can either climb 1 or 2 steps. In how many distinct ways can you
 * climb to the top?
 *
 * Constraints:
 * - 1 <= n <= 45
 */
public class ClimbingStairs {

    public static void main(String[] args) {
        Assert.assertEquals(climbStairsDP(5), 8);
        Assert.assertEquals(climbStairsOptimal(5), 8);
        Assert.assertEquals(climbStairsBruteForce(5), 8);
    }

    /**
     * APPROACH 1: Dynamic Programming with Array
     *
     * Time Complexity: O(N)
     * - Single pass through the array.
     *
     * Space Complexity: O(N)
     * - Uses array of size n+1.
     */
    public static int climbStairsDP(int n) {
        if (n <= 2) {
            return n;
        }
        int[] dp = new int[n + 1];
        dp[1] = 1;
        dp[2] = 2;
        for (int i = 3; i <= n; i++) {
            dp[i] = dp[i - 1] + dp[i - 2];
        }
        return dp[n];
    }

    /**
     * APPROACH 2: Constant Space (Optimal)
     *
     * Time Complexity: O(N)
     * - Single pass through the array.
     *
     * Space Complexity: O(1)
     */
}
```

```

    * - Uses constant extra space.
    */
    public static int climbStairsOptimal(int n) {
        if (n <= 2) {
            return n;
        }
        int prev2 = 1;
        int prev1 = 2;
        for (int i = 3; i <= n; i++) {
            int current = prev1 + prev2;
            prev2 = prev1;
            prev1 = current;
        }
        return prev1;
    }

    /**
     * APPROACH 3: Brute Force (Recursion)
     *
     * Time Complexity:  $O(2^N)$ 
     * - Exponential due to repeated calculations.
     *
     * Space Complexity:  $O(N)$ 
     * - Recursion stack depth.
     */
    public static int climbStairsBruteForce(int n) {
        if (n <= 2) {
            return n;
        }
        return climbStairsBruteForce(n - 1) + climbStairsBruteForce(n - 2);
    }
}

```

CoinChange.java

```

package programs.dp;

import org.testng.Assert;

import java.util.Arrays;

/**
 * LeetCode 322: Coin Change
 *
 * Problem Statement:
 * You are given an integer array coins representing coins of different
 * denominations and an integer
 * amount representing a total amount of money. Return the fewest number of coins
 * that you need to make up that

```

```

    * amount. If that amount of money cannot be made up by any combination of the
    coins, return -1.
    *
    * Constraints:
    * - 1 <= coins.length <= 12
    * - 1 <= coins[i] <= 2^31 - 1
    * - 0 <= amount <= 10^4
    */
    public class CoinChange {

        public static void main(String[] args) {
            int[] coins1 = {1, 2, 5};
            Assert.assertEquals(coinChangeDP(coins1, 11), 3);
            Assert.assertEquals(coinChangeBruteForce(coins1, 11), 3);

            int[] coins2 = {2};
            Assert.assertEquals(coinChangeDP(coins2, 3), -1);
        }

        /**
         * APPROACH 1: Dynamic Programming (Optimal)
         *
         * Time Complexity: O(N * amount)
         * - N coins, amount iterations.
         *
         * Space Complexity: O(amount)
         * - DP array of size amount + 1.
         */
        public static int coinChangeDP(int[] coins, int amount) {
            int[] dp = new int[amount + 1];
            Arrays.fill(dp, amount + 1);
            dp[0] = 0;

            for (int i = 1; i <= amount; i++) {
                for (int coin : coins) {
                    if (coin <= i) {
                        dp[i] = Math.min(dp[i], dp[i - coin] + 1);
                    }
                }
            }

            return dp[amount] > amount ? -1 : dp[amount];
        }

        /**
         * APPROACH 2: Brute Force (Recursion)
         *
         * Time Complexity: O(N^amount)
         * - Exponential due to repeated calculations.
         *
         * Space Complexity: O(amount)
         * - Recursion stack depth.
         */
        public static int coinChangeBruteForce(int[] coins, int amount) {

```

```

        return coinChangeHelper(coins, amount, 0);
    }

    private static int coinChangeHelper(int[] coins, int remaining, int count) {
        if (remaining == 0) return count;
        if (remaining < 0) return -1;

        int minCoins = Integer.MAX_VALUE;
        for (int coin : coins) {
            int result = coinChangeHelper(coins, remaining - coin, count + 1);
            if (result >= 0 && result < minCoins) {
                minCoins = result;
            }
        }
        return minCoins == Integer.MAX_VALUE ? -1 : minCoins;
    }
}

```

EditDistance.java

```

package programs.dp;

import org.testng.Assert;

/**
 * LeetCode 72: Edit Distance
 *
 * Problem Statement:
 * Given two strings word1 and word2, return the minimum number of operations
 * required to convert
 * word1 to word2. You have the following three operations permitted on a word:
 * - Insert a character
 * - Delete a character
 * - Replace a character
 *
 * Constraints:
 * - 0 <= word1.length, word2.length <= 500
 * - word1 and word2 consist of lowercase English letters.
 */
public class EditDistance {

    public static void main(String[] args) {
        Assert.assertEquals(minDistanceDP("horse", "ros"), 3);
        Assert.assertEquals(minDistanceDP("intention", "execution"), 5);
        Assert.assertEquals(minDistanceBruteForce("horse", "ros"), 3);
    }

    /**
     * APPROACH 1: Dynamic Programming (Optimal)
     */
}

```

```

* Time Complexity:  $O(m * n)$ 
* - m and n are lengths of the two strings.
*
* Space Complexity:  $O(m * n)$ 
* - DP table of size  $(m+1) \times (n+1)$ .
*/
public static int minDistanceDP(String word1, String word2) {
    int m = word1.length();
    int n = word2.length();

    int[][] dp = new int[m + 1][n + 1];

    // Initialize base cases
    for (int i = 0; i <= m; i++) {
        dp[i][0] = i; // delete all characters
    }
    for (int j = 0; j <= n; j++) {
        dp[0][j] = j; // insert all characters
    }

    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (word1.charAt(i - 1) == word2.charAt(j - 1)) {
                dp[i][j] = dp[i - 1][j - 1];
            } else {
                dp[i][j] = 1 + Math.min(dp[i - 1][j - 1], // replace
                    Math.min(dp[i - 1][j], // delete
                        dp[i][j - 1])); // insert
            }
        }
    }

    return dp[m][n];
}

/**
 * APPROACH 2: Brute Force (Recursion)
 *
 * Time Complexity:  $O(3^{(m+n)})$ 
 * - Exponential due to three choices at each step.
 *
 * Space Complexity:  $O(m + n)$ 
 * - Recursion stack depth.
 */
public static int minDistanceBruteForce(String word1, String word2) {
    return minDistanceHelper(word1, word2, word1.length(), word2.length());
}

private static int minDistanceHelper(String word1, String word2, int m, int n)
{
    if (m == 0) return n;
    if (n == 0) return m;

    if (word1.charAt(m - 1) == word2.charAt(n - 1)) {

```

```

        return minDistanceHelper(word1, word2, m - 1, n - 1);
    }

    return 1 + Math.min(
        minDistanceHelper(word1, word2, m, n - 1), // insert
        Math.min(
            minDistanceHelper(word1, word2, m - 1, n), // delete
            minDistanceHelper(word1, word2, m - 1, n - 1) // replace
        )
    );
}
}

```

HouseRobber.java

```

package programs.dp;

import org.testng.Assert;

/**
 * LeetCode 198: House Robber
 *
 * Problem Statement:
 * You are a professional robber planning to rob houses along a street. Each house
 * has a certain
 * amount of money stashed, the only constraint stopping you from robbing each of
 * them is that adjacent
 * houses have security systems connected and it will automatically contact the
 * police if two adjacent houses
 * were broken into on the same night. Given an integer array nums representing
 * the amount of money at each
 * house, return the maximum amount of money you can rob tonight without alerting
 * the police.
 *
 * Constraints:
 * - 1 <= nums.length <= 100
 * - 0 <= nums[i] <= 400
 */
public class HouseRobber {

    public static void main(String[] args) {
        int[] nums1 = {1, 2, 3, 1};
        Assert.assertEquals(robOptimal(nums1), 4);
        Assert.assertEquals(robDP(nums1), 4);
        Assert.assertEquals(robBruteForce(nums1), 4);

        int[] nums2 = {2, 7, 9, 3, 1};
        Assert.assertEquals(robOptimal(nums2), 12);
    }
}

```

```
/**
 * APPROACH 1: Constant Space (Optimal)
 *
 * Time Complexity: O(N)
 * - Single pass through the array.
 *
 * Space Complexity: O(1)
 * - Uses constant extra space.
 */
public static int robOptimal(int[] nums) {
    if (nums == null || nums.length == 0) return 0;
    if (nums.length == 1) return nums[0];

    int prev2 = 0;
    int prev1 = 0;

    for (int num : nums) {
        int current = Math.max(prev1, prev2 + num);
        prev2 = prev1;
        prev1 = current;
    }

    return prev1;
}

/**
 * APPROACH 2: Dynamic Programming with Array
 *
 * Time Complexity: O(N)
 * - Single pass through the array.
 *
 * Space Complexity: O(N)
 * - Uses DP array of size n.
 */
public static int robDP(int[] nums) {
    if (nums == null || nums.length == 0) return 0;
    if (nums.length == 1) return nums[0];

    int[] dp = new int[nums.length];
    dp[0] = nums[0];
    dp[1] = Math.max(nums[0], nums[1]);

    for (int i = 2; i < nums.length; i++) {
        dp[i] = Math.max(dp[i - 1], dp[i - 2] + nums[i]);
    }

    return dp[nums.length - 1];
}

/**
 * APPROACH 3: Brute Force (Recursion)
 *
 * Time Complexity: O(2^N)
 * - Exponential due to two choices at each house.
```

```

    *
    * Space Complexity: O(N)
    * - Recursion stack depth.
    */
    public static int robBruteForce(int[] nums) {
        return robHelper(nums, 0);
    }

    private static int robHelper(int[] nums, int index) {
        if (index >= nums.length) return 0;

        int robCurrent = nums[index] + robHelper(nums, index + 2);
        int skipCurrent = robHelper(nums, index + 1);

        return Math.max(robCurrent, skipCurrent);
    }
}

```

HouseRobberII.java

```

package programs.dp;

import org.testng.Assert;

/**
 * LeetCode 213: House Robber II
 *
 * Problem Statement:
 * You are a professional robber planning to rob houses along a street. Each house
 * has a certain
 * amount of money stashed. All houses at this place are arranged in a circle.
 * This means the first house is
 * the neighbor of the last one. Return the maximum amount of money you can rob
 * tonight without alerting the police.
 *
 * Constraints:
 * - 1 <= nums.length <= 100
 * - 0 <= nums[i] <= 1000
 */
public class HouseRobberII {

    public static void main(String[] args) {
        int[] nums1 = {2, 3, 2};
        Assert.assertEquals(rob(nums1), 3);

        int[] nums2 = {1, 2, 3, 1};
        Assert.assertEquals(rob(nums2), 4);

        int[] nums3 = {1, 2, 3};
        Assert.assertEquals(rob(nums3), 3);
    }
}

```



```

    }

    /**
     * APPROACH: Two Pass DP (Optimal)
     *
     * Time Complexity: O(N)
     * - Two passes through the array.
     *
     * Space Complexity: O(1)
     * - Uses constant extra space.
     */
    public static int rob(int[] nums) {
        if (nums == null || nums.length == 0) return 0;
        if (nums.length == 1) return nums[0];

        // Rob houses from 0 to n-2 (exclude last house)
        int max1 = robLinear(nums, 0, nums.length - 2);
        // Rob houses from 1 to n-1 (exclude first house)
        int max2 = robLinear(nums, 1, nums.length - 1);

        return Math.max(max1, max2);
    }

    private static int robLinear(int[] nums, int start, int end) {
        int prev2 = 0, prev1 = 0;

        for (int i = start; i <= end; i++) {
            int current = Math.max(prev1, prev2 + nums[i]);
            prev2 = prev1;
            prev1 = current;
        }

        return prev1;
    }
}

```

LongestCommonSubsequence.java

```

package programs.dp;

import org.testng.Assert;

/**
 * LeetCode 1143: Longest Common Subsequence
 *
 * Problem Statement:
 * Given two strings text1 and text2, return the length of their longest common
 * subsequence.
 * A subsequence of a string is a new string generated from the original string
 * with some characters

```

```

    * (can be none) deleted without changing the relative order of the remaining
    characters.
    *
    * Constraints:
    * - 1 <= text1.length, text2.length <= 1000
    * - text1 and text2 consist of only lowercase English characters.
    */
    public class LongestCommonSubsequence {

        public static void main(String[] args) {
            Assert.assertEquals(longestCommonSubsequenceDP("abcde", "ace"), 3);
            Assert.assertEquals(longestCommonSubsequenceDP("abc", "abc"), 3);
            Assert.assertEquals(longestCommonSubsequenceDP("abc", "def"), 0);
            Assert.assertEquals(longestCommonSubsequenceBruteForce("abcde", "ace"),
3);
        }

        /**
         * APPROACH 1: Dynamic Programming (Optimal)
         *
         * Time Complexity: O(m * n)
         * - m and n are lengths of the two strings.
         *
         * Space Complexity: O(m * n)
         * - DP table of size (m+1) x (n+1).
         */
        public static int longestCommonSubsequenceDP(String text1, String text2) {
            int m = text1.length();
            int n = text2.length();

            int[][] dp = new int[m + 1][n + 1];

            for (int i = 1; i <= m; i++) {
                for (int j = 1; j <= n; j++) {
                    if (text1.charAt(i - 1) == text2.charAt(j - 1)) {
                        dp[i][j] = dp[i - 1][j - 1] + 1;
                    } else {
                        dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
                    }
                }
            }

            return dp[m][n];
        }

        /**
         * APPROACH 2: Brute Force (Recursion)
         *
         * Time Complexity: O(2^(m+n))
         * - Exponential due to two choices at each character.
         *
         * Space Complexity: O(m + n)
         * - Recursion stack depth.
         */
    }

```

```

    public static int longestCommonSubsequenceBruteForce(String text1, String
text2) {
        return lcsHelper(text1, text2, text1.length(), text2.length());
    }

    private static int lcsHelper(String text1, String text2, int m, int n) {
        if (m == 0 || n == 0) return 0;

        if (text1.charAt(m - 1) == text2.charAt(n - 1)) {
            return 1 + lcsHelper(text1, text2, m - 1, n - 1);
        }

        return Math.max(
            lcsHelper(text1, text2, m - 1, n),
            lcsHelper(text1, text2, m, n - 1)
        );
    }
}

```

LongestIncreasingSubsequence.java

```

package programs.dp;

import org.testng.Assert;

/**
 * LeetCode 300: Longest Increasing Subsequence
 *
 * Problem Statement:
 * Given an integer array nums, return the length of the longest strictly
increasing subsequence.
 *
 * Constraints:
 * - 1 <= nums.length <= 2500
 * - -10^4 <= nums[i] <= 10^4
 */
public class LongestIncreasingSubsequence {

    public static void main(String[] args) {
        int[] nums1 = {10, 9, 2, 5, 3, 7, 101, 18};
        Assert.assertEquals(lengthOfLISBinarySearch(nums1), 4);
        Assert.assertEquals(lengthOfLISDP(nums1), 4);
        Assert.assertEquals(lengthOfLISBruteForce(nums1), 4);

        int[] nums2 = {0, 1, 0, 3, 2, 3};
        Assert.assertEquals(lengthOfLISBinarySearch(nums2), 4);
    }

    /**
     * APPROACH 1: Binary Search with Tails Array (Optimal)

```

```
*
* Time Complexity:  $O(N \log N)$ 
* - Binary search for each element.
*
* Space Complexity:  $O(N)$ 
* - Tails array of size n.
*/
public static int lengthOfLISBinarySearch(int[] nums) {
    if (nums == null || nums.length == 0) return 0;

    int[] tails = new int[nums.length];
    int size = 0;

    for (int num : nums) {
        int left = 0, right = size;
        while (left < right) {
            int mid = left + (right - left) / 2;
            if (tails[mid] < num) {
                left = mid + 1;
            } else {
                right = mid;
            }
        }

        tails[left] = num;
        if (left == size) {
            size++;
        }
    }

    return size;
}

/**
* APPROACH 2: Dynamic Programming
*
* Time Complexity:  $O(N^2)$ 
* - Nested loops to compare all pairs.
*
* Space Complexity:  $O(N)$ 
* - DP array of size n.
*/
public static int lengthOfLISDP(int[] nums) {
    if (nums == null || nums.length == 0) return 0;

    int[] dp = new int[nums.length];
    int maxLen = 1;

    for (int i = 0; i < nums.length; i++) {
        dp[i] = 1;
        for (int j = 0; j < i; j++) {
            if (nums[j] < nums[i]) {
                dp[i] = Math.max(dp[i], dp[j] + 1);
            }
        }
    }
}
```

```

        }
        maxLen = Math.max(maxLen, dp[i]);
    }

    return maxLen;
}

/**
 * APPROACH 3: Brute Force (Recursion)
 *
 * Time Complexity: O(2^N)
 * - Exponential due to include/exclude choices.
 *
 * Space Complexity: O(N)
 * - Recursion stack depth.
 */
public static int lengthOfLISBruteForce(int[] nums) {
    if (nums == null || nums.length == 0) return 0;
    return lisHelper(nums, -1, 0);
}

private static int lisHelper(int[] nums, int prevIndex, int currentIndex) {
    if (currentIndex == nums.length) return 0;

    int include = 0;
    if (prevIndex == -1 || nums[currentIndex] > nums[prevIndex]) {
        include = 1 + lisHelper(nums, currentIndex, currentIndex + 1);
    }

    int exclude = lisHelper(nums, prevIndex, currentIndex + 1);

    return Math.max(include, exclude);
}
}

```

MaximumProductSubarray.java

```

package programs.dp;

import org.testng.Assert;

/**
 * LeetCode 152: Maximum Product Subarray
 *
 * Problem Statement:
 * Given an integer array nums, find the contiguous subarray within an array
 * (containing at least
 * one number) which has the largest product.
 *
 * Constraints:

```

```
* - 1 <= nums.length <= 2 * 10^4
* - -10 <= nums[i] <= 10
* - The product of any prefix or suffix of nums is guaranteed to fit in a 32-bit
integer.
*/
public class MaximumProductSubarray {

    public static void main(String[] args) {
        int[] nums1 = {2, 3, -2, 4};
        Assert.assertEquals(maxProductOptimal(nums1), 6);
        Assert.assertEquals(maxProductBruteForce(nums1), 6);

        int[] nums2 = {-2, 0, -1};
        Assert.assertEquals(maxProductOptimal(nums2), 0);
    }

    /**
     * APPROACH 1: Track Max and Min (Optimal)
     *
     * Time Complexity: O(N)
     * - Single pass through the array.
     *
     * Space Complexity: O(1)
     * - Uses constant extra space.
     */
    public static int maxProductOptimal(int[] nums) {
        if (nums == null || nums.length == 0) return 0;

        int maxProduct = nums[0];
        int minProduct = nums[0];
        int result = nums[0];

        for (int i = 1; i < nums.length; i++) {
            int num = nums[i];

            // If current number is negative, swap max and min
            if (num < 0) {
                int temp = maxProduct;
                maxProduct = minProduct;
                minProduct = temp;
            }

            maxProduct = Math.max(num, maxProduct * num);
            minProduct = Math.min(num, minProduct * num);

            result = Math.max(result, maxProduct);
        }

        return result;
    }

    /**
     * APPROACH 2: Brute Force
     *

```

```

    * Time Complexity:  $O(N^2)$ 
    * - For each starting position, calculate all subarray products.
    *
    * Space Complexity:  $O(1)$ 
    * - Uses constant extra space.
    */
    public static int maxProductBruteForce(int[] nums) {
        if (nums == null || nums.length == 0) return 0;

        int maxProduct = nums[0];

        for (int i = 0; i < nums.length; i++) {
            int product = 1;
            for (int j = i; j < nums.length; j++) {
                product *= nums[j];
                maxProduct = Math.max(maxProduct, product);
            }
        }

        return maxProduct;
    }
}

```

UniquePaths.java

```

package programs.dp;

import org.testng.Assert;

import java.util.Arrays;

/**
 * LeetCode 62: Unique Paths
 *
 * Problem Statement:
 * There is a robot on an m x n grid. The robot can only move either down or right
 * at any point in time.
 * The robot is trying to reach the bottom-right corner of the grid. How many
 * possible unique paths are there?
 *
 * Constraints:
 * -  $1 \leq m, n \leq 100$ 
 * - The answer will be less than or equal to  $2 * 10^9$ 
 */
public class UniquePaths {

    public static void main(String[] args) {
        Assert.assertEquals(uniquePathsOptimal(3, 7), 28);
        Assert.assertEquals(uniquePathsDP(3, 7), 28);
        Assert.assertEquals(uniquePathsBruteForce(3, 7), 28);
    }
}

```

```
}

/**
 * APPROACH 1: 1D DP Array (Optimal Space)
 *
 * Time Complexity: O(m * n)
 * - Nested loops through grid.
 *
 * Space Complexity: O(n)
 * - Single row DP array.
 */
public static int uniquePathsOptimal(int m, int n) {
    int[] dp = new int[n];
    Arrays.fill(dp, 1);

    for (int i = 1; i < m; i++) {
        for (int j = 1; j < n; j++) {
            dp[j] += dp[j - 1];
        }
    }

    return dp[n - 1];
}

/**
 * APPROACH 2: 2D DP Array
 *
 * Time Complexity: O(m * n)
 * - Nested loops through grid.
 *
 * Space Complexity: O(m * n)
 * - Full DP table.
 */
public static int uniquePathsDP(int m, int n) {
    int[][] dp = new int[m][n];

    for (int i = 0; i < m; i++) {
        dp[i][0] = 1;
    }
    for (int j = 0; j < n; j++) {
        dp[0][j] = 1;
    }

    for (int i = 1; i < m; i++) {
        for (int j = 1; j < n; j++) {
            dp[i][j] = dp[i - 1][j] + dp[i][j - 1];
        }
    }

    return dp[m - 1][n - 1];
}

/**
 * APPROACH 3: Brute Force (Recursion)
```



```
*
* Time Complexity:  $O(2^{(m+n)})$ 
* - Exponential due to two choices at each step.
*
* Space Complexity:  $O(m + n)$ 
* - Recursion stack depth.
*/
public static int uniquePathsBruteForce(int m, int n) {
    return uniquePathsHelper(m, n, 0, 0);
}

private static int uniquePathsHelper(int m, int n, int row, int col) {
    if (row == m - 1 && col == n - 1) return 1;
    if (row >= m || col >= n) return 0;

    return uniquePathsHelper(m, n, row + 1, col) + uniquePathsHelper(m, n,
row, col + 1);
}
}
```

Topic: fileio

Code Solutions

AppendFile.java

```
package programs.fileio;

import org.testng.Assert;

import java.io.*;

/**
 * Append to File
 *
 * Problem Statement:
 * Append text content to an existing file and then read the file to verify the
content.
 *
 * Constraints:
 * - File path must be valid and writable.
 * - File must exist for append operation.
 */
public class AppendFile {

    public static void main(String[] args) {
        String text = "Welcome to GeeksforGeeks\nHappy Learning!";
        String filePath = "src/test/resources/file3.txt";
    }
}
```

```

        boolean appendSuccess = appendToFile(filePath, text);
        Assert.assertTrue(appendSuccess, "File append operation failed");
    }

    /**
     * APPROACH: BufferedWriter with FileReader (Optimal)
     *
     * Time Complexity: O(N)
     * - N is the length of text to append.
     *
     * Space Complexity: O(1)
     * - Constant buffer size.
     */
    public static boolean appendToFile(String filePath, String text) {
        try (BufferedWriter writer = new BufferedWriter(new FileWriter(filePath,
true))) {
            writer.write(text);
            System.out.println("Successfully appended to the file: " + filePath);
        } catch (IOException e) {
            System.err.println("An error occurred while appending to the file.");
            e.printStackTrace();
            return false;
        }

        System.out.println("\n--- Reading file contents ---");

        try (BufferedReader reader = new BufferedReader(new FileReader(filePath)))
        {
            String currentLine;
            while ((currentLine = reader.readLine()) != null) {
                System.out.println(currentLine);
            }
        } catch (IOException e) {
            System.err.println("An error occurred while reading the file.");
            e.printStackTrace();
            return false;
        }
        return true;
    }
}

```

CompareFiles.java

```

package programs.fileio;

import org.testng.Assert;

import java.io.IOException;
import java.nio.file.Files;

```

```

import java.nio.file.Paths;
import java.util.Iterator;
import java.util.stream.Stream;

/**
 * Compare Files
 *
 * Problem Statement:
 * Compare two text files line by line and report any differences found.
 *
 * Constraints:
 * - Both file paths must be valid and readable.
 * - Files should be text files.
 */
public class CompareFiles {

    public static void main(String[] args) {
        String file1 = "src/test/resources/file1.txt";
        String file2 = "src/test/resources/file2.txt";
        boolean comparisonSuccess = compareFiles(file1, file2);
        Assert.assertTrue(comparisonSuccess, "File comparison failed");
    }

    /**
     * APPROACH: Stream API with Iterators (Optimal)
     *
     * Time Complexity: O(N)
     * - N is the total number of lines in both files.
     *
     * Space Complexity: O(1)
     * - Constant space for iterators.
     */
    public static boolean compareFiles(String file1, String file2) {
        try (
            Stream<String> fileStream1 = Files.lines(Paths.get(file1));
            Stream<String> fileStream2 = Files.lines(Paths.get(file2))
        ) {
            Iterator<String> it1 = fileStream1.iterator();
            Iterator<String> it2 = fileStream2.iterator();
            int lineNum = 1;
            boolean hasDifferences = false;

            while (it1.hasNext() || it2.hasNext()) {
                String line1 = it1.hasNext() ? it1.next() : "<no line>";
                String line2 = it2.hasNext() ? it2.next() : "<no line>";
                if (!line1.equals(line2)) {
                    System.out.printf("Difference at line %d:\nFile1: %s\nFile2: %s\n\n", lineNum, line1, line2);
                    hasDifferences = true;
                }
                lineNum++;
            }

            if (!hasDifferences) {

```

```

        System.out.println("Files are identical.");
    }
    return true;

} catch (IOException e) {
    System.err.println("Error comparing files: " + e.getMessage());
    e.printStackTrace();
    return false;
}
}
}

```

CountLogsTest.java

```

package programs.fileio;

import org.testng.Assert;
import org.testng.annotations.Test;

import java.io.BufferedReader;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.util.Comparator;
import java.util.HashMap;
import java.util.Map;
import java.util.Optional;
import java.util.function.Function;
import java.util.stream.Collectors;
import java.util.stream.Stream;

/**
 * Count and Analyze Logs
 *
 * Problem Statement:
 * Analyze log files to count log levels by frequency and find the longest line in
a file.
 *
 * Constraints:
 * - Input file must exist and be readable.
 * - Log format must be consistent for level extraction.
 */
public class CountLogsTest {

    @Test
    public void countLogLevelsBufferedTest() {
        Path path = Paths.get("src/test/resources/application.log");
        Map<String, Integer> logLevels = countLogLevelsBuffered(path);
        Assert.assertNotNull(logLevels, "Log levels map should not be null");
    }
}

```

```

        System.out.println(logLevels);
    }

    @Test
    public void countLogLevelsTest() {
        Path path = Paths.get("src/test/resources/application.log");
        Map<String, Long> logLevels = countLogLevels(path);
        Assert.assertNotNull(logLevels, "Log levels map should not be null");
        System.out.println(logLevels);
    }

    @Test
    public void findLongestLineTest() {
        Path path = Paths.get("src/test/resources/emp.txt");
        Optional<String> longestLine = findLongestLine(path);
        Assert.assertTrue(longestLine.isPresent(), "Should find a longest line");
        longestLine.ifPresent(line -> {
            System.out.println("Longest Line:");
            System.out.println(line);
            System.out.println("\nLength:");
            System.out.println(line.length());
        });
    }

    /**
     * APPROACH 1: Stream API with Collectors (Modern NIO)
     *
     * Time Complexity: O(N)
     * - N is the total number of lines.
     *
     * Space Complexity: O(K)
     * - K is the number of unique log levels.
     */
    public static Map<String, Long> countLogLevels(Path path) {
        try (Stream<String> lines = Files.lines(path)) {
            return lines
                .map(String::trim)
                .filter(line -> !line.isEmpty())
                .map(line -> line.split("\\s+")[2])
                .collect(Collectors.groupingBy(Function.identity(),
Collectors.counting()));
        } catch (IOException e) {
            throw new RuntimeException("Failed to read log file", e);
        }
    }

    /**
     * APPROACH 2: Stream with Comparator (Modern NIO)
     *
     * Time Complexity: O(N)
     * - N is the total number of lines.
     *
     * Space Complexity: O(1)
     * - Constant space for comparison.

```

```

    */
    public static Optional<String> findLongestLine(Path path) {
        try (Stream<String> lines = Files.lines(path)) {
            return lines.max(Comparator.comparingInt(String::length));
        } catch (IOException e) {
            throw new RuntimeException("Failed to read lines for longest line
check", e);
        }
    }

    /**
     * APPROACH 3: BufferedReader (Traditional I/O)
     *
     * Time Complexity: O(N)
     * - N is the total number of lines.
     *
     * Space Complexity: O(K)
     * - K is the number of unique log levels.
     */
    public static Map<String, Integer> countLogLevelsBuffered(Path path) {
        Map<String, Integer> counts = new HashMap<>();
        try (BufferedReader reader = Files.newBufferedReader(path)) {
            String line;
            while ((line = reader.readLine()) != null) {
                line = line.trim();
                if (line.isEmpty()) continue;
                String level = line.split("\\s+")[2];
                counts.merge(level, 1, Integer::sum);
            }
        } catch (IOException e) {
            throw new RuntimeException(e);
        }
        return counts;
    }
}

```

FileIO.java

```

package programs.fileio;

import org.testng.Assert;

import java.io.*;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.nio.file.StandardOpenOption;
import java.util.stream.Stream;

```

```
/**
 * File I/O Operations
 *
 * Problem Statement:
 * Demonstrate different approaches for reading and writing to files using both
traditional I/O and modern NIO APIs.
 *
 * Constraints:
 * - File path must be valid and writable.
 * - File must exist for append operations.
 */
public class FileIO {

    public static void main(String[] args) {
        boolean bufferSuccess =
readWriteWithBuffer("src/test/resources/file3.txt");
        Assert.assertTrue(bufferSuccess, "Buffer I/O operation failed");

        boolean streamSuccess =
readWriteWithStream("src/test/resources/file3.txt");
        Assert.assertTrue(streamSuccess, "Stream I/O operation failed");
    }

    /**
     * APPROACH 1: BufferedReader/Writer (Traditional I/O)
     *
     * Time Complexity: O(N)
     * - N is the total content size.
     *
     * Space Complexity: O(1)
     * - Constant buffer size.
     */
    public static boolean readWriteWithBuffer(String filePath) {
        try (BufferedWriter writer = new BufferedWriter(new OutputStreamWriter(new
FileOutputStream(filePath, true), StandardCharsets.UTF_8))) {
            writer.write("Hello World");
            writer.newLine();
            writer.write(String.valueOf(Math.random()));
            writer.newLine();
        } catch (Exception e) {
            System.err.println("Error writing to file: " + e.getMessage());
            e.printStackTrace();
            return false;
        }

        try (BufferedReader reader = new BufferedReader(new InputStreamReader(new
FileInputStream(filePath), StandardCharsets.UTF_8))) {
            String line;
            while ((line = reader.readLine()) != null) {
                System.out.println(line);
            }
        } catch (Exception e) {
            System.err.println("Error reading from file: " + e.getMessage());
        }
    }
}
```

```

        e.printStackTrace();
        return false;
    }
    return true;
}

/**
 * APPROACH 2: Files.lines() and Files.writeString() (Modern NIO)
 *
 * Time Complexity: O(N)
 * - N is the total content size.
 *
 * Space Complexity: O(1)
 * - Stream processing with lazy evaluation.
 */
public static boolean readWriteWithStream(String filePath) {
    Path path = Paths.get(filePath);

    try {
        Files.writeString(path, "Hello World\n" + Math.random() + "\n",
            StandardCharsets.UTF_8, StandardOpenOption.CREATE, StandardOpenOption.APPEND);
    } catch (IOException e) {
        System.err.println("Error writing to file: " + e.getMessage());
        e.printStackTrace();
        return false;
    }

    try (Stream<String> lines = Files.lines(path, StandardCharsets.UTF_8)) {
        lines.forEach(System.out::println);
    } catch (IOException e) {
        System.err.println("Error reading from file: " + e.getMessage());
        e.printStackTrace();
        return false;
    }

    return true;
}
}

```

MasterStudyGuideBundler.java

```

package programs.fileio;

import org.testng.Assert;

import java.io.IOException;
import java.nio.file.*;
import java.util.Comparator;
import java.util.stream.Stream;

```



```

/**
 * Master Study Guide Bundler
 *
 * Problem Statement:
 * Bundle all Java source files from all subpackages into a single master markdown
file.
 * Includes existing GUIDE.md content if present in each subpackage.
 *
 * Constraints:
 * - Root directory must exist.
 * - Files must be readable.
 */
public class MasterStudyGuideBundler {

    private static final String INTERVIEW_PREP_PATH = "src/test/java/programs";
    private static final String OUTPUT_FILE_NAME = "programs.md";

    public static void main(String[] args) {
        boolean success = bundleAllPackages();
        Assert.assertTrue(success, "Master guide bundling failed");
    }

    /**
     * APPROACH: Files.list() with Stream API (Modern NIO)
     *
     * Time Complexity: O(N)
     * - N is the total size of all files to read.
     *
     * Space Complexity: O(N)
     * - StringBuilder accumulates all content.
     */
    public static boolean bundleAllPackages() {
        Path rootDir = Paths.get(INTERVIEW_PREP_PATH);
        Path outputFile = rootDir.resolve(OUTPUT_FILE_NAME);

        if (!Files.exists(rootDir) || !Files.isDirectory(rootDir)) {
            System.err.println("Root directory does not exist: " +
rootDir.toAbsolutePath());
            return false;
        }

        StringBuilder masterContent = new StringBuilder();

        masterContent.append("# Complete Interview Prep Master Guide\n\n");
        masterContent.append("> *Generated automatically on:
").append(java.time.LocalDate.now()).append("*\n\n");
        masterContent.append("---\n\n");

        try (Stream<Path> subDirs = Files.list(rootDir)) {
            subDirs.filter(Files::isDirectory)
                .sorted(Comparator.comparing(Path::getFileName))
                .forEach(packageDir -> processSubpackage(packageDir,
masterContent));

```

```

        Files.writeString(outputFile, masterContent.toString(),
StandardOpenOption.CREATE, StandardOpenOption.TRUNCATE_EXISTING);
        System.out.println("✅ Master guide created successfully at: " +
outputFile.toAbsolutePath());
        return true;
    } catch (IOException e) {
        System.err.println("Error bundling master guide: " + e.getMessage());
        return false;
    }
}

private static void processSubpackage(Path packageDir, StringBuilder
masterContent) {
    String packageName = packageDir.getFileName().toString();

    masterContent.append("# Topic: ").append(packageName).append("\n\n");

    Path guideFile = packageDir.resolve("GUIDE.md");
    if (Files.exists(guideFile)) {
        try {
            masterContent.append("## Notes & Concept Guide\n\n");
            masterContent.append(Files.readString(guideFile)).append("\n\n");
        } catch (IOException e) {
            System.err.println("Error reading guide in " + packageName + ": "
+ e.getMessage());
        }
    }

    masterContent.append("## Code Solutions\n\n");

    try (Stream<Path> files = Files.walk(packageDir, 1)) {
        files.filter(Files::isRegularFile)
            .filter(p -> p.toString().endsWith(".java"))
            .sorted(Comparator.comparing(Path::getFileName))
            .forEach(javaFile -> {
                try {
                    String fileName = javaFile.getFileName().toString();
                    String code = Files.readString(javaFile);

                    masterContent.append("###
").append(fileName).append("\n\n");
                    masterContent.append("`java\n");
                    masterContent.append(code).append("\n");
                    masterContent.append("`");
                } catch (IOException e) {
                    System.err.println("Error reading " + javaFile + ": "
+ e.getMessage());
                }
            });

        masterContent.append("\n---\n\n");

    } catch (IOException e) {

```

```

        System.err.println("Error traversing files in " + packageName + ": " +
e.getMessage());
    }
}
}

```

PasswordSanitizer.java

```

package programs.fileio;

import org.testng.Assert;

import java.util.ArrayList;
import java.util.List;

/**
 * Password Sanitizer
 *
 * Problem Statement:
 * Filter passwords that meet the following criteria:
 * - At least 5 characters long
 * - Must be alphanumeric (contains both letters and numbers)
 *
 * Constraints:
 * - Password list can be empty.
 * - Passwords can contain any characters.
 */
public class PasswordSanitizer {

    public static void main(String[] args) {
        List<String> passwords = new ArrayList<>();
        passwords.add("peC1");
        passwords.add("pass@123");
        passwords.add("word@321");
        passwords.add("wordpass");
        passwords.add("987345");
        String result = filter(passwords);
        Assert.assertNotNull(result, "Filter result should not be null");
        System.out.println(result);
    }

    /**
     * APPROACH: Iterative Filtering (Optimal)
     *
     * Time Complexity: O(N * M)
     * - N is the number of passwords, M is the average password length.
     *
     * Space Complexity: O(N)
     * - List for valid passwords.
     */
    public static String filter(List<String> passwords) {

```

```

        List<String> validPass = new ArrayList<>();

        for (String password : passwords) {
            boolean isLongEnough = password.length() >= 5;
            boolean containsOnlyLet = password.matches("[a-zA-Z]+");
            boolean containsOnlyNum = password.matches("[0-9]+");
            boolean isMixed = !(containsOnlyLet || containsOnlyNum);

            if (isLongEnough && isMixed) {
                validPass.add(password);
            }
        }

        return String.join(" ", validPass);
    }

    /**
     * APPROACH 2: Stream API (Modern)
     *
     * Time Complexity: O(N * M)
     * - N is the number of passwords, M is the average password length.
     *
     * Space Complexity: O(N)
     * - Stream collects valid passwords.
     */
    public static String filterStream(List<String> passwords) {
        return passwords.stream()
            .filter(password -> password.length() >= 5)
            .filter(password -> !password.matches("[a-zA-Z]+"))
            .filter(password -> !password.matches("[0-9]+"))
            .collect(java.util.stream.Collectors.joining(" "));
    }
}

```

ReadLogs.java

```

package programs.fileio;

import org.testng.Assert;

import java.io.*;
import java.nio.file.Paths;

/**
 * Read and Filter Logs
 *
 * Problem Statement:

```

```

* Read a log file and extract only lines containing "ERROR" to a separate output
file.
*
* Constraints:
* - Input file must exist and be readable.
* - Output file path must be writable.
*/
public class ReadLogs {

    public static void main(String[] args) {
        String inputFile = "src/test/resources/application.log";
        String outputFile = "src/test/resources/error.log";
        boolean success = extractErrorLogs(inputFile, outputFile);
        Assert.assertTrue(success, "Log extraction failed");
    }

    /**
     * APPROACH: BufferedReader/Writer (Traditional I/O)
     *
     * Time Complexity: O(N)
     * - N is the total number of lines in the log file.
     *
     * Space Complexity: O(1)
     * - Constant buffer size.
     */
    public static boolean extractErrorLogs(String inputFile, String outputFile) {
        try (BufferedReader reader = new BufferedReader(new
FileReader(inputFile));
            BufferedWriter writer = new BufferedWriter(new
FileWriter(outputFile))) {
            String currentLine;
            while ((currentLine = reader.readLine()) != null) {
                if (currentLine.contains("ERROR")) {
                    writer.write(currentLine);
                    writer.newLine();
                }
            }
            System.out.println("Successfully extracted error logs to: " +
Paths.get(outputFile).toAbsolutePath());
        } catch (IOException e) {
            System.err.println("An I/O error occurred: " + e.getMessage());
            e.printStackTrace();
            return false;
        }
        return true;
    }
}

```

ReadLogsModern.java

```
package programs.fileio;

import org.testng.Assert;

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.nio.file.StandardOpenOption;
import java.util.List;
import java.util.stream.Collectors;

/**
 * Read and Filter Logs (Modern NIO)
 *
 * Problem Statement:
 * Read a log file using modern NIO API and extract only lines containing "ERROR"
to a separate output file.
 *
 * Constraints:
 * - Input file must exist and be readable.
 * - Output file path must be writable.
 */
public class ReadLogsModern {

    public static void main(String[] args) {
        Path inputFile = Paths.get("src/test/resources/application.log");
        Path outputFile = Paths.get("src/test/resources/error.log");
        boolean success = extractErrorLogsModern(inputFile, outputFile);
        Assert.assertTrue(success, "Modern log extraction failed");
    }

    /**
     * APPROACH: Files.lines() with Stream API (Modern NIO)
     *
     * Time Complexity: O(N)
     * - N is the total number of lines in the log file.
     *
     * Space Complexity: O(M)
     * - M is the number of error lines collected.
     */
    public static boolean extractErrorLogsModern(Path inputFile, Path outputFile)
    {
        try {
            List<String> errorLines = Files.lines(inputFile)
                .filter(line -> line.contains("ERROR"))
                .collect(Collectors.toList());
            Files.write(outputFile, errorLines, StandardOpenOption.CREATE,
StandardOpenOption.APPEND);
            System.out.println("Successfully extracted error logs to: " +
outputFile.toAbsolutePath());
            return true;
        }
    }
}
```

```

        } catch (IOException e) {
            System.err.println("An I/O error occurred: " + e.getMessage());
            e.printStackTrace();
            return false;
        }
    }
}

```

StudyGuideBundler.java

```

package programs.fileio;

import org.testng.Assert;

import java.io.IOException;
import java.nio.file.*;
import java.util.Comparator;
import java.util.stream.Stream;

/**
 * Study Guide Bundler
 *
 * Problem Statement:
 * Bundle all Java source files from a subpackage into a single markdown file for
 * study purposes.
 * Includes existing GUIDE.md content if present.
 *
 * Constraints:
 * - Target directory must exist.
 * - Files must be readable.
 */
public class StudyGuideBundler {

    private static final String BASE_PATH = "src/test/java/InterviewPrep";

    public static void main(String[] args) {
        boolean success = bundleSubpackage("Array");
        Assert.assertTrue(success, "Study guide bundling failed");
    }

    /**
     * APPROACH: Files.walk() with Stream API (Modern NIO)
     *
     * Time Complexity: O(N)
     * - N is the total size of all files to read.
     *
     * Space Complexity: O(N)
     * - StringBuilder accumulates all content.
     */
    public static boolean bundleSubpackage(String subpackageName) {

```

```

Path targetDir = Paths.get(BASE_PATH, subpackageName);
Path guideFile = targetDir.resolve("GUIDE.md");
Path outputFile = targetDir.resolve("BUNDLE_" + subpackageName + ".md");

if (!Files.exists(targetDir) || !Files.isDirectory(targetDir)) {
    System.err.println("Directory does not exist: " +
targetDir.toAbsolutePath());
    return false;
}

StringBuilder content = new StringBuilder();

if (Files.exists(guideFile)) {
    try {
        content.append(Files.readString(guideFile)).append("\n\n");
    } catch (IOException e) {
        System.err.println("Failed to read GUIDE.md: " + e.getMessage());
    }
} else {
    content.append("# ").append(subpackageName).append(" Revision
Guide\n\n");
}

content.append("---\n\n## Java Source Code Solutions\n\n");

try (Stream<Path> paths = Files.walk(targetDir, 1)) {
    paths.filter(Files::isRegularFile)
        .filter(p -> p.toString().endsWith(".java"))
        .sorted(Comparator.comparing(Path::getFileName))
        .forEach(javaFile -> {
            try {
                String fileName = javaFile.getFileName().toString();
                String code = Files.readString(javaFile);

                content.append("###
").append(fileName).append("\n\n");
                content.append("` ` `java\n");
                content.append(code).append("\n");
                content.append("` ` `\n\n");
            } catch (IOException e) {
                System.err.println("Could not read file: " +
javaFile);
            }
        });

    Files.writeString(outputFile, content.toString(),
StandardOpenOption.CREATE, StandardOpenOption.TRUNCATE_EXISTING);
    System.out.println("✅ Bundled guide successfully generated: " +
outputFile.toAbsolutePath());
    return true;
} catch (IOException e) {
    System.err.println("Error processing folder: " + e.getMessage());
    return false;
}

```



```
}  
}  
}
```

Topic: hashmaps

Notes & Concept Guide

HashMaps & Frequency Problems - Interview Prep Guide

Overview

This folder contains **9 problems** focused on hash-based data structures, frequency counting, and grouping patterns.

Problem Categories

1. Anagram Problems (2 problems)

- `GroupAnagrams.java` - Group all anagrams together (multiple approaches)
- `ValidAnagram.java` - Check if two strings are anagrams (multiple approaches)

Techniques:

- Sorting characters: Time $O(n \log n)$, Space $O(n)$
- HashMap: Time $O(n)$, Space $O(n)$
- Character array (26 letters): Time $O(n)$, Space $O(1)$

Interview Focus: Compare approaches, discuss tradeoffs

2. Character Frequency (3 problems)

- `CharacterFrequency.java` - Count character occurrences
- `FirstNonRepeatingCharacter.java` - First non-repeating character
- `SecondMostFrequentChar.java` - Second most frequent
- `MaxMinCharacterFrequency.java` - Max and min frequency

Techniques: HashMap, LinkedHashMap (for order), Array counting

3. Integer/Element Frequency (3 problems)

- `IntegerFrequency.java` - Count integer occurrences
- `ArrayElementFrequency.java` - Frequency of array elements

Data Structures: `HashMap<Integer, Integer>`, `HashMap<Integer, List>`

4. String Frequency (1 problem)

- `SecondMostFrequentString.java` - Frequency in strings
-

Key Techniques to Master

1. HashMap for Frequency Counting

```
Map<Character, Integer> freq = new HashMap<>();
for (char c : str.toCharArray()) {
    freq.put(c, freq.getOrDefault(c, 0) + 1);
}
```

Use Case: Anagrams, frequency counting, duplicates **Time:** $O(n)$, **Space:** $O(k)$ where k = unique elements

2. Sorting Approach

```
char[] chars = str.toCharArray();
Arrays.sort(chars);
String key = new String(chars);
```

Use Case: Grouping anagrams, checking anagrams **Time:** $O(n \log n)$, **Space:** $O(n)$

3. Character Array (Optimization)

For lowercase English letters only:

```
int[] freq = new int[26];
for (char c : str.toCharArray()) {
    freq[c - 'a']++;
}
```

Time: $O(n)$, **Space:** $O(1)$ - only 26 letters **Advantage:** Faster than HashMap for this specific case

4. LinkedHashMap for Order

Maintain insertion order while counting:

```
Map<Character, Integer> map = new LinkedHashMap<>();
```

Use Case: Finding first non-repeating character while maintaining order

Interview Preparation Checklist

Easy Level (Start Here)

- ☐ [CharacterFrequency.java](#) - Simple frequency counting
- ☐ [IntegerFrequency.java](#) - Count occurrences
- ☐ [ValidAnagram.java](#) - String comparison method

Medium Level

- ☐ [GroupAnagrams.java](#) - **MOST IMPORTANT** - Multiple approaches
- ☐ [FirstNonRepeatingCharacter.java](#) - First non-repeating
- ☐ [SecondMostFrequentChar.java](#) - Min heap approach
- ☐ [ArrayElementFrequency.java](#) - Map all elements

Hard Level

- ☐ [MaxMinCharacterFrequency.java](#) - Complex frequency queries
- ☐ [SecondMostFrequentString.java](#) - Sorting frequencies

Common Interview Questions

1. Check if Two Strings are Anagrams

Problem: Determine if str1 and str2 are anagrams **Solutions:**

```
Approach 1: Sort and compare
- Time: O(n log n), Space: O(n)

Approach 2: Use HashMap
- Time: O(n), Space: O(1) for 26 letters

Approach 3: Character count array
- Time: O(n), Space: O(1)
```

See: [ValidAnagram.java](#)

2. Group Anagrams

Problem: Given array of strings, group anagrams together **Key Insight:** Anagrams have same sorted characters **Solution Strategy:**

1. Sort each word's characters
2. Use as HashMap key
3. Group words by key **See:** [GroupAnagrams.java](#) (has multiple approaches!)

3. Find First Non-Repeating Character

Problem: Return first character that appears only once **Solution:**

1. Count all frequencies (HashMap)
2. Iterate through string to maintain order
3. Return first char with count = 1 **See:** [FirstNonRepeatingCharacter.java](#)

4. Character Frequency Analysis

Problem: Analyze character distribution **Applications:**

- Frequency analysis in cryptography
- Data compression analysis
- Natural language processing **See:** [CharacterFrequency.java](#)

Comparison of Approaches

For Anagram Detection:

Approach	Time	Space	Pros	Cons
Sort	$O(n \log n)$	$O(n)$	Simple	Slower
HashMap	$O(n)$	$O(k)$	Fast	More code
Array[26]	$O(n)$	$O(1)$	Fastest, fixed space	English only
Compare counts	$O(n)$	$O(k)$	Most readable	Still $O(n)$

Interview Recommendation: Mention all three, implement HashMap approach for $O(n)$ complexity

Related Files

String folder has related problems:

- [PermutationInString.java](#) - Similar to anagram detection
- [LongestUniqueCharSubstring.java](#) - Uses frequency map

Array folder:

- [ArrayElementFrequency.java](#) - Similar frequency concept
-

Code Patterns to Remember

Pattern 1: Count Frequency

```
Map<T, Integer> freq = new HashMap<>();
for (T item : collection) {
    freq.put(item, freq.getOrDefault(item, 0) + 1);
}
```

Pattern 2: Group by Key

```
Map<K, List<V>> groups = new HashMap<>();
for (V value : values) {
    K key = getKey(value);
    groups.computeIfAbsent(key, k -> new ArrayList<>()).add(value);
}
```

Pattern 3: Find Most Frequent

```
Map<T, Integer> freq = getFrequency(items);
T mostFrequent = freq.entrySet().stream()
    .max(Comparator.comparingInt(Map.Entry::getValue))
    .map(Map.Entry::getKey)
    .orElse(null);
```

Time Complexity Reference

Problem	Time	Space	Approach
Anagram Check	$O(n)$	$O(1)$	Array[26]
Valid Anagram	$O(n)$	$O(1)$	Freq count
Group Anagrams	$O(n \cdot m \log m)$	$O(n)$	Sort + HashMap
First Non-Rep	$O(n)$	$O(k)$	HashMap
Frequency Count	$O(n)$	$O(k)$	HashMap
Most Frequent	$O(n)$	$O(k)$	HashMap + Max

n = string/array length, m = average word length, k = unique elements

Practice Tips

When to use HashMap vs Array:

- **HashMap**: When you don't know character range
- **Array[26]**: When you know it's only a-z
- **Array[256]**: For extended ASCII
- **Array[128]**: For basic ASCII

Interview Strategy:

1. Start with simple sorting approach
2. Mention time complexity: $O(n \log n)$

3. Optimize to HashMap: $O(n)$
4. Further optimize to array: $O(n)$ with $O(1)$ space
5. Discuss which is best depending on context

Common Mistakes:

- Forgetting `getOrDefault()` method
- Not handling case sensitivity (uppercase vs lowercase)
- Not considering Unicode characters
- Inefficient iteration order

Generated: Interview Prep Package - HashMaps Folder

Code Solutions

ArrayElementFrequency.java

```
package programs.hashmaps;

import org.testng.Assert;

import java.util.HashMap;

/**
 * Array Element Frequency
 *
 * Problem Statement:
 * Given an array of integers, count the frequency of each element and return the
 * result.
 *
 * Constraints:
 * -  $1 \leq \text{arr.length} \leq 10^5$ 
 * -  $-10^9 \leq \text{arr}[i] \leq 10^9$ 
 */
public class ArrayElementFrequency {

    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 2, 1, 2, 3, 4, 2, 3};
        HashMap<Integer, Integer> frequencyMap = countFrequency(arr);
        Assert.assertEquals(frequencyMap.get(1).intValue(), 2);
        Assert.assertEquals(frequencyMap.get(2).intValue(), 4);
        Assert.assertEquals(frequencyMap.get(3).intValue(), 3);
        Assert.assertEquals(frequencyMap.get(4).intValue(), 2);
    }

    /**
     * APPROACH: HashMap (Optimal)
     *
     * Time Complexity:  $O(N)$ 
     * - Single pass through the array.
     */
}
```

```

    * Space Complexity: O(N)
    * - HashMap storing unique elements.
    */
    public static HashMap<Integer, Integer> countFrequency(int[] arr) {
        HashMap<Integer, Integer> frequencyMap = new HashMap<>();
        for (int element : arr) {
            frequencyMap.put(element, frequencyMap.getOrDefault(element, 0) + 1);
        }
        return frequencyMap;
    }
}

```

CharacterFrequency.java

```

package programs.hashmaps;

import org.testng.Assert;

/**
 * Character Frequency
 *
 * Problem Statement:
 * Given a string, calculate the frequency of each character and return the
 * results.
 *
 * Constraints:
 * - 1 <= str.length <= 10^5
 * - str consists of ASCII characters.
 */
public class CharacterFrequency {

    public static void main(String[] args) {
        int[] freq = calculateFrequency("Picture Perfect");
        Assert.assertEquals(freq['P'], 2);
        Assert.assertEquals(freq['i'], 1);
        Assert.assertEquals(freq['c'], 2);
    }

    /**
     * APPROACH: Frequency Array (Optimal for ASCII)
     *
     * Time Complexity: O(N)
     * - Single pass through the string.
     *
     * Space Complexity: O(1)
     * - Fixed size array of 256 for ASCII.
     */
    public static int[] calculateFrequency(String str) {
        int[] countArr = new int[256];
    }
}

```

```

        char[] arr = str.toCharArray();
        for (char c : arr) {
            countArr[c]++;
        }
        return countArr;
    }
}

```

FirstNonRepeatingCharacter.java

```

package programs.hashmaps;

import org.testng.Assert;

import java.util.HashMap;
import java.util.Map;

/**
 * LeetCode 387: First Unique Character in a String
 *
 * Problem Statement:
 * Given a string s, find the first non-repeating character in it and return its
 * index.
 * If it does not exist, return -1.
 *
 * Constraints:
 * - 1 <= s.length <= 10^5
 * - s consists of only lowercase English letters.
 */
public class FirstNonRepeatingCharacter {

    public static void main(String[] args) {
        Assert.assertEquals(nonRepCharWithMap("geeksforgeeks"), 5);
        Assert.assertEquals(nonRepChar("geeksforgeeks"), 5);
        Assert.assertEquals(nonRepCharBruteForce("geeksforgeeks"), 5);
    }

    /**
     * APPROACH 1: HashMap (Optimal for general case)
     *
     * Time Complexity: O(N)
     * - Two passes through the string.
     *
     * Space Complexity: O(N)
     * - HashMap storing character frequencies.
     */
    public static int nonRepCharWithMap(String str) {
        Map<Character, Integer> hm = new HashMap<>();
        for (Character c : str.toLowerCase().toCharArray()) {
            hm.put(c, hm.getOrDefault(c, 0) + 1);
        }
    }
}

```



```
    }
    for (int i = 0; i < str.length(); i++) {
        if (hm.get(str.charAt(i)) == 1) return i;
    }
    return -1;
}

/**
 * APPROACH 2: Frequency Array (Optimal for lowercase)
 *
 * Time Complexity: O(N)
 * - Two passes through the string.
 *
 * Space Complexity: O(1)
 * - Fixed size array of 26.
 */
public static int nonRepChar(String s) {
    int[] freq = new int[26];
    for (char c : s.toCharArray()) {
        freq[c - 'a']++;
    }
    for (int i = 0; i < s.length(); i++) {
        if (freq[s.charAt(i) - 'a'] == 1) {
            return i;
        }
    }
    return -1;
}

/**
 * APPROACH 3: Brute Force
 *
 * Time Complexity: O(N2)
 * - For each character, check all other characters.
 *
 * Space Complexity: O(1)
 * - Uses constant extra space.
 */
public static int nonRepCharBruteForce(String s) {
    for (int i = 0; i < s.length(); i++) {
        boolean isUnique = true;
        for (int j = 0; j < s.length(); j++) {
            if (i != j && s.charAt(i) == s.charAt(j)) {
                isUnique = false;
                break;
            }
        }
        if (isUnique) return i;
    }
    return -1;
}
}
```

GroupAnagrams.java

```
package programs.hashmaps;

import org.testng.Assert;

import java.util.*;

/**
 * LeetCode 49: Group Anagrams
 *
 * Problem Statement:
 * Given an array of strings strs, group the anagrams together. You can return the
 * answer in any order.
 * An anagram is a word or phrase formed by rearranging the letters of a different
 * word or phrase.
 *
 * Constraints:
 * - 1 <= strs.length <= 10^4
 * - 0 <= strs[i].length <= 100
 * - strs[i] consists of lowercase English letters.
 */
public class GroupAnagrams {
    public static void main(String[] args) {
        String[] arr = {"eat", "tea", "tan", "ate", "nat", "bat"};

        List<List<String>> result1 = groupAnagramsOptimized(arr.clone());
        Assert.assertEquals(result1.size(), 3);

        List<List<String>> result2 = groupAnagramsBruteForce(arr.clone());
        Assert.assertEquals(result2.size(), 3);
    }

    /**
     * APPROACH 1: HashMap with Sorted Keys (Optimal)
     *
     * Time Complexity: O(N * K log K)
     * - N strings, K is average string length for sorting.
     *
     * Space Complexity: O(N * K)
     * - HashMap storing all strings.
     */
    public static List<List<String>> groupAnagramsOptimized(String[] strs) {
        Map<String, List<String>> anagramMap = new HashMap<>();
        for (String str : strs) {
            char[] chars = str.toCharArray();
            Arrays.sort(chars);
            String key = new String(chars);
            anagramMap.computeIfAbsent(key, k -> new ArrayList<>()).add(str);
        }
        return new ArrayList<>(anagramMap.values());
    }
}
```

```

    }

    /**
     * APPROACH 2: Brute Force
     *
     * Time Complexity: O(N² * K log K)
     * - N strings, comparing each pair with sorting.
     *
     * Space Complexity: O(N * K)
     * - Result list storing all strings.
     */
    public static List<List<String>> groupAnagramsBruteForce(String[] strs) {
        List<List<String>> result = new ArrayList<>();
        for (int i = 0; i < strs.length; i++) {
            List<String> anagrams = new ArrayList<>();
            if (strs[i] != null) {
                char[] sortedWordArray = strs[i].toCharArray();
                Arrays.sort(sortedWordArray);
                anagrams.add(strs[i]);
                for (int j = i + 1; j < strs.length; j++) {
                    if (strs[j] != null) {
                        char[] sortedWordArray1 = strs[j].toCharArray();
                        Arrays.sort(sortedWordArray1);
                        if (Arrays.equals(sortedWordArray, sortedWordArray1)) {
                            anagrams.add(strs[j]);
                            strs[j] = null;
                        }
                    }
                }
                result.add(anagrams);
            }
        }
        return result;
    }
}

```

IntegerFrequency.java

```

package programs.hashmaps;

import org.testng.Assert;

/**
 * Integer Frequency (Without HashMap)
 *
 * Problem Statement:
 * Given an array of integers, count the frequency of each element without using
 * additional data structures like HashMap.
 *
 * Constraints:

```

```

* - 1 <= arr.length <= 10^4
* - -10^9 <= arr[i] <= 10^9
*/
public class IntegerFrequency {

    public static void main(String[] args) {
        int[] arr = new int[]{1, 2, 3, 1, 2, 3, 4, 44};
        countOccurrences(arr);
    }

    /**
     * APPROACH: Brute Force with Visited Array
     *
     * Time Complexity: O(N^2)
     * - Nested loops to count occurrences.
     *
     * Space Complexity: O(N)
     * - Visited array to track counted elements.
     */
    public static void countOccurrences(int[] arr) {
        boolean[] visited = new boolean[arr.length];
        for (int i = 0; i < arr.length; i++) {
            if (!visited[i]) {
                int count = 1;
                for (int j = i + 1; j < arr.length; j++) {
                    if (arr[i] == arr[j]) {
                        count++;
                        visited[j] = true;
                    }
                }
                System.out.println("Count of " + arr[i] + " is " + count);
            }
        }
    }
}

```

MaxMinCharacterFrequency.java

```

package programs.hashmaps;

import org.testng.Assert;

/**
 * Max Min Character Frequency
 *
 * Problem Statement:
 * Given a string, find the character with the maximum frequency and the character
 * with the minimum frequency (excluding characters that don't appear).
 *
 * Constraints:

```

```

* - 1 <= str.length <= 10^5
* - str consists of ASCII characters.
*/
public class MaxMinCharacterFrequency {

    public static void main(String[] args) {
        char[] result = maxMinOccurence("TeeeesstAutooooooooomationn");
        Assert.assertEquals(result[0], 'o');
        Assert.assertEquals(result[1], 'm');
    }

    /**
     * APPROACH: Frequency Array (Optimal for ASCII)
     *
     * Time Complexity: O(N)
     * - Single pass through the string.
     *
     * Space Complexity: O(1)
     * - Fixed size array of 256 for ASCII.
     */
    public static char[] maxMinOccurence(String str) {
        int[] counter = new int[256];
        for (char c : str.toCharArray()) {
            counter[c]++;
        }

        int maxCount = Integer.MIN_VALUE;
        int minCount = Integer.MAX_VALUE;
        char maxChar = ' ';
        char minChar = ' ';

        for (int i = 0; i < counter.length; i++) {
            if (counter[i] > maxCount) {
                maxCount = counter[i];
                maxChar = (char) i;
            }
            if (counter[i] >= 1 && counter[i] < minCount) {
                minCount = counter[i];
                minChar = (char) i;
            }
        }
        return new char[]{maxChar, minChar};
    }
}

```

SecondMostFrequentChar.java

```

package programs.hashmaps;

import org.testng.Assert;

```

```
/**
 * Second Most Frequent Character
 *
 * Problem Statement:
 * Given a string, find the second most frequent character in it.
 *
 * Constraints:
 * - 1 <= str.length <= 10^5
 * - str consists of lowercase English letters.
 */
public class SecondMostFrequentChar {

    public static void main(String[] args) {
        Assert.assertEquals(findSecondMostFrequent("banana"), 'n');
    }

    /**
     * APPROACH: Frequency Array (Optimal for lowercase)
     *
     * Time Complexity: O(N)
     * - Single pass through the string.
     *
     * Space Complexity: O(1)
     * - Fixed size array of 26.
     */
    public static char findSecondMostFrequent(String str) {
        char[] arr = str.toCharArray();
        int[] counter = new int[arr.length];

        for (int i = 0; i < arr.length; i++) {
            int loopCount = 1;
            for (int j = i + 1; j < arr.length; j++) {
                if (arr[i] == arr[j]) {
                    loopCount++;
                    counter[j] = -1;
                }
            }
            if (counter[i] != -1) {
                counter[i] = loopCount;
            }
        }

        int largest = 0;
        int secLar = 0;

        for (int i = 0; i < arr.length; i++) {
            if (counter[i] > counter[largest]) {
                secLar = largest;
                largest = i;
            } else if (counter[i] > counter[secLar] && counter[i] !=
counter[largest]) {
                secLar = i;
            }
        }
    }
}
```

```

    }

    return arr[secLar];
}
}

```

SecondMostFrequentString.java

```

package programs.hashmaps;

import org.testng.Assert;

import java.util.HashMap;
import java.util.Map;

/**
 * Second Most Frequent String
 *
 * Problem Statement:
 * Given a string of space-separated words, find the second most frequent word in
the string.
 * If there is no second most frequent word (i.e., all words have the same
frequency), return null.
 *
 * Constraints:
 * - 1 <= input.length <= 10^5
 * - Words consist of lowercase English letters.
 */
public class SecondMostFrequentString {

    public static void main(String[] args) {
        String input = "apple banana apple orange banana apple banana grape banana
orange orange";
        String secondMostFrequent = getSecondMostFrequent(input);
        Assert.assertEquals(secondMostFrequent, "orange");
    }

    /**
     * APPROACH: HashMap (Optimal)
     *
     * Time Complexity: O(N)
     * - Single pass through words.
     *
     * Space Complexity: O(N)
     * - HashMap storing word frequencies.
     */
    public static String getSecondMostFrequent(String input) {
        String[] words = input.split(" ");
    }

```

```

    Map<String, Integer> frequencyMap = new HashMap<>();
    for (String word : words) {
        if (!word.isEmpty()) {
            frequencyMap.put(word, frequencyMap.getOrDefault(word, 0) + 1);
        }
    }

    String secondMostFrequent = null;
    int secondMaxFrequency = 0;
    int maxFrequency = 0;

    for (Map.Entry<String, Integer> entry : frequencyMap.entrySet()) {
        String word = entry.getKey();
        int frequency = entry.getValue();

        if (frequency > maxFrequency) {
            secondMaxFrequency = maxFrequency;
            secondMostFrequent = word;
            maxFrequency = frequency;
        } else if (frequency > secondMaxFrequency && frequency < maxFrequency)
        {
            secondMaxFrequency = frequency;
            secondMostFrequent = word;
        }
    }
    return secondMostFrequent;
}

```

ValidAnagram.java

```

package programs.hashmaps;

import org.testng.Assert;

import java.util.Arrays;

/**
 * LeetCode 242: Valid Anagram
 *
 * Problem Statement:
 * Given two strings s and t, return true if t is an anagram of s, and false
 otherwise.
 * An anagram is a word or phrase formed by rearranging the letters of a different
 word or phrase.
 *
 * Constraints:
 * - 1 <= s.length, t.length <= 5 * 10^4
 * - s and t consist of lowercase English letters.

```



```
*/
public class ValidAnagram {

    public static void main(String[] args) {
        // Basic anagram check
        Assert.assertTrue(isAnagramSorting("anagram", "nagaram"));
        Assert.assertTrue(isAnagramFrequency("anagram", "nagaram"));

        // Anagram with spaces and case-insensitive
        Assert.assertTrue(isAnagramWithSpaces("Saurav is Singh Hexaware", "Singh
Saurav Hexaware is"));
    }

    /**
     * APPROACH 1: Sorting
     *
     * Time Complexity: O(N log N)
     * - Sorting both strings.
     *
     * Space Complexity: O(N)
     * - Character arrays for sorting.
     */
    public static boolean isAnagramSorting(String s, String t) {
        if (s.length() != t.length()) return false;

        char[] arrayS = s.toCharArray();
        char[] arrayT = t.toCharArray();

        Arrays.sort(arrayS);
        Arrays.sort(arrayT);

        return Arrays.equals(arrayS, arrayT);
    }

    /**
     * APPROACH 2: Frequency Array (Optimal)
     *
     * Time Complexity: O(N)
     * - Single pass through both strings.
     *
     * Space Complexity: O(1)
     * - Fixed size array of 26 for lowercase letters.
     */
    public static boolean isAnagramFrequency(String s, String t) {
        if (s.length() != t.length()) return false;

        int[] counts = new int[26];

        for (int i = 0; i < s.length(); i++) {
            counts[s.charAt(i) - 'a']++;
            counts[t.charAt(i) - 'a']--;
        }

        for (int count : counts) {
```

```
        if (count != 0) return false;
    }

    return true;
}

/**
 * APPROACH 3: Sorting with space/ignore handling
 *
 * Time Complexity: O(N log N)
 * - Sorting after preprocessing.
 *
 * Space Complexity: O(N)
 * - Character arrays for sorting.
 */
public static boolean isAnagramWithSpaces(String str1, String str2) {
    if (str1.length() != str2.length())
        return false;

    str1 = str1.replaceAll("\\s", "").toLowerCase();
    str2 = str2.replaceAll("\\s", "").toLowerCase();

    char[] arr1 = str1.toCharArray();
    char[] arr2 = str2.toCharArray();

    Arrays.sort(arr1);
    Arrays.sort(arr2);

    return Arrays.equals(arr1, arr2);
}
}
```

Topic: heaps

Notes & Concept Guide

Heaps / Priority Queue Problems - Interview Prep Guide

Overview

This folder contains **7 problems** focused on heap data structure applications, covering priority queue patterns and optimizations.

Problem Categories

1. K-Element Problems (2 problems)

- `KLargestElements.java` - Find K largest elements
- `TopKFrequentElements.java` - Find K most frequent elements

Technique: Min heap of size K **Time Complexity:** $O(n \log k)$, **Space Complexity:** $O(k)$

2. Merge Problems (1 problem)

- `MergeKSortedLists.java` - Merge K sorted linked lists

Technique: Min heap with linked list nodes **Interview Frequency:** Very Common

3. Median & Statistics (1 problem)

- `FindMedianFromDataStream.java` - Find median while adding numbers

Technique: Two heaps (max and min) **Time Complexity:** $O(\log n)$ per operation

4. Scheduling Problems (2 problems)

- `MeetingRoomsII.java` - Minimum rooms needed for meetings
- `TaskScheduler.java` - Minimum time with cooldown

Technique: Heap or sorting with sweep line

5. Sliding Window (1 problem)

- `SlidingWindowMaximum.java` - Find max in each sliding window

Technique: Deque heap **Interview Frequency:** Important

Key Techniques to Master

1. Min Heap for K Elements

```
PriorityQueue<Integer> minHeap = new PriorityQueue<>();
for (int num : nums) {
    minHeap.offer(num);
    if (minHeap.size() > k) {
        minHeap.poll(); // Remove smallest
    }
}
```

Use Case: K largest/smallest elements **Time:** $O(n \log k)$, **Space:** $O(k)$

2. Two Heaps for Median

```

PriorityQueue<Integer> maxHeap = new PriorityQueue<>((a, b) -> b - a); // Left
half
PriorityQueue<Integer> minHeap = new PriorityQueue<>(); // Right half
// Balance sizes and maintain max(left) <= min(right)

```

Use Case: Median from data stream **Time:** $O(\log n)$ per operation, **Space:** $O(n)$

3. Heap for Merging

```

PriorityQueue<ListNode> minHeap = new PriorityQueue<>((a, b) -> a.val - b.val);
for (ListNode list : lists) {
    if (list != null) minHeap.offer(list);
}
while (!minHeap.isEmpty()) {
    ListNode node = minHeap.poll();
    // Process node
    if (node.next != null) minHeap.offer(node.next);
}

```

Use Case: Merging sorted lists **Time:** $O(n \log k)$, **Space:** $O(k)$

4. Heap for Scheduling

```

PriorityQueue<int[]> maxHeap = new PriorityQueue<>((a, b) -> b[1] - a[1]);
// Sort by start time, use heap for end times
while (!maxHeap.isEmpty() && maxHeap.peek()[1] <= current[0]) {
    maxHeap.poll(); // Free up resource
}
maxHeap.offer(current);

```

Use Case: Meeting rooms, task scheduling **Time:** $O(n \log n)$, **Space:** $O(n)$

Interview Preparation Checklist

Easy Level (Start Here)

- ☐ `KLargestElements.java` - Basic heap usage
- ☐ `TopKFrequentElements.java` - Frequency + heap

Medium Level

- ☐ `MergeKSortedLists.java` - **MOST IMPORTANT**
- ☐ `FindMedianFromDataStream.java` - Two heaps pattern
- ☐ `MeetingRoomsII.java` - Scheduling with heap
- ☐ `TaskScheduler.java` - Greedy + heap

Hard Level

- ☐ [SlidingWindowMaximum.java](#) - Deque heap

Common Interview Questions

1. K Largest Elements

Problem: Find K largest elements in array **Solution:** Min heap of size K **See:** [KLargestElements.java](#)

2. Merge K Sorted Lists

Problem: Merge multiple sorted linked lists **Key Insight:** Use min heap to always get smallest element **See:** [MergeKSortedLists.java](#)

3. Median from Data Stream

Problem: Find median while adding numbers **Strategy:** Two heaps - max heap for lower half, min heap for upper half **See:** [FindMedianFromDataStream.java](#)

4. Meeting Rooms II

Problem: Minimum rooms for meetings **Technique:** Sort by start time, use heap for end times **See:** [MeetingRoomsII.java](#)

Time Complexity Summary

Problem	Time	Space	Technique
K Largest	$O(n \log k)$	$O(k)$	Min heap
Top K Frequent	$O(n \log k)$	$O(n)$	Frequency + heap
Merge K Lists	$O(n \log k)$	$O(k)$	Min heap
Find Median	$O(\log n)$	$O(n)$	Two heaps
Meeting Rooms II	$O(n \log n)$	$O(n)$	Heap
Task Scheduler	$O(n \log n)$	$O(n)$	Greedy + heap
Sliding Window Max	$O(n)$	$O(k)$	Deque heap

n = total elements, k = K value

Related Files

Array folder:

- Similar selection problems

Sorting folder:

- Heap sort concepts
-

Code Patterns to Remember

Pattern 1: K Elements with Min Heap

```
PriorityQueue<Integer> minHeap = new PriorityQueue<>();
for (int num : nums) {
    minHeap.offer(num);
    if (minHeap.size() > k) minHeap.poll();
}
```

Pattern 2: Two Heaps for Median

```
PriorityQueue<Integer> maxHeap = new PriorityQueue<>((a, b) -> b - a);
PriorityQueue<Integer> minHeap = new PriorityQueue<>();
// Balance: maxHeap.size() == minHeap.size() or maxHeap.size() == minHeap.size() + 1
```

Pattern 3: Merge with Heap

```
PriorityQueue<Node> heap = new PriorityQueue<>(comparator);
for (Node node : nodes) heap.offer(node);
while (!heap.isEmpty()) {
    Node current = heap.poll();
    result.add(current);
    if (current.next != null) heap.offer(current.next);
}
```

Practice Tips

When to use Heap:

- **K elements:** When you need top/bottom K elements
- **Merging:** When merging sorted sequences
- **Median:** When need median from streaming data
- **Scheduling:** When optimizing resource allocation

Min Heap vs Max Heap:

- **Min heap:** For K largest elements (removes smallest)
- **Max heap:** For K smallest elements (removes largest)

- **Default:** Java PriorityQueue is min heap

Common Mistakes:

- Using wrong heap type (min vs max)
- Not handling empty heap cases
- Inefficient heap operations ($O(n)$ instead of $O(\log n)$)
- Not considering space optimization

Generated: Interview Prep Package - Heaps Folder

Code Solutions

FindMedianFromDataStream.java

```
package programs.heaps;

import org.testng.Assert;

import java.util.PriorityQueue;

/**
 * LeetCode 295: Find Median from Data Stream
 *
 * Problem Statement:
 * The median is the middle value in an ordered integer list. If the size of the
 * list is even,
 * there is no middle value and the median is the mean of the two middle values.
 * Implement the MedianFinder class with addNum and findMedian methods.
 *
 * Constraints:
 * -  $-10^5 \leq \text{num} \leq 10^5$ 
 * - There will be at least 1 element in the data structure before calling
 * findMedian.
 * - At most  $5 * 10^4$  calls will be made to addNum and findMedian.
 */
public class FindMedianFromDataStream {

    private PriorityQueue<Integer> maxHeap;
    private PriorityQueue<Integer> minHeap;

    public FindMedianFromDataStream() {
        maxHeap = new PriorityQueue<>((a, b) -> b - a);
        minHeap = new PriorityQueue<>();
    }

    public static void main(String[] args) {
        FindMedianFromDataStream medianFinder = new FindMedianFromDataStream();
        medianFinder.addNum(1);
        medianFinder.addNum(2);
        Assert.assertEquals(medianFinder.findMedian(), 1.5);
    }
}
```

```

        medianFinder.addNum(3);
        Assert.assertEquals(medianFinder.findMedian(), 2.0);
    }

    /**
     * APPROACH: Two Heaps (Optimal)
     *
     * Time Complexity: O(log N) for addNum, O(1) for findMedian
     * - Heap operations for insertion.
     *
     * Space Complexity: O(N)
     * - Storing all elements in two heaps.
     */
    public void addNum(int num) {
        maxHeap.offer(num);

        if (!minHeap.isEmpty() && maxHeap.peek() > minHeap.peek()) {
            minHeap.offer(maxHeap.poll());
        }

        if (maxHeap.size() > minHeap.size() + 1) {
            minHeap.offer(maxHeap.poll());
        } else if (minHeap.size() > maxHeap.size()) {
            maxHeap.offer(minHeap.poll());
        }
    }

    public double findMedian() {
        if (maxHeap.size() == minHeap.size()) {
            return (maxHeap.peek() + minHeap.peek()) / 2.0;
        }
        return maxHeap.peek();
    }
}

```

KLargestElements.java

```

package programs.heaps;

import org.testng.Assert;

import java.util.Arrays;
import java.util.PriorityQueue;

/**
 * K Largest Elements
 *
 * Problem Statement:
 * Given an integer array nums and an integer k, return the k largest elements in
 * the array.

```



```
*
* Constraints:
* - 1 <= nums.length <= 10^5
* - -10^4 <= nums[i] <= 10^4
* - 1 <= k <= nums.length
*/
public class K Largest Elements {

    public static void main(String[] args) {
        int[] nums = {3, 2, 1, 5, 6, 4};
        int k = 2;
        int[] result = findKLargestHeap(nums, k);
        Arrays.sort(result);
        Assert.assertEquals(result, new int[]{5, 6});
    }

    /**
     * APPROACH 1: Min Heap (Optimal for small k)
     *
     * Time Complexity: O(N log K)
     * - Heap operations for N elements.
     *
     * Space Complexity: O(K)
     * - Heap of size k.
     */
    public static int[] findKLargestHeap(int[] nums, int k) {
        if (nums == null || nums.length == 0 || k <= 0) return new int[0];

        PriorityQueue<Integer> minHeap = new PriorityQueue<>();

        for (int num : nums) {
            minHeap.offer(num);
            if (minHeap.size() > k) {
                minHeap.poll();
            }
        }

        int[] result = new int[k];
        for (int i = k - 1; i >= 0; i--) {
            result[i] = minHeap.poll();
        }

        return result;
    }

    /**
     * APPROACH 2: Sorting (Optimal for large k)
     *
     * Time Complexity: O(N log N)
     * - Sorting the entire array.
     *
     * Space Complexity: O(1)
     * - In-place sort (if using arrays.sort).
     */
}
```

```

    public static int[] findKLargestSort(int[] nums, int k) {
        if (nums == null || nums.length == 0 || k <= 0) return new int[0];

        Arrays.sort(nums);
        int[] result = new int[k];
        for (int i = 0; i < k; i++) {
            result[i] = nums[nums.length - 1 - i];
        }

        return result;
    }
}

```

MeetingRoomsII.java

```

package programs.heaps;

import org.testng.Assert;

import java.util.Arrays;
import java.util.PriorityQueue;

/**
 * LeetCode 253: Meeting Rooms II
 *
 * Problem Statement:
 * Given an array of meeting time intervals intervals where intervals[i] =
 [start_i, end_i],
 * return the minimum number of conference rooms required.
 *
 * Constraints:
 * - 1 <= intervals.length <= 10^4
 * - 0 <= start_i < end_i <= 10^6
 */
public class MeetingRoomsII {

    public static void main(String[] args) {
        int[][] intervals1 = {{0, 30}, {5, 10}, {15, 20}};
        Assert.assertEquals(minMeetingRoomsHeap(intervals1), 2);
        Assert.assertEquals(minMeetingRoomsSort(intervals1), 2);

        int[][] intervals2 = {{7, 10}, {2, 4}};
        Assert.assertEquals(minMeetingRoomsHeap(intervals2), 1);
    }

    /**
     * APPROACH 1: Min Heap (Optimal)
     *
     * Time Complexity: O(N log N)
     * - Sorting O(N log N), heap operations O(N log N).
     */
}

```

```
*
* Space Complexity: O(N)
* - Heap storing end times.
*/
public static int minMeetingRoomsHeap(int[][] intervals) {
    if (intervals == null || intervals.length == 0) return 0;

    Arrays.sort(intervals, (a, b) -> a[0] - b[0]);

    PriorityQueue<Integer> minHeap = new PriorityQueue<>();
    minHeap.offer(intervals[0][1]);

    for (int i = 1; i < intervals.length; i++) {
        if (intervals[i][0] >= minHeap.peek()) {
            minHeap.poll();
        }
        minHeap.offer(intervals[i][1]);
    }

    return minHeap.size();
}

/**
 * APPROACH 2: Sorting with Two Pointers
 *
 * Time Complexity: O(N log N)
 * - Sorting both arrays.
 *
 * Space Complexity: O(N)
 * - Arrays for start and end times.
 */
public static int minMeetingRoomsSort(int[][] intervals) {
    if (intervals == null || intervals.length == 0) return 0;

    int[] starts = new int[intervals.length];
    int[] ends = new int[intervals.length];

    for (int i = 0; i < intervals.length; i++) {
        starts[i] = intervals[i][0];
        ends[i] = intervals[i][1];
    }

    Arrays.sort(starts);
    Arrays.sort(ends);

    int rooms = 0;
    int endPointer = 0;

    for (int i = 0; i < starts.length; i++) {
        if (starts[i] < ends[endPointer]) {
            rooms++;
        } else {
            endPointer++;
        }
    }
}
```

```

    }

    return rooms;
}
}

```

MergeKSortedLists.java

```

package programs.heaps;

import org.testng.Assert;

import java.util.PriorityQueue;

/**
 * LeetCode 23: Merge K Sorted Lists
 *
 * Problem Statement:
 * You are given an array of k linked-lists lists, each linked-list is sorted in
 * ascending order.
 * Merge all the linked-lists into one sorted linked-list and return it.
 *
 * Constraints:
 * - k == lists.length
 * - 0 <= k <= 10^4
 * - 0 <= lists[i].length <= 500
 * - -10^4 <= lists[i][j] <= 10^4
 * - lists[i] is sorted in ascending order.
 */
public class MergeKSortedLists {

    public static class ListNode {
        int val;
        ListNode next;

        ListNode() {
        }

        ListNode(int val) {
            this.val = val;
        }

        ListNode(int val, ListNode next) {
            this.val = val;
            this.next = next;
        }
    }

    public static void main(String[] args) {
        ListNode list1 = new ListNode(1, new ListNode(4, new ListNode(5)));
    }
}

```

```

        ListNode list2 = new ListNode(1, new ListNode(3, new ListNode(4)));
        ListNode list3 = new ListNode(2, new ListNode(6));

        ListNode[] lists = {list1, list2, list3};
        ListNode merged = mergeKListsHeap(lists);

        StringBuilder sb = new StringBuilder();
        while (merged != null) {
            sb.append(merged.val);
            merged = merged.next;
        }
        Assert.assertEquals(sb.toString(), "11234456");
    }

    /**
     * APPROACH 1: Min Heap (Optimal)
     *
     * Time Complexity: O(N log K)
     * - N total nodes, K lists.
     *
     * Space Complexity: O(K)
     * - Heap of size K.
     */
    public static ListNode mergeKListsHeap(ListNode[] lists) {
        if (lists == null || lists.length == 0) return null;

        PriorityQueue<ListNode> minHeap = new PriorityQueue<>((a, b) -> a.val -
b.val);

        for (ListNode list : lists) {
            if (list != null) {
                minHeap.offer(list);
            }
        }

        ListNode dummy = new ListNode();
        ListNode current = dummy;

        while (!minHeap.isEmpty()) {
            ListNode node = minHeap.poll();
            current.next = node;
            current = current.next;

            if (node.next != null) {
                minHeap.offer(node.next);
            }
        }

        return dummy.next;
    }

    /**
     * APPROACH 2: Divide and Conquer
     *

```

```

    * Time Complexity: O(N log K)
    * - Merging pairs of lists.
    *
    * Space Complexity: O(1)
    * - Constant extra space (ignoring recursion stack).
    */
    public static ListNode mergeKListsDivideConquer(ListNode[] lists) {
        if (lists == null || lists.length == 0) return null;
        return mergeKListsHelper(lists, 0, lists.length - 1);
    }

    private static ListNode mergeKListsHelper(ListNode[] lists, int left, int
right) {
        if (left == right) return lists[left];
        if (left > right) return null;

        int mid = left + (right - left) / 2;
        ListNode l1 = mergeKListsHelper(lists, left, mid);
        ListNode l2 = mergeKListsHelper(lists, mid + 1, right);
        return mergeTwoLists(l1, l2);
    }

    private static ListNode mergeTwoLists(ListNode l1, ListNode l2) {
        ListNode dummy = new ListNode();
        ListNode current = dummy;

        while (l1 != null && l2 != null) {
            if (l1.val <= l2.val) {
                current.next = l1;
                l1 = l1.next;
            } else {
                current.next = l2;
                l2 = l2.next;
            }
            current = current.next;
        }

        current.next = l1 != null ? l1 : l2;
        return dummy.next;
    }
}

```

SlidingWindowMaximum.java

```

package programs.heaps;

import org.testng.Assert;

import java.util.*;

```

```
/**
 * LeetCode 239: Sliding Window Maximum
 *
 * Problem Statement:
 * You are given an array of integers nums, there is a sliding window of size k
moving from the
 * very left of the array to the very right. You can only see the k numbers in the
window. Return the
 * max sliding window.
 *
 * Constraints:
 * - 1 <= nums.length <= 10^5
 * - -10^4 <= nums[i] <= 10^4
 * - 1 <= k <= nums.length
 */
public class SlidingWindowMaximum {

    public static void main(String[] args) {
        int[] nums = {1, 3, -1, -3, 5, 3, 6, 7};
        int k = 3;
        int[] result = maxSlidingWindowHeap(nums, k);
        Assert.assertEquals(result, new int[]{3, 3, 5, 5, 6, 7});
    }

    /**
     * APPROACH 1: Max Heap
     *
     * Time Complexity: O(N log K)
     * - Heap operations for N elements.
     *
     * Space Complexity: O(K)
     * - Heap of size k.
     */
    public static int[] maxSlidingWindowHeap(int[] nums, int k) {
        if (nums == null || nums.length == 0 || k <= 0) return new int[0];

        int n = nums.length;
        int[] result = new int[n - k + 1];

        PriorityQueue<int[]> maxHeap = new PriorityQueue<>((a, b) -> b[0] - a[0]);

        for (int i = 0; i < n; i++) {
            maxHeap.offer(new int[]{nums[i], i});

            while (maxHeap.peek()[1] <= i - k) {
                maxHeap.poll();
            }

            if (i >= k - 1) {
                result[i - k + 1] = maxHeap.peek()[0];
            }
        }

        return result;
    }
}
```

```

    }

    /**
     * APPROACH 2: Deque (Optimal)
     *
     * Time Complexity: O(N)
     * - Each element is added and removed at most once.
     *
     * Space Complexity: O(K)
     * - Deque of size k.
     */
    public static int[] maxSlidingWindowDeque(int[] nums, int k) {
        if (nums == null || nums.length == 0 || k <= 0) return new int[0];

        int n = nums.length;
        int[] result = new int[n - k + 1];
        Deque<Integer> deque = new ArrayDeque<>();

        for (int i = 0; i < n; i++) {
            // Remove elements outside the window
            while (!deque.isEmpty() && deque.peekFirst() <= i - k) {
                deque.pollFirst();
            }

            // Remove smaller elements from the back
            while (!deque.isEmpty() && nums[deque.peekLast()] <= nums[i]) {
                deque.pollLast();
            }

            deque.offerLast(i);

            if (i >= k - 1) {
                result[i - k + 1] = nums[deque.peekFirst()];
            }
        }

        return result;
    }
}

```

TaskScheduler.java

```

package programs.heaps;

import org.testng.Assert;

import java.util.*;

/**
 * LeetCode 621: Task Scheduler

```



```

*
* Problem Statement:
* Given a characters array tasks, representing the tasks a CPU needs to do, where
each letter
* represents a different task. Tasks could be done in any order. Each task is
done in one unit of time.
* For each unit of time, the CPU could complete either one task or just be idle.
However, there is a
* non-negative integer n that represents the cooldown period between two same
tasks. Return the least
* number of units of times that the CPU will take to finish all the given tasks.
*
* Constraints:
* - 1 <= tasks.length <= 10^4
* - tasks[i] is an uppercase English letter.
* - 0 <= n <= 100
*/
public class TaskScheduler {

    public static void main(String[] args) {
        char[] tasks1 = {'A', 'A', 'A', 'B', 'B', 'B'};
        Assert.assertEquals(leastIntervalHeap(tasks1, 2), 8);
        Assert.assertEquals(leastIntervalMath(tasks1, 2), 8);

        char[] tasks2 = {'A', 'A', 'A', 'B', 'B', 'B'};
        Assert.assertEquals(leastIntervalHeap(tasks2, 0), 6);
    }

    /**
     * APPROACH 1: Max Heap (Simulation)
     *
     * Time Complexity: O(N log N)
     * - Building frequency map O(N), heap operations O(N log N).
     *
     * Space Complexity: O(N)
     * - Frequency map and heap.
     */
    public static int leastIntervalHeap(char[] tasks, int n) {
        if (tasks == null || tasks.length == 0) return 0;

        Map<Character, Integer> frequencyMap = new HashMap<>();
        for (char task : tasks) {
            frequencyMap.put(task, frequencyMap.getOrDefault(task, 0) + 1);
        }

        PriorityQueue<Integer> maxHeap = new PriorityQueue<>((a, b) -> b - a);
        maxHeap.addAll(frequencyMap.values());

        int intervals = 0;

        while (!maxHeap.isEmpty()) {
            int cycle = n + 1;
            int tasksExecuted = 0;
            List<Integer> temp = new ArrayList<>();

```

```

        while (cycle-- > 0 && !maxHeap.isEmpty()) {
            int freq = maxHeap.poll();
            if (freq > 1) {
                temp.add(freq - 1);
            }
            tasksExecuted++;
        }

        maxHeap.addAll(temp);
        intervals += maxHeap.isEmpty() ? tasksExecuted : n + 1;
    }

    return intervals;
}

/**
 * APPROACH 2: Mathematical Formula (Optimal)
 *
 * Time Complexity: O(N)
 * - Single pass to count frequencies.
 *
 * Space Complexity: O(1)
 * - Fixed size array of 26 for uppercase letters.
 */
public static int leastIntervalMath(char[] tasks, int n) {
    if (tasks == null || tasks.length == 0) return 0;

    int[] freq = new int[26];
    for (char task : tasks) {
        freq[task - 'A']++;
    }

    Arrays.sort(freq);

    int maxFreq = freq[25];
    int idleTime = (maxFreq - 1) * n;

    for (int i = 24; i >= 0 && idleTime > 0; i--) {
        idleTime -= Math.min(maxFreq - 1, freq[i]);
    }

    idleTime = Math.max(0, idleTime);

    return tasks.length + idleTime;
}
}

```

TopKFrequentElements.java

```
package programs.heaps;

import org.testng.Assert;

import java.util.*;

/**
 * LeetCode 347: Top K Frequent Elements
 *
 * Problem Statement:
 * Given an integer array nums and an integer k, return the k most frequent
 * elements.
 * You may return the answer in any order.
 *
 * Constraints:
 * - 1 <= nums.length <= 10^5
 * - -10^4 <= nums[i] <= 10^4
 * - k is in the range [1, the number of unique elements in the array].
 */
public class TopKFrequentElements {

    public static void main(String[] args) {
        int[] nums = {1, 1, 1, 2, 2, 3};
        int k = 2;
        int[] result = topKFrequentHeap(nums, k);
        Arrays.sort(result);
        Assert.assertEquals(result, new int[]{1, 2});
    }

    /**
     * APPROACH 1: Min Heap (Optimal for small k)
     *
     * Time Complexity: O(N log K)
     * - Building frequency map O(N), heap operations O(N log K).
     *
     * Space Complexity: O(N)
     * - Frequency map and heap.
     */
    public static int[] topKFrequentHeap(int[] nums, int k) {
        if (nums == null || nums.length == 0 || k <= 0) return new int[0];

        Map<Integer, Integer> frequencyMap = new HashMap<>();
        for (int num : nums) {
            frequencyMap.put(num, frequencyMap.getOrDefault(num, 0) + 1);
        }

        PriorityQueue<Integer> minHeap = new PriorityQueue<>(
            (a, b) -> frequencyMap.get(a) - frequencyMap.get(b)
        );

        for (int num : frequencyMap.keySet()) {
            minHeap.offer(num);
        }
    }
}
```

```
        if (minHeap.size() > k) {
            minHeap.poll();
        }
    }

    int[] result = new int[k];
    for (int i = k - 1; i >= 0; i--) {
        result[i] = minHeap.poll();
    }

    return result;
}

/**
 * APPROACH 2: Bucket Sort (Optimal for large k)
 *
 * Time Complexity: O(N)
 * - Single pass through array and buckets.
 *
 * Space Complexity: O(N)
 * - Frequency map and buckets.
 */
public static int[] topKFrequentBucketSort(int[] nums, int k) {
    if (nums == null || nums.length == 0 || k <= 0) return new int[0];

    Map<Integer, Integer> frequencyMap = new HashMap<>();
    for (int num : nums) {
        frequencyMap.put(num, frequencyMap.getOrDefault(num, 0) + 1);
    }

    List<Integer>[] buckets = new List[nums.length + 1];
    for (int num : frequencyMap.keySet()) {
        int freq = frequencyMap.get(num);
        if (buckets[freq] == null) {
            buckets[freq] = new ArrayList<>();
        }
        buckets[freq].add(num);
    }

    int[] result = new int[k];
    int index = 0;
    for (int i = buckets.length - 1; i >= 0 && index < k; i--) {
        if (buckets[i] != null) {
            for (int num : buckets[i]) {
                result[index++] = num;
                if (index == k) break;
            }
        }
    }

    return result;
}
```

```
}
```

Topic: maths

Notes & Concept Guide

Math Problems - Interview Prep Guide

Overview

This folder contains **10 problems** focused on mathematical algorithms and number theory problems.

Problem Categories

1. Fibonacci & Series (2 problems)

- `FibonacciSeries.java` - Generate Fibonacci sequence
- `ClimbingStairs.java` - DP variation of Fibonacci

Concepts:

- Recursive approach: $O(2^n)$ time
- Memoization: $O(n)$ time, $O(n)$ space
- Iterative: $O(n)$ time, $O(1)$ space
- Matrix exponentiation: $O(\log n)$ time

2. Factorials & Permutations (1 problem)

- `Factorial.java` - Calculate factorial

Approaches:

- Recursive: Simple but can overflow for large n
- Iterative: Better for large numbers
- BigInteger: For very large factorials

3. Number Classification (3 problems)

- `ArmstrongNumbers.java` - Check if Armstrong number (narcissistic)
 - Example: $153 = 1^3 + 5^3 + 3^3$
- `PrimeNumber.java` - Check if prime
 - Optimization: Check only up to $\sqrt{n}$
- `LeapYear.java` - Determine if leap year

Time Complexity:

- Prime check: $O(\sqrt{n})$
- Armstrong: $O(\log n)$ for digit count
- Leap year: $O(1)$

4. Number Operations (3 problems)

- `AddBinary.java` - Binary addition
- `IntegerReverse.java` - Reverse integer digits
- `PlusOne.java` - Add 1 to number array

Techniques:

- Bit manipulation for binary operations
- Modulo/division for digit operations
- Array manipulation for number representation

5. advanced Math (2 problems)

- `SquareRoot.java` - Calculate square root (Newton's method, binary search)
- `CoinChangeMinimum.java` - Minimum coins for amount (DP/Greedy)

Key Algorithms to Master

1. Fibonacci Sequence

Problem: Generate or find Nth Fibonacci number

Approaches:

1. Recursive (Naive): $O(2^n)$ - AVOID
2. Memoization: $O(n)$ time, $O(n)$ space
3. Iterative: $O(n)$ time, $O(1)$ space ← BEST
4. Matrix: $O(\log n)$ time - ADVANCED

See: `FibonacciSeries.java`, `ClimbingStairs.java`

2. Prime Number Check

Optimization: Only check divisors up to $\sqrt{n}$

```
boolean isPrime(int n) {
    if (n <= 1) return false;
    if (n <= 3) return true;
    if (n % 2 == 0 || n % 3 == 0) return false;
    for (int i = 5; i * i <= n; i += 6) {
        if (n % i == 0 || n % (i + 2) == 0) return false;
    }
}
```

```
    return true;  
}
```

Time: $O(\sqrt{n})$

3. Armstrong Number (Narcissistic)

Check if sum of digits each raised to power of digit count equals number

```
// 153: 1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153
```

See: [ArmstrongNumbers.java](#)

4. Square Root Calculation

Three approaches:

- **Binary Search:** $O(\log n)$
- **Newton's Method:** $O(\log n)$, faster convergence
- **Built-in:** `Math.sqrt()` - $O(1)$

5. Digit Manipulation

```
// Reverse: 12345 → 54321  
// Add one to array: [1,2,3] → [1,2,4]  
// Check leading zeros
```

Interview Preparation Checklist

Easy Level (Start Here)

- ☐ [Factorial.java](#) - Basic factorial
- ☐ [PrimeNumber.java](#) - Prime checking
- ☐ [LeapYear.java](#) - Simple logic
- ☐ [IntegerReverse.java](#) - Digit manipulation
- ☐ [AddBinary.java](#) - String conversion

Medium Level

- ☐ [FibonacciSeries.java](#) - Optimization techniques
- ☐ [SquareRoot.java](#) - Binary search variant
- ☐ [PlusOne.java](#) - Array manipulation
- ☐ [ArmstrongNumbers.java](#) - Digit extraction

Hard Level

- ☐ `ClimbingStairs.java` - DP optimization
 - ☐ `CoinChangeMinimum.java` - DP/Greedy analysis
 - ☐ Multiple Fibonacci approaches - Compare implementations
-

Common Interview Questions

1. Fibonacci Number

Problem: Find Nth Fibonacci number efficiently **Solutions:**

Approach 1: Iterative (BEST for interviews)

- Time: $O(n)$, Space: $O(1)$

Approach 2: Memoization

- Time: $O(n)$, Space: $O(n)$

Approach 3: Matrix Exponentiation (advanced)

- Time: $O(\log n)$, Space: $O(1)$

Follow-up: "How would you handle very large n ?"

2. Prime Number Validation

Problem: Check if number is prime **Optimization:** Check only up to $\sqrt{n}$ **Why:** If $n = a \times b$ and $a > \sqrt{n}$, then $b < \sqrt{n}$ (contradiction)

3. Square Root without Using Library

Problem: Calculate $\sqrt{n}$ without `Math.sqrt()` **Solutions:**

- Binary Search: Best for programs
- Newton's Method: Faster convergence
- Bit shifting: For perfect squares

See: `SquareRoot.java`

4. Number Reversal

Problem: Reverse digits of integer (handle negative, overflow) **Edge Cases:**

- Negative numbers
- Leading zeros after reversal
- Integer overflow

5. Climbing Stairs (DP)

Problem: How many ways to climb n stairs (1 or 2 steps per move) **Solution:** $f(n) = f(n-1) + f(n-2)$ - it's Fibonacci! **See:** `ClimbingStairs.java`

Related Math Concepts

Bit Manipulation (for Binary)

- AND, OR, XOR operations
- Bit shifting (>> and <<)
- Check if power of 2: $n \& (n-1) == 0$

Modular Arithmetic

- Division by hand using modulo
- Check divisibility: $n \% d == 0$
- Remainders in array problems

Combinatorics

- Factorial: $n!$
- Permutations: $P(n,r) = n! / (n-r)!$
- Combinations: $C(n,r) = n! / (r! \times (n-r)!)$

Time Complexity Reference

Problem	Time	Space	Method
Fibonacci	$O(n)$	$O(1)$	Iterative
Prime Check	$O(\sqrt{n})$	$O(1)$	Trial division
Armstrong	$O(\log n)$	$O(1)$	Digit extraction
Factorial	$O(n)$	$O(1)$	Iterative
Binary Add	$O(\max(m,n))$	$O(1)$	String processing
Reverse	$O(\log n)$	$O(1)$	Modulo division
Square Root	$O(\log n)$	$O(1)$	Binary search

Code Patterns

Pattern 1: Check Divisibility

```
for (int i = 2; i * i <= n; i++) {
    if (n % i == 0) {
        // n is divisible by i
    }
}
```

Pattern 2: Extract Digits

```
while (n > 0) {  
    int digit = n % 10;  
    // Process digit  
    n /= 10;  
}
```

Pattern 3: Fibonacci Loop

```
int a = 0, b = 1;  
for (int i = 0; i < n; i++) {  
    int next = a + b;  
    a = b;  
    b = next;  
}
```

Pattern 4: Binary Search for Root

```
long left = 0, right = x;  
while (left <= right) {  
    long mid = left + (right - left) / 2;  
    if (mid * mid == x) return mid;  
    else if (mid * mid < x) left = mid + 1;  
    else right = mid - 1;  
}  
return right; // floor of square root
```

Interview Strategy

For Fibonacci Question:

1. Explain recursive approach (2^n)
2. Optimize with memoization (n)
3. Further optimize to iterative (n time, 1 space)
4. Show code for iterative approach
5. If time: mention matrix exponentiation for large n

For Any Math Problem:

1. Clarify constraints (range of input)
2. Ask about overflow handling
3. Discuss edge cases
4. Optimize if possible
5. Code with explanation

Common Mistakes:

- Forgot to handle negative numbers
 - Integer overflow (use long for large calculations)
 - Off-by-one errors in loops
 - Not checking edge cases (0, 1, negative)
-

Practice Progression

1. **Start:** Factorial, LeapYear - Simple logic
 2. **Next:** PrimeNumber - Optimization technique
 3. **Then:** IntegerReverse, AddBinary - Digit manipulation
 4. **advanced:** Fibonacci variants, SquareRoot - Algorithm optimization
 5. **Expert:** MinCoins, ClimbingStairs - DP approach
-

Generated: Interview Prep Package - Math Folder

Code Solutions

AddBinary.java

```
package programs.maths;

import org.testng.Assert;

/**
 * LeetCode 67: Add Binary
 *
 * Problem Statement:
 * Given two binary strings a and b, return their sum as a binary string.
 *
 * Constraints:
 * - 1 <= a.length, b.length <= 10^4
 * - a and b consist only of '0' or '1' characters.
 * - Each string does not contain leading zeros except for the zero itself.
 */
public class AddBinary {

    public static void main(String[] args) {
        Assert.assertEquals(addBinary("11", "1"), "100");
        Assert.assertEquals(addBinary("1010", "1011"), "10101");
    }

    /**
     * APPROACH: Bit Manipulation with Carry (Optimal)
     *
     * Time Complexity: O(max(N, M))
     * - N and M are lengths of the two strings.
     */
}
```

```

    * Space Complexity: O(max(N, M))
    * - Result string.
    */
    public static String addBinary(String a, String b) {
        StringBuilder result = new StringBuilder();
        int carry = 0;
        int i = a.length() - 1;
        int j = b.length() - 1;

        while (i >= 0 || j >= 0 || carry > 0) {
            int sum = carry;
            if (i >= 0) {
                sum += a.charAt(i--) - '0';
            }
            if (j >= 0) {
                sum += b.charAt(j--) - '0';
            }

            result.append(sum % 2);
            carry = sum / 2;
        }

        return result.reverse().toString();
    }
}

```

ArmstrongNumbers.java

```

package programs.maths;

import org.testng.Assert;

/**
 * Armstrong Number
 *
 * Problem Statement:
 * An Armstrong number (also known as narcissistic number) is a number that is
 * equal to
 * the sum of its own digits each raised to the power of the number of digits.
 * Write a function to check if a given number is an Armstrong number.
 *
 * Constraints:
 * - 0 <= num <= 10^9
 */
public class ArmstrongNumbers {

    public static void main(String[] args) {
        Assert.assertTrue(isArmstrong(153));
        Assert.assertTrue(isArmstrong(370));
    }
}

```

```
    Assert.assertTrue(isArmstrong(371));
    Assert.assertTrue(isArmstrong(0));
    Assert.assertTrue(isArmstrong(1));
    Assert.assertFalse(isArmstrong(123));
    Assert.assertFalse(isArmstrong(100));
}

/**
 * APPROACH: Digit Extraction with Early Termination (Optimal)
 *
 * Time Complexity: O(D)
 * - D is the number of digits in the number.
 *
 * Space Complexity: O(1)
 * - Constant extra space.
 */
static boolean isArmstrong(int num) {
    int originalNum = num;
    int digits = countDigits(num);
    int pow = 0;

    while (num != 0) {
        int rem = num % 10;
        pow += (int) Math.pow(rem, digits);
        if (pow > originalNum) {
            return false; // Early termination
        }
        num /= 10;
    }

    return pow == originalNum;
}

/**
 * APPROACH 2: String Conversion (Alternative)
 *
 * Time Complexity: O(D)
 * - D is the number of digits.
 *
 * Space Complexity: O(D)
 * - String representation of the number.
 */
static boolean isArmstrongString(int num) {
    String numStr = String.valueOf(num);
    int digits = numStr.length();
    int sum = 0;

    for (char c : numStr.toCharArray()) {
        int digit = c - '0';
        sum += Math.pow(digit, digits);
        if (sum > num) {
            return false;
        }
    }
}
```

```

        return sum == num;
    }

    static int countDigits(int num) {
        int digits = 0;
        while (num != 0) {
            num /= 10;
            digits++;
        }
        return digits;
    }
}

```

CoinChangeMinimum.java

```

package programs.maths;

import org.testng.Assert;

/**
 * Coin Change Minimum (Greedy Approach)
 *
 * Problem Statement:
 * Given an array of coin denominations and a target value, find the minimum
 * number of coins
 * required to make up that value. You may assume that you have an infinite supply
 * of each coin type.
 * Note: This greedy approach works only for canonical coin systems (like standard
 * currency).
 *
 * Constraints:
 * - 1 <= coins.length <= 100
 * - 1 <= coins[i] <= 10^4
 * - 1 <= value <= 10^4
 */
public class CoinChangeMinimum {

    public static void main(String[] args) {
        int[] coinArray = {1, 2, 5, 10};
        Assert.assertEquals(minCoinsGreedy(coinArray, 13), 3);
        Assert.assertEquals(minCoinsGreedy(coinArray, 20), 2);
    }

    /**
     * APPROACH: Greedy (Optimal for canonical coin systems)
     *
     * Time Complexity: O(N)
     * - Single pass through sorted coins.
     *
     * Space Complexity: O(1)
     */
}

```

```

    * - Constant extra space.
    */
    private static int minCoinsGreedy(int[] coinArray, int value) {
        int count = 0;
        for (int i = coinArray.length - 1; i >= 0; i--) {
            while (value >= coinArray[i]) {
                value -= coinArray[i];
                count++;
            }
        }
        return count;
    }
}

```

Factorial.java

```

package programs.maths;

import org.testng.Assert;

/**
 * Factorial
 *
 * Problem Statement:
 * Calculate the factorial of a given number.
 * Factorial of n (n!) is the product of all positive integers from 1 to n.
 *
 * Constraints:
 * - 0 <= n <= 20 (to avoid integer overflow)
 */
public class Factorial {

    public static void main(String[] args) {
        Assert.assertEquals(factorialRecursive(5), 120);
        Assert.assertEquals(factorialIterative(5), 120);
    }

    /**
     * APPROACH 1: Recursion
     *
     * Time Complexity: O(N)
     * - N recursive calls.
     *
     * Space Complexity: O(N)
     * - Recursion stack depth.
     */
    public static int factorialRecursive(int n) {
        if (n == 0) {
            return 1;
        }
    }
}

```

```

        return n * factorialRecursive(n - 1);
    }

    /**
     * APPROACH 2: Iterative (Optimal Space)
     *
     * Time Complexity: O(N)
     * - Single loop from 1 to n.
     *
     * Space Complexity: O(1)
     * - Constant extra space.
     */
    public static int factorialIterative(int n) {
        int result = 1;
        for (int i = 2; i <= n; i++) {
            result *= i;
        }
        return result;
    }
}

```

FibonacciSeries.java

```

package programs.maths;

import org.testng.Assert;

/**
 * Fibonacci Series
 *
 * Problem Statement:
 * Generate the first n numbers of the Fibonacci series.
 * The Fibonacci sequence is a series where each number is the sum of the two
 * preceding ones.
 *
 * Constraints:
 * - 1 <= n <= 50
 */
public class FibonacciSeries {

    public static void main(String[] args) {
        int[] result = fibonacciDP(10);
        Assert.assertEquals(result[9], 34);
    }

    /**
     * APPROACH 1: Dynamic Programming (Optimal)
     *
     * Time Complexity: O(N)
     */
}

```



```
* - Single pass through the sequence.
*
* Space Complexity: O(N)
* - Array to store the sequence.
*/
public static int[] fibonacciDP(int n) {
    if (n <= 0) return new int[0];
    if (n == 1) return new int[]{0};

    int[] fib = new int[n];
    fib[0] = 0;
    fib[1] = 1;

    for (int i = 2; i < n; i++) {
        fib[i] = fib[i - 1] + fib[i - 2];
    }

    return fib;
}

/**
 * APPROACH 2: Brute Force (Recursion)
 *
 * Time Complexity: O(2^N)
 * - Exponential due to repeated calculations.
 *
 * Space Complexity: O(N)
 * - Recursion stack depth.
 */
public static int fibonacciRecursive(int n) {
    if (n <= 1) return n;
    return fibonacciRecursive(n - 1) + fibonacciRecursive(n - 2);
}

/**
 * APPROACH 3: Constant Space (Optimal Space)
 *
 * Time Complexity: O(N)
 * - Single pass through the sequence.
 *
 * Space Complexity: O(1)
 * - Constant extra space.
 */
public static int[] fibonacciConstantSpace(int n) {
    if (n <= 0) return new int[0];
    if (n == 1) return new int[]{0};

    int[] fib = new int[n];
    fib[0] = 0;
    fib[1] = 1;

    int prev2 = 0, prev1 = 1;
    for (int i = 2; i < n; i++) {
        fib[i] = prev1 + prev2;
```

```

        prev2 = prev1;
        prev1 = fib[i];
    }

    return fib;
}
}

```

IntegerReverse.java

```

package programs.maths;

import org.testng.Assert;

/**
 * LeetCode 7: Reverse Integer
 *
 * Problem Statement:
 * Given a signed 32-bit integer x, return x with its digits reversed. If
reversing x causes the value
 * to go outside the signed 32-bit integer range  $[-2^{31}, 2^{31} - 1]$ , then return 0.
 *
 * Constraints:
 * -  $-2^{31} \leq x \leq 2^{31} - 1$ 
 */
public class IntegerReverse {

    public static void main(String[] args) {
        Assert.assertEquals(reverseInteger(12345), 54321);
        Assert.assertEquals(reverseInteger(123), 321);
        Assert.assertEquals(reverseInteger(-123), -321);
    }

    /**
     * APPROACH: Mathematical Reversal (Optimal)
     *
     * Time Complexity:  $O(\log N)$ 
     * - Number of digits in the integer.
     *
     * Space Complexity:  $O(1)$ 
     * - Constant extra space.
     */
    static int reverseInteger(int num) {
        int rev = 0;
        while (num != 0) {
            int digit = num % 10;
            num = num / 10;

            // Check for overflow before updating rev

```

```

        if (rev > Integer.MAX_VALUE / 10 || (rev == Integer.MAX_VALUE / 10 &&
digit > 7)) {
            return 0;
        }
        if (rev < Integer.MIN_VALUE / 10 || (rev == Integer.MIN_VALUE / 10 &&
digit < -8)) {
            return 0;
        }

        rev = rev * 10 + digit;
    }
    return rev;
}
}

```

LeapYear.java

```

package programs.maths;

import org.testng.Assert;

/**
 * Leap Year
 *
 * Problem Statement:
 * Write a function to determine if a given year is a leap year.
 * A leap year is divisible by 4, except for century years which must be divisible
by 400.
 *
 * Constraints:
 * - 1 <= year <= 10^4
 */
public class LeapYear {

    public static void main(String[] args) {
        Assert.assertFalse(isLeapYear(1900));
        Assert.assertTrue(isLeapYear(2000));
        Assert.assertTrue(isLeapYear(2020));
        Assert.assertFalse(isLeapYear(2021));
    }

    /**
     * APPROACH: Mathematical Check (Optimal)
     *
     * Time Complexity: O(1)
     * - Constant time operations.
     *
     * Space Complexity: O(1)
     * - Constant extra space.
     */
}

```

```

    */
    public static boolean isLeapYear(int year) {
        return ((year % 400 == 0) || ((year % 4 == 0) && (year % 100 != 0)));
    }
}

```

PlusOne.java

```

package programs.maths;

import org.testng.Assert;
import org.testng.annotations.Test;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/**
 * LeetCode 66: Plus One
 *
 * Problem Statement:
 * Given a non-negative integer represented as an array of digits, add one to the
integer.
 * The digits are stored such that the most significant digit is at the head of
the list, and each
 * element in the array contains a single digit. You may assume the integer does
not contain any
 * leading zero, except the number 0 itself.
 *
 * Constraints:
 * - 1 <= digits.length <= 100
 * - 0 <= digits[i] <= 9
 * - digits does not contain any leading 0, except the number 0 itself.
 */
public class PlusOne {

    public static void main(String[] args) {
        Assert.assertEquals(addOne(new int[]{1, 2, 4}), new int[]{1, 2, 5});
        Assert.assertEquals(addOne(new int[]{9, 9, 9}), new int[]{1, 0, 0, 0});
    }

    /**
     * APPROACH: Carry Propagation (Optimal)
     *
     * Time Complexity: O(N)
     * - Single pass through the array.
     *
     * Space Complexity: O(N)
     * - Result list in worst case.
     */
}

```

```

public static int[] addOne(int[] arr) {
    List<Integer> result = new ArrayList<>();
    int carry = 1;
    for (int i = arr.length - 1; i >= 0; i--) {
        int plusOne = arr[i] + carry;
        if (plusOne > 9) {
            result.add(0);
            carry = plusOne / 10;
        } else {
            result.add(plusOne);
            carry = 0;
        }
    }
    if (carry > 0) {
        result.add(carry);
    }
    Collections.reverse(result);
    return result.stream().mapToInt(i -> i).toArray();
}
}

```

PrimeNumber.java

```

package programs.maths;

import org.testng.Assert;

/**
 * Prime Number
 *
 * Problem Statement:
 * Given an integer, determine if it is a prime number. A prime number is a number
 * greater
 * than 1 that has no positive divisors other than 1 and itself.
 *
 * Constraints:
 * -  $-10^4 \leq \text{num} \leq 10^4$ 
 */
public class PrimeNumber {

    public static void main(String[] args) {
        Assert.assertTrue(isPrime(23));
        Assert.assertFalse(isPrime(4));
        Assert.assertTrue(isPrime(2));
        Assert.assertFalse(isPrime(1));
    }

    /**
     * APPROACH: Square Root Optimization (Optimal)
     */
}

```

```

    * Time Complexity: O(sqrt(N))
    * - Check divisibility up to sqrt(n).
    *
    * Space Complexity: O(1)
    * - Constant extra space.
    */
    static boolean isPrime(int num) {
        if (num <= 1) {
            return false;
        }

        for (int i = 2; i <= Math.sqrt(num); i++) {
            if (num % i == 0) {
                return false;
            }
        }

        return true;
    }

    /**
    * APPROACH 2: Brute Force
    *
    * Time Complexity: O(N)
    * - Check divisibility up to n-1.
    *
    * Space Complexity: O(1)
    * - Constant extra space.
    */
    static boolean isPrimeBruteForce(int num) {
        if (num <= 1) {
            return false;
        }

        for (int i = 2; i < num; i++) {
            if (num % i == 0) {
                return false;
            }
        }

        return true;
    }
}

```

SquareRoot.java

```

package programs.maths;

import org.testng.Assert;

```

```
/**
 * LeetCode 69: Sqrt(x)
 *
 * Problem Statement:
 * Given a non-negative integer x, return the square root of x rounded down to the
nearest integer.
 * The returned integer should be non-negative as well.
 *
 * Constraints:
 * -  $0 \leq x \leq 2^{31} - 1$ 
 */
public class SquareRoot {

    public static void main(String[] args) {
        Assert.assertEquals(mySqrtBinarySearch(9), 3);
        Assert.assertEquals(mySqrtBinarySearch(4), 2);
        Assert.assertEquals(mySqrtBinarySearch(8), 2);
    }

    /**
     * APPROACH 1: Binary Search (Optimal)
     *
     * Time Complexity:  $O(\log N)$ 
     * - Binary search on the range  $[1, x]$ .
     *
     * Space Complexity:  $O(1)$ 
     * - Constant extra space.
     */
    public static int mySqrtBinarySearch(int x) {
        if (x == 0) {
            return 0;
        }

        long left = 1;
        long right = x;
        long ans = 1;

        while (left <= right) {
            long mid = (left + right) / 2;
            if (mid * mid <= x) {
                ans = mid;
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
        return (int) ans;
    }

    /**
     * APPROACH 2: Brute Force
     *
     * Time Complexity:  $O(\sqrt{N})$ 
     */
}
```

```
* - Linear search from 1 to sqrt(x).
*
* Space Complexity: O(1)
* - Constant extra space.
*/
public static int mySqrtBruteForce(int x) {
    if (x == 0) return 0;

    int i = 1;
    while (i <= x / i) {
        i++;
    }
    return i - 1;
}
```

Topic: searching

Code Solutions

BinarySearch.java

```
package programs.searching;

import org.testng.Assert;

import java.util.Arrays;

/**
 * Binary Search
 *
 * Problem Statement:
 * Implement binary search to find a target element in a sorted array.
 * Return the index if found, otherwise return -1.
 *
 * Constraints:
 * - 1 <= arr.length <= 10^4
 * - -10^4 <= arr[i] <= 10^4
 * - arr is sorted in ascending order.
 */
public class BinarySearch {

    public static void main(String[] args) {
        int[] arr = {2, 5, 6, 7, 9};
        Assert.assertEquals(binarySearch(arr, 7), 3);
        Assert.assertEquals(binarySearch(arr, 8), -1);
    }
}
```



```
/**
 * APPROACH 1: Iterative Binary Search (Optimal)
 *
 * Time Complexity:  $O(\log N)$ 
 * - Halving the search space each iteration.
 *
 * Space Complexity:  $O(1)$ 
 * - Constant extra space.
 */
static int binarySearch(int[] arr, int num) {
    int left = 0;
    int right = arr.length - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (arr[mid] == num) {
            return mid;
        } else if (arr[mid] > num) {
            right = mid - 1;
        } else {
            left = mid + 1;
        }
    }

    return -1;
}

/**
 * APPROACH 2: Recursive Binary Search
 *
 * Time Complexity:  $O(\log N)$ 
 * - Halving the search space each recursion.
 *
 * Space Complexity:  $O(\log N)$ 
 * - Recursion stack depth.
 */
static int binarySearchRecursive(int[] arr, int num) {
    return binarySearchHelper(arr, num, 0, arr.length - 1);
}

private static int binarySearchHelper(int[] arr, int num, int left, int right)
{
    if (left > right) return -1;

    int mid = left + (right - left) / 2;

    if (arr[mid] == num) {
        return mid;
    } else if (arr[mid] > num) {
        return binarySearchHelper(arr, num, left, mid - 1);
    } else {
        return binarySearchHelper(arr, num, mid + 1, right);
    }
}
```

```

    }
}

/**
 * APPROACH 3: Linear Search (Brute Force)
 *
 * Time Complexity: O(N)
 * - Scanning through the entire array.
 *
 * Space Complexity: O(1)
 * - Constant extra space.
 */
static int linearSearch(int[] arr, int num) {
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] == num) {
            return i;
        }
    }
    return -1;
}
}

```

KthLargestElement.java

```

package programs.searching;

import org.testng.Assert;

import java.util.Arrays;
import java.util.PriorityQueue;

/**
 * Kth Largest Element
 *
 * Problem Statement:
 * Given an array of integers and a number k, find the kth largest element in the array.
 *
 * Constraints:
 * - 1 <= arr.length <= 10^4
 * - -10^4 <= arr[i] <= 10^4
 * - 1 <= k <= arr.length
 */
public class KthLargestElement {

    public static void main(String[] args) {
        int[] arr = {3, 2, 1, 5, 6, 4, 7};
        Assert.assertEquals(kthLargestHeap(arr, 3), 5);
        Assert.assertEquals(kthLargestSort(arr, 3), 5);
    }
}

```

```
/**
 * APPROACH 1: Min Heap (Optimal for small k)
 *
 * Time Complexity:  $O(N \log K)$ 
 * - Heap operations for N elements.
 *
 * Space Complexity:  $O(K)$ 
 * - Heap of size k.
 */
public static int kthLargestHeap(int[] arr, int k) {
    PriorityQueue<Integer> minHeap = new PriorityQueue<>();
    for (int val : arr) {
        minHeap.add(val);
        if (minHeap.size() > k) {
            minHeap.poll();
        }
    }
    return minHeap.peek();
}

/**
 * APPROACH 2: Sorting (Optimal for large k)
 *
 * Time Complexity:  $O(N \log N)$ 
 * - Sorting the entire array.
 *
 * Space Complexity:  $O(1)$ 
 * - In-place sort (if using arrays.sort()).
 */
public static int kthLargestSort(int[] arr, int k) {
    Arrays.sort(arr);
    return arr[arr.length - k];
}

/**
 * APPROACH 3: QuickSelect (Optimal average case)
 *
 * Time Complexity:  $O(N)$  average,  $O(N^2)$  worst case
 * - Partition-based selection.
 *
 * Space Complexity:  $O(1)$ 
 * - In-place partitioning.
 */
public static int kthLargestQuickSelect(int[] arr, int k) {
    return quickSelect(arr, 0, arr.length - 1, arr.length - k);
}

private static int quickSelect(int[] arr, int left, int right, int k) {
    if (left == right) return arr[left];

    int pivotIndex = partition(arr, left, right);

    if (k == pivotIndex) {

```

```
        return arr[k];
    } else if (k < pivotIndex) {
        return quickSelect(arr, left, pivotIndex - 1, k);
    } else {
        return quickSelect(arr, pivotIndex + 1, right, k);
    }
}

private static int partition(int[] arr, int left, int right) {
    int pivot = arr[right];
    int i = left;

    for (int j = left; j < right; j++) {
        if (arr[j] <= pivot) {
            swap(arr, i, j);
            i++;
        }
    }
    swap(arr, i, right);
    return i;
}

private static void swap(int[] arr, int i, int j) {
    int temp = arr[i];
    arr[i] = arr[j];
    arr[j] = temp;
}
}
```

Topic: sorting

Code Solutions

BubbleSort.java

```
package programs.sorting;

import org.testng.Assert;

import java.util.Arrays;

/**
 * Bubble Sort
 *
 * Problem Statement:
 * Implement bubble sort algorithm to sort an array of integers in ascending
 * order.
 */
```

```
* Bubble sort repeatedly steps through the list, compares adjacent elements, and
swaps them if they are in the wrong order.
*
* Constraints:
* - 1 <= arr.length <= 10^4
* - -10^4 <= arr[i] <= 10^4
*/
public class BubbleSort {

    public static void main(String[] args) {
        int[] array = {9, 8, 7, 4, 5, 6, 2, 1, 3};
        int[] sorted = bubbleSort(array.clone());
        Assert.assertEquals(sorted, new int[]{1, 2, 3, 4, 5, 6, 7, 8, 9});
    }

    /**
     * APPROACH 1: Bubble Sort with Optimization
     *
     * Time Complexity:  $O(N^2)$ 
     * - Nested loops through the array.
     *
     * Space Complexity:  $O(1)$ 
     * - In-place sorting.
     */
    public static int[] bubbleSort(int[] array) {
        int n = array.length;
        for (int i = 0; i < n - 1; i++) {
            boolean swapped = false;
            for (int j = 0; j < n - i - 1; j++) {
                if (array[j] > array[j + 1]) {
                    int temp = array[j];
                    array[j] = array[j + 1];
                    array[j + 1] = temp;
                    swapped = true;
                }
            }
            if (!swapped) break;
        }
        return array;
    }

    /**
     * APPROACH 2: Built-in Sort (Optimal)
     *
     * Time Complexity:  $O(N \log N)$ 
     * - Uses dual-pivot quicksort.
     *
     * Space Complexity:  $O(\log N)$ 
     * - Stack space for quicksort.
     */
    public static int[] builtInSort(int[] array) {
        Arrays.sort(array);
        return array;
    }
}
```

```
}
```

SortCharArray.java

```
package programs.sorting;

import org.testng.Assert;

import java.util.Arrays;

/**
 * Sort Character Array
 *
 * Problem Statement:
 * Given a string, sort its characters in ascending order.
 *
 * Constraints:
 * - 1 <= str.length <= 10^4
 * - str consists of lowercase English letters.
 */
public class SortCharArray {

    public static void main(String[] args) {
        Assert.assertEquals(sortStringBubble("saurav"), "aarsuv");
        Assert.assertEquals(sortStringBuiltIn("saurav"), "aarsuv");
    }

    /**
     * APPROACH 1: Bubble Sort
     *
     * Time Complexity: O(N^2)
     * - Nested loops through the array.
     *
     * Space Complexity: O(N)
     * - Character array.
     */
    public static String sortStringBubble(String input) {
        char[] arr = input.toCharArray();

        for (int i = 0; i < arr.length - 1; i++) {
            for (int j = 0; j < arr.length - i - 1; j++) {
                if (arr[j] > arr[j + 1]) {
                    char temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
        }
    }
}
```

```

        return new String(arr);
    }

    /**
     * APPROACH 2: Built-in Sort (Optimal)
     *
     * Time Complexity: O(N log N)
     * - Uses dual-pivot quicksort.
     *
     * Space Complexity: O(N)
     * - Character array.
     */
    public static String sortStringBuiltIn(String input) {
        char[] arr = input.toCharArray();
        Arrays.sort(arr);
        return new String(arr);
    }
}

```

SortHashMapByValue.java

```

package programs.sorting;

import org.testng.Assert;

import java.util.*;

/**
 * Sort HashMap By Value
 *
 * Problem Statement:
 * Given a HashMap, sort it by its values in both ascending and descending order.
 *
 * Constraints:
 * - 1 <= map.size() <= 10^4
 * - Values are comparable integers.
 */
public class SortHashMapByValue {

    public static void main(String[] args) {
        HashMap<String, Integer> unsortedMap = new HashMap<>();
        unsortedMap.put("apple", 10);
        unsortedMap.put("banana", 5);
        unsortedMap.put("cherry", 15);
        unsortedMap.put("date", 2);
        unsortedMap.put("elderberry", 10);

        Map<String, Integer> sortedMapAscending = sortByValue(unsortedMap);
    }
}

```

```

        Assert.assertEquals(sortedMapAscending.size(), 5);

        Map<String, Integer> sortedMapDescending =
sortByValueDescending(unsortedMap);
        Assert.assertEquals(sortedMapDescending.size(), 5);
    }

    /**
     * APPROACH 1: Stream API (Optimal)
     *
     * Time Complexity: O(N log N)
     * - Sorting the entry list.
     *
     * Space Complexity: O(N)
     * - New LinkedHashMap.
     */
    public static <K, V extends Comparable<? super V>> Map<K, V>
sortByValue(Map<K, V> map) {
        List<Map.Entry<K, V>> list = new ArrayList<>(map.entrySet());
        list.sort(Map.Entry.comparingByValue());
        Map<K, V> sortedMap = new LinkedHashMap<>();
        for (Map.Entry<K, V> entry : list) {
            sortedMap.put(entry.getKey(), entry.getValue());
        }
        return sortedMap;
    }

    /**
     * APPROACH 2: Stream API with Reverse
     *
     * Time Complexity: O(N log N)
     * - Sorting the entry list.
     *
     * Space Complexity: O(N)
     * - New LinkedHashMap.
     */
    public static <K, V extends Comparable<? super V>> Map<K, V>
sortByValueDescending(Map<K, V> map) {
        List<Map.Entry<K, V>> list = new ArrayList<>(map.entrySet());
        list.sort(Map.Entry.<K, V>comparingByValue().reversed());
        Map<K, V> sortedMap = new LinkedHashMap<>();
        for (Map.Entry<K, V> entry : list) {
            sortedMap.put(entry.getKey(), entry.getValue());
        }
        return sortedMap;
    }
}

```

Topic: stack

Notes & Concept Guide

Stack Problems - Interview Prep Guide

Overview

This folder contains **8 problems** focused on stack data structure applications, covering fundamental to advanced techniques.

Problem Categories

1. Basic Stack Operations (2 problems)

- `ValidBrackets.java` - Check if parentheses/brackets/braces are balanced
- `MinStack.java` - Stack that returns minimum element in $O(1)$

Time Complexity: $O(n)$ for ValidBrackets, $O(1)$ for MinStack operations **Space Complexity:** $O(n)$

2. Monotonic Stack Problems (3 problems)

- `DailyTemperatures.java` - Find next warmer temperature for each day
- `NextGreaterElement.java` - Find next greater element (circular array)
- `LargestRectangleInHistogram.java` - Find largest rectangular area in histogram

Technique: Maintain stack in increasing/decreasing order **Interview Frequency:** Very Common

3. Expression Evaluation (2 problems)

- `EvaluateRPN.java` - Evaluate Reverse Polish Notation expressions
- `DecodeString.java` - Decode strings with nested number patterns

Applications: Calculator implementations, string parsing

4. Greedy + Stack (1 problem)

- `RemoveKDigits.java` - Remove K digits to get smallest number

Technique: Use stack to maintain monotonic property

Key Techniques to Master

1. Monotonic Stack Pattern

```
Stack<Integer> stack = new Stack<>();
for (int i = 0; i < n; i++) {
    while (!stack.isEmpty() && nums[i] > nums[stack.peek()]) {
        // Process previous elements
        stack.pop();
    }
}
```

```
    stack.push(i);  
}
```

Use Case: Next greater/smaller element, histogram problems **Time:** $O(n)$, **Space:** $O(n)$

2. Stack for Expression Parsing

```
Stack<Character> stack = new Stack<>();  
for (char c : expression.toCharArray()) {  
    if (isOperator(c)) {  
        // Process operators  
    } else {  
        stack.push(c);  
    }  
}
```

Use Case: RPN evaluation, string decoding **Time:** $O(n)$, **Space:** $O(n)$

3. Auxiliary Stack for Tracking

```
Stack<Integer> mainStack = new Stack<>();  
Stack<Integer> minStack = new Stack<>();  
// Track minimums alongside main operations
```

Use Case: MinStack, tracking additional properties **Time:** $O(1)$ per operation, **Space:** $O(n)$

Interview Preparation Checklist

Easy Level (Start Here)

- ☐ [ValidBrackets.java](#) - Basic stack usage
- ☐ [MinStack.java](#) - Auxiliary stack pattern
- ☐ [ImplementStackUsingQueue.java](#) - Data structure design

Medium Level

- ☐ [DailyTemperatures.java](#) - Monotonic stack
- ☐ [EvaluateRPN.java](#) - Expression evaluation
- ☐ [DecodeString.java](#) - Nested pattern handling
- ☐ [NextGreaterElement.java](#) - Circular array variant

Hard Level

- ☐ [LargestRectangleInHistogram.java](#) - Complex monotonic stack
- ☐ [RemoveKDigits.java](#) - Greedy + stack

Common Interview Questions

1. Check for Balanced Parentheses

Problem: Determine if string has balanced brackets **Solution:** Use stack to match opening/closing pairs **See:** [ValidBrackets.java](#)

2. Find Next Greater Element

Problem: For each element, find next greater element **Key Insight:** Use monotonic decreasing stack **See:** [NextGreaterElement.java](#)

3. Evaluate Mathematical Expressions

Problem: Evaluate expressions in RPN format **Strategy:** Stack for operands, pop for operations **See:** [EvaluateRPN.java](#)

4. Largest Rectangle in Histogram

Problem: Find maximum rectangular area **Technique:** Stack to maintain increasing heights **See:** [LargestRectangleInHistogram.java](#)

Time Complexity Summary

Problem	Time	Space	Technique
Valid Brackets	$O(n)$	$O(n)$	Simple stack
Min Stack	$O(1)$	$O(n)$	Auxiliary stack
Daily Temperatures	$O(n)$	$O(n)$	Monotonic stack
Largest Rectangle	$O(n)$	$O(n)$	Monotonic stack
Evaluate RPN	$O(n)$	$O(n)$	Expression parsing
Decode String	$O(n)$	$O(n)$	Nested stack
Next Greater	$O(n)$	$O(n)$	Monotonic stack
Remove K Digits	$O(n)$	$O(n)$	Greedy + stack

Related Files

Array folder:

- Similar two-pointer techniques in some problems

Graphs folder:

- Stack-based DFS implementations
-

Code Patterns to Remember

Pattern 1: Monotonic Stack

```
Stack<Integer> stack = new Stack<>();
for (int i = 0; i < n; i++) {
    while (!stack.isEmpty() && condition) {
        process(stack.pop());
    }
    stack.push(i);
}
```

Pattern 2: Expression Evaluation

```
Stack<Integer> stack = new Stack<>();
for (String token : tokens) {
    if (isOperator(token)) {
        int b = stack.pop();
        int a = stack.pop();
        stack.push(apply(a, b, token));
    } else {
        stack.push(Integer.parseInt(token));
    }
}
```

Pattern 3: Nested Structure Handling

```
Stack<Integer> countStack = new Stack<>();
Stack<StringBuilder> stringStack = new Stack<>();
// Handle nested patterns by maintaining separate stacks
```

Practice Tips

When to use Stack:

- **LIFO order:** When you need to process in reverse order
- **Nested structures:** Parentheses, brackets, nested expressions
- **Monotonic sequences:** Finding next greater/smaller elements
- **Backtracking:** When you need to undo operations

Interview Strategy:

1. Start with simple stack operations
2. Identify if problem needs monotonic property

3. Consider space-time tradeoffs
4. Handle edge cases (empty stack, single element)

Common Mistakes:

- Forgetting to check if stack is empty before peek/pop
- Not handling circular arrays correctly
- Inefficient nested loops instead of monotonic stack
- Not considering stack overflow for large inputs

Generated: Interview Prep Package - Stack Folder

Code Solutions

DailyTemperatures.java

```
package programs.stack;

import org.testng.Assert;

import java.util.Stack;

/**
 * LeetCode 739: Daily Temperatures
 *
 * Problem Statement:
 * Given an array of integers temperatures represents the daily temperatures,
 * return an array answer such that answer[i] is the number of days you have to
wait
 * after the ith day to get a warmer temperature. If there is no future day for
which
 * this is possible, keep answer[i] == 0 instead.
 *
 * Constraints:
 * - 1 <= temperatures.length <= 10^5
 * - 30 <= temperatures[i] <= 100
 */
public class DailyTemperatures {

    public static void main(String[] args) {
        int[] temperatures = {73, 74, 75, 71, 69, 72, 76, 73};
        int[] result = dailyTemperatures(temperatures);
        Assert.assertEquals(result, new int[]{1, 1, 4, 2, 1, 1, 0, 0});
    }

    /**
     * APPROACH: Monotonic Stack (Optimal)
     *
     * Time Complexity: O(N)
     * - Each element is pushed and popped at most once.
     */
}
```

```

    * Space Complexity: O(N)
    * - Stack storing indices.
    */
    public static int[] dailyTemperatures(int[] temperatures) {
        int n = temperatures.length;
        int[] answer = new int[n];
        Stack<Integer> stack = new Stack<>();

        for (int i = 0; i < n; i++) {
            while (!stack.isEmpty() && temperatures[i] >
temperatures[stack.peek()]) {
                int prevIndex = stack.pop();
                answer[prevIndex] = i - prevIndex;
            }
            stack.push(i);
        }

        return answer;
    }
}

```

DecodeString.java

```

package programs.stack;

import org.testng.Assert;

import java.util.Stack;

/**
 * LeetCode 394: Decode String
 *
 * Problem Statement:
 * Given an encoded string, return its decoded string.
 * The encoding rule is: k[encoded_string], where the encoded_string inside the
square brackets
 * is being repeated exactly k times. Note that k is guaranteed to be a positive
integer.
 *
 * Constraints:
 * - 1 <= s.length <= 30
 * - s consists of lowercase English letters, digits, and square brackets '['].
 * - s is a valid encoded string.
 */
public class DecodeString {

    public static void main(String[] args) {
        Assert.assertEquals(decodeString("3[a]2[bc]"), "aaabcbcb");
        Assert.assertEquals(decodeString("3[a2[c]]"), "accaccacc");
        Assert.assertEquals(decodeString("2[abc]3[cd]ef"), "abcbcbccdcdef");
    }
}

```

```

    }

    /**
     * APPROACH: Two Stacks (Optimal)
     *
     * Time Complexity: O(N)
     * - Single pass through the string.
     *
     * Space Complexity: O(N)
     * - Stacks for counts and strings.
     */
    public static String decodeString(String s) {
        Stack<Integer> countStack = new Stack<>();
        Stack<StringBuilder> stringStack = new Stack<>();
        StringBuilder currentString = new StringBuilder();
        int k = 0;

        for (char ch : s.toCharArray()) {
            if (Character.isDigit(ch)) {
                k = k * 10 + (ch - '0');
            } else if (ch == '[') {
                countStack.push(k);
                stringStack.push(currentString);
                currentString = new StringBuilder();
                k = 0;
            } else if (ch == ']') {
                int count = countStack.pop();
                StringBuilder decodedString = stringStack.pop();
                for (int i = 0; i < count; i++) {
                    decodedString.append(currentString);
                }
                currentString = decodedString;
            } else {
                currentString.append(ch);
            }
        }

        return currentString.toString();
    }
}

```

EvaluateRPN.java

```

package programs.stack;

import org.testng.Assert;

import java.util.Stack;

/**

```

```

* LeetCode 150: Evaluate Reverse Polish Notation
*
* Problem Statement:
* Evaluate the value of an arithmetic expression in Reverse Polish Notation.
* Valid operators are: +, -, *, /. Each operand may be an integer or another
expression.
* Note: Division between two integers should truncate toward zero.
*
* Constraints:
* - 1 <= tokens.length <= 10^4
* - tokens[i] is either an operator: "+", "-", "*", or "/", or an integer.
*/
public class EvaluateRPN {

    public static void main(String[] args) {
        String[] tokens1 = {"2", "1", "+", "3", "*"};
        Assert.assertEquals(evalRPN(tokens1), 9);

        String[] tokens2 = {"4", "13", "5", "/", "+"};
        Assert.assertEquals(evalRPN(tokens2), 6);

        String[] tokens3 = {"10", "6", "9", "3", "+", "-11", "*", "/", "*", "17",
"+", "5", "+"};
        Assert.assertEquals(evalRPN(tokens3), 22);
    }

    /**
     * APPROACH: Stack (Optimal)
     *
     * Time Complexity: O(N)
     * - Single pass through tokens.
     *
     * Space Complexity: O(N)
     * - Stack storing operands.
     */
    public static int evalRPN(String[] tokens) {
        Stack<Integer> stack = new Stack<>();

        for (String token : tokens) {
            if (isOperator(token)) {
                int b = stack.pop();
                int a = stack.pop();
                int result = applyOperator(token, a, b);
                stack.push(result);
            } else {
                stack.push(Integer.parseInt(token));
            }
        }

        return stack.pop();
    }

    private static boolean isOperator(String token) {
        return token.equals("+") || token.equals("-") || token.equals("*") ||

```



```

token.equals("/");
    }

    private static int applyOperator(String operator, int a, int b) {
        switch (operator) {
            case "+":
                return a + b;
            case "-":
                return a - b;
            case "*":
                return a * b;
            case "/":
                return a / b;
            default:
                throw new IllegalArgumentException("Unknown operator: " +
operator);
        }
    }
}

```

ImplementStackUsingQueue.java

```

package programs.stack;

import org.testng.Assert;

import java.util.LinkedList;
import java.util.Queue;

/**
 * Implement Stack Using Queue
 *
 * Problem Statement:
 * Implement a last-in-first-out (LIFO) stack using only two queues.
 * The implemented stack should support all the functions of a normal stack (push,
top, pop, and empty).
 *
 * Constraints:
 * - 1 <= x <= 9
 * - At most 100 calls will be made to push, pop, top, and empty.
 */
public class ImplementStackUsingQueue {

    private Queue<Integer> queue1;
    private Queue<Integer> queue2;

    public ImplementStackUsingQueue() {
        queue1 = new LinkedList<>();
        queue2 = new LinkedList<>();
    }
}

```

```
public static void main(String[] args) {
    ImplementStackUsingQueue stack = new ImplementStackUsingQueue();
    stack.push(1);
    stack.push(2);
    stack.push(3);
    Assert.assertEquals(stack.top(), 3);
    Assert.assertEquals(stack.pop(), 3);
    Assert.assertEquals(stack.pop(), 2);
    Assert.assertFalse(stack.empty());
    Assert.assertEquals(stack.pop(), 1);
    Assert.assertTrue(stack.empty());
}

/**
 * APPROACH: Two Queues
 *
 * Time Complexity: O(1) for push, O(N) for pop and top
 * - Push is constant, pop/top require transferring elements.
 *
 * Space Complexity: O(N)
 * - Storing all elements in queues.
 */
public void push(int x) {
    queue2.offer(x);
    while (!queue1.isEmpty()) {
        queue2.offer(queue1.poll());
    }
    Queue<Integer> temp = queue1;
    queue1 = queue2;
    queue2 = temp;
}

public int pop() {
    if (empty()) throw new RuntimeException("Stack is empty");
    return queue1.poll();
}

public int top() {
    if (empty()) throw new RuntimeException("Stack is empty");
    return queue1.peek();
}

public boolean empty() {
    return queue1.isEmpty();
}
}
```

LargestRectangleInHistogram.java

```
package programs.stack;

import org.testng.Assert;

import java.util.Stack;

/**
 * LeetCode 84: Largest Rectangle in Histogram
 *
 * Problem Statement:
 * Given an array of integers heights representing the histogram's bar height
 * where the width of each bar is 1, return the area of the largest rectangle in
 * the histogram.
 *
 * Constraints:
 * - 1 <= heights.length <= 10^5
 * - 0 <= heights[i] <= 10^4
 */
public class LargestRectangleInHistogram {

    public static void main(String[] args) {
        int[] heights1 = {2, 1, 5, 6, 2, 3};
        Assert.assertEquals(largestRectangleArea(heights1), 10);

        int[] heights2 = {2, 4};
        Assert.assertEquals(largestRectangleArea(heights2), 4);
    }

    /**
     * APPROACH: Monotonic Stack (Optimal)
     *
     * Time Complexity: O(N)
     * - Each element is pushed and popped at most once.
     *
     * Space Complexity: O(N)
     * - Stack storing indices.
     */
    public static int largestRectangleArea(int[] heights) {
        Stack<Integer> stack = new Stack<>();
        int maxArea = 0;
        int n = heights.length;

        for (int i = 0; i <= n; i++) {
            int currentHeight = (i == n) ? 0 : heights[i];

            while (!stack.isEmpty() && currentHeight < heights[stack.peek()]) {
                int height = heights[stack.pop()];
                int width = stack.isEmpty() ? i : i - stack.peek() - 1;
                maxArea = Math.max(maxArea, height * width);
            }

            stack.push(i);
        }
    }
}
```

```

    }

    return maxArea;
}
}

```

MinStack.java

```

package programs.stack;

import org.testng.Assert;

import java.util.Stack;

/**
 * LeetCode 155: Min Stack
 *
 * Problem Statement:
 * Design a stack that supports push, pop, top, and retrieving the minimum element
 * in constant time.
 * Implement the MinStack class:
 * - MinStack() initializes the stack object.
 * - void push(int val) pushes the element val onto the stack.
 * - void pop() removes the element on the top of the stack.
 * - int top() gets the top element of the stack.
 * - int getMin() retrieves the minimum element in the stack.
 *
 * Constraints:
 * -  $-2^{31} \leq \text{val} \leq 2^{31} - 1$ 
 * - Methods pop, top and getMin operations will always be called on non-empty
 * stacks.
 * - At most  $3 * 10^4$  calls will be made to push, pop, top, and getMin.
 */
public class MinStack {

    private Stack<Integer> stack;
    private Stack<Integer> minStack;

    public MinStack() {
        stack = new Stack<>();
        minStack = new Stack<>();
    }

    public static void main(String[] args) {
        MinStack minStack = new MinStack();
        minStack.push(-2);
        minStack.push(0);
        minStack.push(-3);
        Assert.assertEquals(minStack.getMin(), -3);
        minStack.pop();
    }
}

```

```

        Assert.assertEquals(minStack.top(), 0);
        Assert.assertEquals(minStack.getMin(), -2);
    }

    /**
     * APPROACH: Two Stacks (Optimal)
     *
     * Time Complexity: O(1) for all operations
     * - Constant time for all stack operations.
     *
     * Space Complexity: O(N)
     * - Two stacks storing elements.
     */
    public void push(int val) {
        stack.push(val);
        if (minStack.isEmpty() || val <= minStack.peek()) {
            minStack.push(val);
        }
    }

    public void pop() {
        if (stack.pop().equals(minStack.peek())) {
            minStack.pop();
        }
    }

    public int top() {
        return stack.peek();
    }

    public int getMin() {
        return minStack.peek();
    }
}

```

NextGreaterElement.java

```

package programs.stack;

import org.testng.Assert;

import java.util.Stack;

/**
 * LeetCode 503: Next Greater Element II
 *
 * Problem Statement:
 * Given a circular integer array nums (i.e., the next element of nums[nums.length
 * - 1] is nums[0]),
 * return the next greater number for every element in nums.
 */

```

```

* The next greater number of a number x is the first greater number to its
traversing-order next in the array.
* If it doesn't exist, return -1 for this number.
*
* Constraints:
* - 1 <= nums.length <= 10^4
* - 0 <= nums[i] <= 10^9
*/
public class NextGreaterElement {

    public static void main(String[] args) {
        int[] nums1 = {1, 2, 1};
        int[] result1 = nextGreaterElements(nums1);
        Assert.assertEquals(result1, new int[]{2, -1, 2});

        int[] nums2 = {1, 2, 3, 4, 3};
        int[] result2 = nextGreaterElements(nums2);
        Assert.assertEquals(result2, new int[]{2, 3, 4, -1, 4});
    }

    /**
     * APPROACH: Monotonic Stack (Optimal)
     *
     * Time Complexity: O(N)
     * - Each element is pushed and popped at most once.
     *
     * Space Complexity: O(N)
     * - Stack storing indices.
     */
    public static int[] nextGreaterElements(int[] nums) {
        int n = nums.length;
        int[] result = new int[n];
        Stack<Integer> stack = new Stack<>();

        for (int i = 0; i < n; i++) {
            result[i] = -1;
        }

        for (int i = 0; i < 2 * n; i++) {
            int actualIndex = i % n;
            while (!stack.isEmpty() && nums[actualIndex] > nums[stack.peek()]) {
                int prevIndex = stack.pop();
                result[prevIndex] = nums[actualIndex];
            }
            if (i < n) {
                stack.push(actualIndex);
            }
        }

        return result;
    }
}

```

RemoveKDigits.java

```
package programs.stack;

import org.testng.Assert;

import java.util.Stack;

/**
 * LeetCode 402: Remove K Digits
 *
 * Problem Statement:
 * Given string num representing a non-negative integer num, and an integer k,
 * return the smallest possible integer after removing k digits from num.
 *
 * Constraints:
 * - 1 <= k <= num.length <= 10^5
 * - num consists of only digits.
 * - num does not have any leading zeros except for the zero itself.
 */
public class RemoveKDigits {

    public static void main(String[] args) {
        Assert.assertEquals(removeKDigits("1432219", 3), "1219");
        Assert.assertEquals(removeKDigits("10200", 1), "200");
        Assert.assertEquals(removeKDigits("10", 2), "0");
    }

    /**
     * APPROACH: Monotonic Stack (Optimal)
     *
     * Time Complexity: O(N)
     * - Each digit is pushed and popped at most once.
     *
     * Space Complexity: O(N)
     * - Stack storing digits.
     */
    public static String removeKDigits(String num, int k) {
        if (k == 0) return num;
        if (k >= num.length()) return "0";

        Stack<Character> stack = new Stack<>();

        for (char digit : num.toCharArray()) {
            while (k > 0 && !stack.isEmpty() && stack.peek() > digit) {
                stack.pop();
                k--;
            }
            stack.push(digit);
        }
    }
}
```

```

        while (k > 0) {
            stack.pop();
            k--;
        }

        StringBuilder result = new StringBuilder();
        boolean leadingZero = true;
        for (char digit : stack) {
            if (leadingZero && digit == '0') continue;
            leadingZero = false;
            result.append(digit);
        }

        return result.length() == 0 ? "0" : result.toString();
    }
}

```

ValidBrackets.java

```

package programs.stack;

import org.testng.Assert;

import java.util.Stack;

/**
 * LeetCode 20: Valid Parentheses
 *
 * Problem Statement:
 * Given a string containing just the characters '(', ')', '{', '}', '[' and ']',
 * determine if the input string is valid. An input string is valid if open
brackets are closed
 * by the same type of brackets and in the correct order.
 *
 * Constraints:
 * - 1 <= s.length <= 10^4
 * - s consists of parentheses only '()[]{}'.
 */
public class ValidBrackets {

    public static void main(String[] args) {
        Assert.assertTrue(isValid("()[]{}"));
        Assert.assertTrue(isValid("{[]}"));
        Assert.assertFalse(isValid("]"));
        Assert.assertFalse(isValid("(]"));
    }

    /**
     * APPROACH: Stack (Optimal)
     */
}

```



```
* Time Complexity: O(N)
* - Single pass through the string.
*
* Space Complexity: O(N)
* - Stack storing opening brackets.
*/
public static boolean isValid(String s) {
    Stack<Character> stack = new Stack<>();

    for (char c : s.toCharArray()) {
        if (c == '(' || c == '{' || c == '[') {
            stack.push(c);
        } else if (c == ')' && !stack.isEmpty() && stack.peek() == '(') {
            stack.pop();
        } else if (c == '}' && !stack.isEmpty() && stack.peek() == '{') {
            stack.pop();
        } else if (c == ']' && !stack.isEmpty() && stack.peek() == '[') {
            stack.pop();
        } else {
            return false;
        }
    }
    return stack.isEmpty();
}
```

Topic: strings

Notes & Concept Guide

String Problems - Interview Prep Guide

Overview

This folder contains **24 problems** focused on string manipulation and pattern matching.

Problem Categories

1. Palindrome Problems (2 problems)

- [Palindrome.java](#) - Check if string is palindrome (multiple approaches)
- [NumberPalindrome.java](#) - Check if number is palindrome
- [LongestPalindromicSubstring.java](#) - Find longest palindromic substring

Techniques: Two-pointers, Expand around center, DP **Time:** O(n) to O(n²)

2. Reversal Problems (5 problems)

- [ReverseCharArray.java](#) - Reverse character array in-place
- [ReverseString.java](#) - Character-by-character reversal
- [ReverseStringPreservingSpaces.java](#) - Reverse with space positions maintained
- [ReverseStringWithoutSpecialChars.java](#) - Reverse only alphanumeric
- [Reverse.java](#) - Additional reversal approach
- [RomanToInt.java](#) - Roman numeral to integer conversion

Techniques: Two-pointers, Recursion, Stack **Challenge:** Do it in-place with O(1) space

3. Substring & Matching (6 problems)

- [LongestUniqueCharSubstring.java](#) - Longest substring with unique chars
- [LongestCommonPrefix.java](#) - Common prefix of strings
- [PermutationInString.java](#) - Check if one string is permutation of another
- [FindSubstringIndex.java](#) - Find first occurrence of substring
- [IsomorphicStrings.java](#) - Check if strings are isomorphic
- [CommonElementsInArrays.java](#) - Common elements in string arrays

Techniques: Sliding window, Hashing, Two-pointers **Best For:** Pattern matching problems

4. Character Operations (6 problems)

- [CharactersWithFrequency.java](#) - Characters repeated at least twice
- [SeparateCharactersByType.java](#) - Separate uppercase, lowercase, digits
- [SortCharactersByFrequency.java](#) - Sort characters by frequency
- [MaxConsecutiveCharacter.java](#) - Find character with max consecutive occurrences
- [PrintTrianglePattern.java](#) - Pattern printing with characters
- [RemoveStringDuplicates.java](#) - Remove duplicate characters

Techniques: HashMap, String manipulation

5. String Transformation (3 problems)

- [StringCompression.java](#) - Compress string (run-length encoding)
- [StringPermutations.java](#) - Generate all permutations
- [LengthOfLastWord.java](#) - Find length of last word

Algorithm: Backtracking for permutations **Challenge:** Generate all permutations efficiently

Key Techniques to Master

1. Two-Pointer Technique

Used in: [ReverseCharArray.java](#), [Palindrome.java](#)

- Check palindrome in O(n) time, O(1) space
- Most efficient approach
- Pattern: left starts at 0, right at length-1

2. Sliding Window

Used in: [LongestUniqueCharSubstring.java](#), [PermutationInString.java](#)

- Track window of characters
- Time: $O(n)$, Space: $O(k)$ where k = window size
- Common in substring problems

3. Expand Around Center

Used in: [LongestPalindromicSubstring.java](#)

- For each potential center, expand outward
- Handles both odd and even length palindromes
- Time: $O(n^2)$, Space: $O(1)$

4. Dynamic Programming

Used in: [LongestPalindromicSubstring.java](#), [LongestUniqueCharSubstring.java](#)

- Build solution from smaller subproblems
- Space-time tradeoff: $O(n^2)$ space vs $O(n)$ time

Interview Preparation Checklist

Easy Level (Start Here)

- ☐ [ReverseCharArray.java](#) - Basic string reversal
- ☐ [Palindrome.java](#) - Palindrome check
- ☐ [LengthOfLastWord.java](#) - Count last word length
- ☐ [FindSubstringIndex.java](#) - Substring search
- ☐ [RomanToInt.java](#) - Numeric conversion
- ☐ [Reverse.java](#) - Alternative reversal

Medium Level

- ☐ [LongestUniqueCharSubstring.java](#) - Sliding window
- ☐ [LongestCommonPrefix.java](#) - Array of strings
- ☐ [PermutationInString.java](#) - Anagram detection
- ☐ [StringCompression.java](#) - Encoding
- ☐ [IsomorphicStrings.java](#) - Pattern matching
- ☐ [RemoveStringDuplicates.java](#) - Duplicate removal
- ☐ [PrintTrianglePattern.java](#) - Pattern printing

Hard Level

- ☐ [LongestPalindromicSubstring.java](#) - DP/Expand approach
- ☐ [StringPermutations.java](#) - Backtracking
- ☐ [ReverseStringPreservingSpaces.java](#) - Edge cases

Common Interview Questions

1. Reverse a String

Problem: Reverse string in-place **Solution:** See [ReverseCharArray.java](#) **Follow-up:** Handle special characters, unicode

2. Palindrome Check

Problem: Check if string is palindrome (ignore case/spaces) **Solution:** See [Palindrome.java](#) **Interview Tip:** Mention two-pointer approach ($O(n)$ time, $O(1)$ space)

3. Longest Substring Without Repeating

Problem: Find longest substring with all unique characters **Solution:** See [LongestUniqueCharSubstring.java](#) **Key Technique:** Sliding window with HashMap

4. Longest Palindromic Substring

Problem: Find longest palindromic subsequence **Solution:** See [LongestPalindromicSubstring.java](#) **Approaches:**

- Expand around center: $O(n^2)$ time, $O(1)$ space ✓
- Dynamic Programming: $O(n^2)$ time, $O(n^2)$ space
- Manacher's Algorithm: $O(n)$ time, $O(n)$ space (advanced)

5. String Permutation

Problem: Check if one string is permutation of another **Solution:** See [PermutationInString.java](#) **Hint:** Anagram detection

Related Files in Other Folders

These files handle string-related work but are categorized differently:

- **HashMaps folder** (String frequency related):
 - [GroupAnagrams.java](#) - Group anagrams
 - [ValidAnagram.java](#) - Check if anagrams
 - [CharacterFrequency.java](#) - Character frequency
- **Array folder** (String array operations):
 - [SortStringCharsTest.java](#) - Sort character array

Time Complexity Reference

Problem	Time	Space	Technique
Reverse String	$O(n)$	$O(1)$	Two pointers

Problem	Time	Space	Technique
Palindrome Check	$O(n)$	$O(1)$	Two pointers
Longest Unique	$O(n)$	$O(k)$	Sliding window
Longest Palindrome	$O(n^2)$	$O(1)$	Expand around
Anagram Check	$O(n)$	$O(1)$	Sorting/Freq
First Occurrence	$O(n*m)$	$O(1)$	String search
Permutations	$O(n!)$	$O(n!)$	Backtracking

Practice Tips

Pattern Recognition

1. **Palindrome Problems** → Use two-pointers
2. **Substring Problems** → Use sliding window with HashMap
3. **Permutation/Combination** → Use backtracking
4. **Comparison** → Use character frequency maps

Optimization Focus

1. **Space**: Can we solve in $O(1)$ extra space?
2. **Time**: Can we reduce from $O(n^2)$ to $O(n)$?
3. **In-place**: Can we modify the input?

Common Mistakes

- Forgetting to handle edge cases (empty string, single char)
- Not considering unicode/special characters
- Off-by-one errors in two-pointer approach
- Not using HashSet/HashMap for frequency problems

Generated: Interview Prep Package - String Folder

Code Solutions

CharsRepeatedTwice.java

```
package programs.strings;

import org.testng.Assert;

import java.util.HashMap;
import java.util.Map;

/**
```

```

* Characters Repeated Twice
*
* Problem Statement:
* Given a string, find all characters that appear at least twice and print their
frequency.
*
* Constraints:
* - 1 <= str.length <= 10^4
* - str consists of lowercase English letters.
*/
public class CharsRepeatedTwice {

    public static void main(String[] args) {
        String str = "aabccdad";
        Map<Character, Integer> result = findCharsRepeatedTwice(str);
        Assert.assertEquals(result.get('a'), Integer.valueOf(3));
        Assert.assertEquals(result.get('c'), Integer.valueOf(2));
        Assert.assertEquals(result.get('d'), Integer.valueOf(2));
    }

    /**
     * APPROACH: Frequency Array (Optimal for lowercase)
     *
     * Time Complexity: O(N)
     * - Single pass through the string.
     *
     * Space Complexity: O(1)
     * - Fixed size array of 26.
     */
    public static Map<Character, Integer> findCharsRepeatedTwice(String str) {
        Map<Character, Integer> result = new HashMap<>();
        int[] counter = new int[26];

        for (char c : str.toCharArray()) {
            counter[c - 'a']++;
        }

        for (int i = 0; i < counter.length; i++) {
            if (counter[i] >= 2) {
                result.put((char) (i + 'a'), counter[i]);
            }
        }
        return result;
    }

    /**
     * APPROACH 2: HashMap (General purpose)
     *
     * Time Complexity: O(N)
     * - Single pass through the string.
     *
     * Space Complexity: O(K)
     * - K is the number of unique characters.
     */

```

```

    public static Map<Character, Integer> findCharsRepeatedTwiceHashMap(String
str) {
        Map<Character, Integer> frequencyMap = new HashMap<>();
        Map<Character, Integer> result = new HashMap<>();

        for (char c : str.toCharArray()) {
            frequencyMap.put(c, frequencyMap.getOrDefault(c, 0) + 1);
        }

        for (Map.Entry<Character, Integer> entry : frequencyMap.entrySet()) {
            if (entry.getValue() >= 2) {
                result.put(entry.getKey(), entry.getValue());
            }
        }
        return result;
    }
}

```

CommonElementsInArrays.java

```

package programs.strings;

import org.testng.Assert;

import java.util.*;

/**
 * Common Elements in Arrays
 *
 * Problem Statement:
 * Given two arrays of strings, find the common elements between them.
 *
 * Constraints:
 * - 1 <= a1.length, a2.length <= 10^4
 * - Arrays contain strings.
 */
public class CommonElementsInArrays {

    public static void main(String[] args) {
        String[] a1 = {"apple", "banana", "single", "Laxmi", "Raghu"};
        String[] a2 = {"june", "july", "apple", "april", "Laxmi"};
        List<String> result = commonString(a1, a2);
        Assert.assertEquals(result, List.of("apple", "Laxmi"));
    }

    /**
     * APPROACH: HashSet (Optimal)
     *
     * Time Complexity: O(N + M)
     * - N and M are lengths of the two arrays.
     */
}

```

```

*
* Space Complexity: O(M)
* - HashSet for the second array.
*/
public static List<String> commonString(String[] a1, String[] a2) {
    Set<String> a2Set = new HashSet<>(Arrays.asList(a2));
    List<String> result = new ArrayList<>();
    for (String ele : a1) {
        if (a2Set.contains(ele)) {
            result.add(ele);
        }
    }
    return result;
}

/**
* APPROACH 2: Two Pointers (If arrays are sorted)
*
* Time Complexity: O(N log N + M log M)
* - Sorting both arrays.
*
* Space Complexity: O(1)
* - Constant extra space (ignoring sort space).
*/
public static List<String> commonStringSorted(String[] a1, String[] a2) {
    Arrays.sort(a1);
    Arrays.sort(a2);
    List<String> result = new ArrayList<>();
    int i = 0, j = 0;

    while (i < a1.length && j < a2.length) {
        if (a1[i].equals(a2[j])) {
            result.add(a1[i]);
            i++;
            j++;
        } else if (a1[i].compareTo(a2[j]) < 0) {
            i++;
        } else {
            j++;
        }
    }
    return result;
}
}

```

FindSubstringIndex.java

```

package programs.strings;

import org.testng.Assert;

```



```
/**
 * LeetCode 28: Find the Index of the First Occurrence in a String
 *
 * Problem Statement:
 * Given two strings needle and haystack, return the index of the first occurrence
 of needle in haystack,
 * or -1 if needle is not part of haystack.
 *
 * Constraints:
 * - 1 <= haystack.length, needle.length <= 10^4
 * - haystack and needle consist of only lowercase English characters.
 */
public class FindSubstringIndex {

    public static void main(String[] args) {
        Assert.assertEquals(strStr("sadbutsad", "sad"), 0);
        Assert.assertEquals(strStr("leetcode", "leeto"), -1);
        Assert.assertEquals(strStr("hello", "ll"), 2);
    }

    /**
     * APPROACH: Brute Force (Simple)
     *
     * Time Complexity: O(M * N)
     * - M is haystack length, N is needle length.
     *
     * Space Complexity: O(1)
     * - Constant extra space.
     */
    public static int strStr(String haystack, String needle) {
        int m = haystack.length(), n = needle.length();
        for (int i = 0; i <= m - n; i++) {
            if (haystack.substring(i, i + n).equals(needle)) {
                return i;
            }
        }
        return -1;
    }

    /**
     * APPROACH 2: Built-in indexOf (Optimal)
     *
     * Time Complexity: O(M * N)
     * - Uses optimized string matching.
     *
     * Space Complexity: O(1)
     * - Constant extra space.
     */
    public static int strStrBuiltIn(String haystack, String needle) {
        return haystack.indexOf(needle);
    }
}
```

IsomorphicStrings.java

```
package programs.strings;

import org.testng.Assert;

import java.util.HashMap;
import java.util.HashSet;

/**
 * LeetCode 205: Isomorphic Strings
 *
 * Problem Statement:
 * Given two strings s and t, determine if they are isomorphic. Two strings are
 * isomorphic
 * if the characters in s can be replaced to get t. All occurrences of a character
 * must be replaced
 * with another character while preserving the order of characters. No two
 * characters may map to
 * the same character, but a character may map to itself.
 *
 * Constraints:
 * - 1 <= s.length <= 5 * 10^4
 * - t.length == s.length
 * - s and t consist of any valid ascii character.
 */
public class IsomorphicStrings {

    public static void main(String[] args) {
        Assert.assertTrue(areIsomorphic("aab", "xxy"));
        Assert.assertFalse(areIsomorphic("bar", "foo"));
        Assert.assertTrue(areIsomorphic("paper", "title"));
        Assert.assertTrue(areIsomorphic("abc", "xyz"));
    }

    /**
     * APPROACH: HashMap and HashSet (Optimal)
     *
     * Time Complexity: O(N)
     * - Single pass through the strings.
     *
     * Space Complexity: O(1)
     * - Fixed size map and set (ASCII characters).
     */
    public static boolean areIsomorphic(String str1, String str2) {
        if (str1.length() != str2.length()) {
            return false;
        }
    }
}
```

```

HashMap<Character, Character> map = new HashMap<>();
HashSet<Character> mappedValues = new HashSet<>();

for (int i = 0; i < str1.length(); i++) {
    char c1 = str1.charAt(i);
    char c2 = str2.charAt(i);

    if (map.containsKey(c1) && map.get(c1) != c2) {
        return false;
    }
    if (!map.containsKey(c1)) {
        if (mappedValues.contains(c2)) {
            return false;
        }
        map.put(c1, c2);
        mappedValues.add(c2);
    }
}
return true;
}

```

LengthOfLastWord.java

```

package programs.strings;

import org.testng.Assert;

/**
 * LeetCode 58: Length of Last Word
 *
 * Problem Statement:
 * Given a string s consisting of words and spaces, return the length of the last
 * word in the string.
 * A word is a maximal substring consisting of non-space characters only.
 *
 * Constraints:
 * - 1 <= s.length <= 10^4
 * - s consists of only English letters and spaces ' '.
 * - There will be at least one word in s.
 */
public class LengthOfLastWord {

    public static void main(String[] args) {
        Assert.assertEquals(lengthOfLastWord("Hello World"), 5);
        Assert.assertEquals(lengthOfLastWord(" fly me to the moon "), 4);
        Assert.assertEquals(lengthOfLastWord("luffy is still joyboy"), 6);
    }

    /**

```

```

    * APPROACH: Split and Get Last (Optimal)
    *
    * Time Complexity: O(N)
    * - Splitting the string.
    *
    * Space Complexity: O(N)
    * - Array for split strings.
    */
    public static int lengthOfLastWord(String s) {
        s = s.trim();
        String[] splitStrings = s.split("\\s+");
        int len = splitStrings.length - 1;
        return splitStrings[len].length();
    }

    /**
    * APPROACH 2: Reverse Traversal (Optimal Space)
    *
    * Time Complexity: O(N)
    * - Single pass from the end.
    *
    * Space Complexity: O(1)
    * - Constant extra space.
    */
    public static int lengthOfLastWordOptimal(String s) {
        int length = 0;
        int i = s.length() - 1;

        // Skip trailing spaces
        while (i >= 0 && s.charAt(i) == ' ') {
            i--;
        }

        // Count characters of the last word
        while (i >= 0 && s.charAt(i) != ' ') {
            length++;
            i--;
        }

        return length;
    }
}

```

LongestCommonPrefix.java

```

package programs.strings;

import org.testng.Assert;

```

```
/**
 * LeetCode 14: Longest Common Prefix
 *
 * Problem Statement:
 * Given an array of strings, find the longest common prefix string amongst the
array.
 * If there is no common prefix, return an empty string.
 *
 * Constraints:
 * - 1 <= strs.length <= 200
 * - 0 <= strs[i].length <= 200
 * - strs[i] consists of only lowercase English letters.
 */
public class LongestCommonPrefix {

    public static void main(String[] args) {
        String[] strs = {"flower", "flow", "flight"};
        Assert.assertEquals(longestCommonPrefix(strs), "fl");
    }

    /**
     * APPROACH: Horizontal Scanning (Optimal)
     *
     * Time Complexity: O(S)
     * - S is the sum of all characters in all strings.
     *
     * Space Complexity: O(1)
     * - Constant extra space.
     */
    static String longestCommonPrefix(String[] arr) {
        if (arr == null || arr.length == 0) return "";
        String result = arr[0];
        for (int i = 1; i < arr.length; i++) {
            int len = commonPrefixLen(result, arr[i]);
            result = result.substring(0, len);
        }
        return result;
    }

    static int commonPrefixLen(String a, String b) {
        int commonLength = 0;
        int minL = Math.min(a.length(), b.length());
        for (int i = 0; i < minL; i++) {
            if (a.charAt(i) == b.charAt(i)) {
                commonLength++;
            } else {
                break;
            }
        }
        return commonLength;
    }
}
```

LongestPalindromicSubstring.java

```
package programs.strings;

import org.testng.Assert;

/**
 * LeetCode 5: Longest Palindromic Substring
 *
 * Problem Statement:
 * Given a string, find the longest palindromic substring within it.
 * A palindrome is a word that reads the same backward as forward.
 *
 * Constraints:
 * - 1 <= s.length <= 1000
 * - s consists of English letters.
 */
public class LongestPalindromicSubstring {

    public static void main(String[] args) {
        String result1 = findLongestPalindromeSubstringBruteForce("babad");
        Assert.assertTrue(result1.equals("bab") || result1.equals("aba"));

        String result2 = findLongestPalindromeSubstringBruteForce("cbbd");
        Assert.assertEquals(result2, "bb");
    }

    /**
     * APPROACH 1: Brute Force
     *
     * Time Complexity:  $O(N^3)$ 
     * - Nested loops  $O(N^2)$  and palindrome check  $O(N)$ .
     *
     * Space Complexity:  $O(1)$ 
     * - Constant extra space.
     */
    static String findLongestPalindromeSubstringBruteForce(String s) {
        int maxLength = 1;
        String longestPalindrome = String.valueOf(s.charAt(0));
        for (int i = 0; i < s.length(); i++) {
            for (int j = i + 1; j < s.length(); j++) {
                if ((j - i + 1) > maxLength && isPalindrome(s, i, j)) {
                    maxLength = j - i + 1;
                    longestPalindrome = s.substring(i, j + 1);
                }
            }
        }
        return longestPalindrome;
    }
}
```

```

    }

    static boolean isPalindrome(String str, int start, int end) {
        while (start < end) {
            if (str.charAt(start) != str.charAt(end)) {
                return false;
            }
            start++;
            end--;
        }
        return true;
    }

    /**
     * APPROACH 2: Expand Around Center (Optimal)
     *
     * Time Complexity: O(N^2)
     * - Expanding around each center.
     *
     * Space Complexity: O(1)
     * - Constant extra space.
     */
    static String findLongestPalindromeSubstringOptimal(String s) {
        if (s == null || s.length() < 1) return "";
        int start = 0, end = 0;

        for (int i = 0; i < s.length(); i++) {
            int len1 = expandAroundCenter(s, i, i);
            int len2 = expandAroundCenter(s, i, i + 1);
            int len = Math.max(len1, len2);

            if (len > end - start) {
                start = i - (len - 1) / 2;
                end = i + len / 2;
            }
        }
        return s.substring(start, end + 1);
    }

    private static int expandAroundCenter(String s, int left, int right) {
        while (left >= 0 && right < s.length() && s.charAt(left) ==
s.charAt(right)) {
            left--;
            right++;
        }
        return right - left - 1;
    }
}

```

LongestUniqueCharSubstring.java

```
package programs.strings;

import org.testng.Assert;

import java.util.HashSet;
import java.util.Set;

/**
 * LeetCode 3: Longest Substring Without Repeating Characters
 *
 * Problem Statement:
 * Given a string, find the length of the longest substring without repeating
 * characters.
 *
 * Constraints:
 * -  $0 \leq s.length \leq 5 * 10^4$ 
 * - s consists of English letters, digits, symbols and spaces.
 */
public class LongestUniqueCharSubstring {

    public static void main(String[] args) {
        String s = "GEEKSFORGEEEKS";
        String result = longestSubStrWithoutRepetition(s);
        Assert.assertEquals(result.length(), 7);
    }

    /**
     * APPROACH: Sliding Window with HashSet (Optimal)
     *
     * Time Complexity: O(N)
     * - Each character is added and removed at most once.
     *
     * Space Complexity: O(min(N, M))
     * - M is the size of the character set.
     */
    static String longestSubStrWithoutRepetition(String str) {
        Set<Character> hs = new HashSet<>();
        int j = 0;
        int maxLength = 0;
        int startIndex = 0;

        for (int i = 0; i < str.length(); i++) {
            while (hs.contains(str.charAt(i))) {
                hs.remove(str.charAt(j));
                j++;
            }
            hs.add(str.charAt(i));

            if (i - j + 1 > maxLength) {
                maxLength = i - j + 1;
                startIndex = j;
            }
        }
    }
}
```



```
    }  
    return str.substring(startIndex, startIndex + maxLength);  
}  
}
```

MaxConsecutiveCharacter.java

```
package programs.strings;  
  
import org.testng.Assert;  
  
/**  
 * Max Consecutive Character  
 *  
 * Problem Statement:  
 * Given a string, find the character that appears consecutively the maximum  
number of times.  
 *  
 * Constraints:  
 * - 1 <= str.length <= 10^4  
 * - str consists of printable ASCII characters.  
 */  
public class MaxConsecutiveCharacter {  
  
    public static void main(String[] args) {  
        String word = "Amazon is a great company as it has Atooozzzz";  
        char result = maxRepeating(word);  
        Assert.assertEquals(result, 'z');  
    }  
  
    /**  
     * APPROACH: Single Pass (Optimal)  
     *  
     * Time Complexity: O(N)  
     * - Single pass through the string.  
     *  
     * Space Complexity: O(1)  
     * - Constant extra space.  
     */  
    static char maxRepeating(String str) {  
        int maxCount = 0;  
        char result = str.charAt(0);  
  
        int currentCount = 1;  
        char currentChar = str.charAt(0);  
  
        for (int i = 1; i < str.length(); i++) {  
            if (str.charAt(i) == currentChar) {  
                currentCount++;  
            } else {
```

```

        currentCount = 1;
        currentChar = str.charAt(i);
    }

    if (currentCount > maxCount) {
        maxCount = currentCount;
        result = currentChar;
    }
}

return result;
}
}

```

NumberPalindrome.java

```

package programs.strings;

import org.testng.Assert;

/**
 * LeetCode 9: Palindrome Number
 *
 * Problem Statement:
 * Given an integer, determine if it is a palindrome. A palindrome number reads
the same
 * backward as forward. Negative numbers are not palindromes.
 *
 * Constraints:
 * -  $-2^{31} \leq x \leq 2^{31} - 1$ 
 */
public class NumberPalindrome {

    public static void main(String[] args) {
        Assert.assertTrue(isPalindrome(121));
        Assert.assertTrue(isPalindrome(0));
        Assert.assertTrue(isPalindrome(1));
        Assert.assertFalse(isPalindrome(-121));
        Assert.assertFalse(isPalindrome(123));
    }

    /**
     * APPROACH: Reverse Half (Optimal)
     *
     * Time Complexity:  $O(\log_{10}(N))$ 
     * - Number of digits in the number.
     *
     * Space Complexity:  $O(1)$ 
     * - Constant extra space.

```

```

    */
    static boolean isPalindrome(int x) {
        if (x < 0) return false;
        if (x < 10) return true;
        if (x % 10 == 0) return false;

        int reversed = 0;
        while (x > reversed) {
            reversed = reversed * 10 + x % 10;
            x /= 10;
        }

        return x == reversed || x == reversed / 10;
    }

    /**
     * APPROACH 2: String Conversion (Simple)
     *
     * Time Complexity: O(N)
     * - N is the number of digits.
     *
     * Space Complexity: O(N)
     * - String representation.
     */
    static boolean isPalindromeString(int x) {
        if (x < 0) return false;
        String str = String.valueOf(x);
        int left = 0, right = str.length() - 1;
        while (left < right) {
            if (str.charAt(left) != str.charAt(right)) {
                return false;
            }
            left++;
            right--;
        }
        return true;
    }
}

```

Palindrome.java

```

package programs.strings;

import org.testng.Assert;

/**
 * Valid Palindrome
 *
 * Problem Statement:

```

```

* Given a string, determine if it is a palindrome. A palindrome reads the same
backward as forward.
*
* Constraints:
* - 1 <= s.length <= 2 * 10^5
* - s consists of printable ASCII characters.
*/
public class Palindrome {

    public static void main(String[] args) {
        Assert.assertTrue(isPalindromeStringBuilder("Madam"));
        Assert.assertTrue(isPalindromeTwoPointer("A man, a plan, a canal:
Panama"));
        Assert.assertFalse(isPalindromeTwoPointer("race a car"));
    }

    /**
     * APPROACH 1: StringBuilder (Simple)
     *
     * Time Complexity: O(N)
     * - Building reversed string.
     *
     * Space Complexity: O(N)
     * - StringBuilder for reversed string.
     */
    public static boolean isPalindromeStringBuilder(String s) {
        StringBuilder rev = new StringBuilder();
        for (int i = s.length() - 1; i >= 0; i--) {
            rev.append(s.charAt(i));
        }
        return s.equalsIgnoreCase(rev.toString());
    }

    /**
     * APPROACH 2: Two Pointers with alphanumeric filtering (Optimal)
     *
     * Time Complexity: O(N)
     * - Single pass through the string.
     *
     * Space Complexity: O(1)
     * - Constant extra space.
     */
    public static boolean isPalindromeTwoPointer(String s) {
        int left = 0, right = s.length() - 1;

        while (left < right) {
            while (left < right && !Character.isLetterOrDigit(s.charAt(left))) {
                left++;
            }
            while (left < right && !Character.isLetterOrDigit(s.charAt(right))) {
                right--;
            }

            if (Character.toLowerCase(s.charAt(left)) !=

```

```

Character.toLowerCase(s.charAt(right))) {
    return false;
}

left++;
right--;
}
return true;
}
}

```

PermutationInString.java

```

package programs.strings;

import org.testng.Assert;

import java.util.Arrays;

/**
 * LeetCode 567: Permutation in String
 *
 * Problem Statement:
 * Given two strings s1 and s2, return true if s2 contains a permutation of s1, or
 * false otherwise.
 * A permutation of a string is any rearrangement of its characters.
 *
 * Constraints:
 * - 1 <= s1.length, s2.length <= 10^4
 * - s1 and s2 consist of lowercase English letters.
 */
public class PermutationInString {

    public static void main(String[] args) {
        Assert.assertTrue(containsPermutations("ab", "eidbaooo"));
        Assert.assertFalse(containsPermutations("ab", "eidboaoo"));
    }

    /**
     * APPROACH: Sliding Window with Frequency Arrays (Optimal)
     *
     * Time Complexity: O(N)
     * - Single pass through s2.
     *
     * Space Complexity: O(1)
     * - Fixed size arrays of 26.
     */
    static boolean containsPermutations(String s1, String s2) {
        if (s1.length() > s2.length()) {

```

```

        return false;
    }

    int[] s1Freq = new int[26];
    int[] windowFreq = new int[26];

    for (char c : s1.toCharArray()) {
        s1Freq[c - 'a']++;
    }

    for (int i = 0; i < s2.length(); i++) {
        windowFreq[s2.charAt(i) - 'a']++;
        if (i >= s1.length()) {
            windowFreq[s2.charAt(i - s1.length()) - 'a']--;
        }
        if (Arrays.equals(s1Freq, windowFreq)) {
            return true;
        }
    }
    return false;
}
}

```

PrintTrianglePattern.java

```

package programs.strings;

import org.testng.Assert;

/**
 * Print Triangle Pattern
 *
 * Problem Statement:
 * Print a right-angled triangle pattern of stars with the given number of rows.
 *
 * Constraints:
 * - 1 <= n <= 20
 */
public class PrintTrianglePattern {

    public static void main(String[] args) {
        int n = 5;
        printRightAngledTriangle(n);
        Assert.assertNotNull(printRightAngledTriangleToString(n));
    }

    /**
     * APPROACH: Nested Loops (Optimal)
     */
}

```

```

    * Time Complexity:  $O(N^2)$ 
    * - Nested loops for pattern printing.
    *
    * Space Complexity:  $O(1)$ 
    * - Constant extra space.
    */
    public static void printRightAngledTriangle(int n) {
        for (int i = 0; i < n; i++) {
            for (int j = n - i; j > 1; j--) {
                System.out.print(" ");
            }
            for (int j = 0; j <= i; j++) {
                System.out.print("* ");
            }
            System.out.println();
        }
    }

    /**
     * APPROACH 2: StringBuilder (For testing)
     *
     * Time Complexity:  $O(N^2)$ 
     * - Nested loops for building string.
     *
     * Space Complexity:  $O(N^2)$ 
     * - StringBuilder for output.
     */
    public static String printRightAngledTriangleToString(int n) {
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < n; i++) {
            for (int j = n - i; j > 1; j--) {
                sb.append(" ");
            }
            for (int j = 0; j <= i; j++) {
                sb.append("* ");
            }
            sb.append("\n");
        }
        return sb.toString();
    }
}

```

RemoveStringDuplicates.java

```

package programs.strings;

import org.testng.Assert;

/**

```

```

* Remove String Duplicates
*
* Problem Statement:
* Given a string, remove duplicate characters from it while preserving the order
of
* first occurrence.
*
* Constraints:
* - 1 <= str.length <= 10^4
* - str consists of ASCII characters.
*/
public class RemoveStringDuplicates {

    public static void main(String[] args) {
        String str = "geeksforgeeks";
        String result = removeDuplicates(str);
        Assert.assertEquals(result, "geksfor");
    }

    /**
     * APPROACH: Boolean Array (Optimal for ASCII)
     *
     * Time Complexity: O(N)
     * - Single pass through the string.
     *
     * Space Complexity: O(1)
     * - Fixed size array of 256 for ASCII.
     */
    public static String removeDuplicates(String str) {
        boolean[] visited = new boolean[256];
        StringBuilder sb = new StringBuilder();

        for (int i = 0; i < str.length(); i++) {
            char ch = str.charAt(i);
            if (!visited[ch]) {
                visited[ch] = true;
                sb.append(ch);
            }
        }

        return sb.toString();
    }

    /**
     * APPROACH 2: LinkedHashSet (General purpose)
     *
     * Time Complexity: O(N)
     * - Single pass through the string.
     *
     * Space Complexity: O(N)
     * - LinkedHashSet for unique characters.
     */
    public static String removeDuplicatesSet(String str) {
        java.util.LinkedHashSet<Character> set = new java.util.LinkedHashSet<>();

```



```

        for (char c : str.toCharArray()) {
            set.add(c);
        }
        StringBuilder sb = new StringBuilder();
        for (char c : set) {
            sb.append(c);
        }
        return sb.toString();
    }
}

```

Reverse.java

```

package programs.strings;

import org.testng.Assert;

/**
 * Reverse String and Words
 *
 * Problem Statement:
 * Given a string, reverse the entire string and also reverse the order of words
 * in the string.
 *
 * Constraints:
 * - 1 <= input.length <= 10^4
 * - input contains English letters and spaces.
 */
public class Reverse {

    public static void main(String[] args) {
        String input = "My Name is Saurav Singh";
        Assert.assertEquals(reverseWords(input), "Singh Saurav is Name My");
        Assert.assertEquals(reverseString(input), "hgniS varuaS si emaN yM");
    }

    /**
     * APPROACH 1: Reverse Words (Optimal)
     *
     * Time Complexity: O(N)
     * - Splitting and reversing words.
     *
     * Space Complexity: O(N)
     * - Array for words and StringBuilder.
     */
    static String reverseWords(String input) {
        String[] words = input.split(" ");
        StringBuilder reversed = new StringBuilder();

        for (int i = words.length - 1; i >= 0; i--) {

```

```

        reversed.append(words[i]).append(" ");
    }

    return reversed.toString().trim();
}

/**
 * APPROACH 2: Reverse String (Optimal)
 *
 * Time Complexity: O(N)
 * - Single pass through characters.
 *
 * Space Complexity: O(N)
 * - StringBuilder for result.
 */
static String reverseString(String input) {
    char[] characters = input.toCharArray();
    StringBuilder reversed = new StringBuilder();

    for (int i = characters.length - 1; i >= 0; i--) {
        reversed.append(characters[i]);
    }

    return reversed.toString();
}
}

```

ReverseCharArray.java

```

package programs.strings;

import org.testng.Assert;

import java.util.Arrays;

/**
 * LeetCode 344: Reverse String
 *
 * Problem Statement:
 * Write a function that reverses a string. The input string is given as an array
 * of characters s.
 * You must do this by modifying the input array in-place with O(1) extra memory.
 *
 * Constraints:
 * - 1 <= s.length <= 10^5
 * - s[i] is a printable ASCII character.
 */
public class ReverseCharArray {

```

```
public static void main(String[] args) {
    char[] arr = new char[]{'h', 'e', 'l', 'l', 'o'};
    reverseString(arr);
    Assert.assertEquals(arr, new char[]{'o', 'l', 'l', 'e', 'h'});
}

/**
 * APPROACH: Two Pointers (Optimal)
 *
 * Time Complexity: O(N)
 * - Single pass with two pointers.
 *
 * Space Complexity: O(1)
 * - In-place swapping.
 */
public static void reverseString(char[] s) {
    int left = 0, right = s.length - 1;
    while (left < right) {
        char temp = s[left];
        s[left] = s[right];
        s[right] = temp;
        left++;
        right--;
    }
}

/**
 * APPROACH 2: Built-in Arrays (Alternative)
 *
 * Time Complexity: O(N)
 * - Built-in reverse operation.
 *
 * Space Complexity: O(N)
 * - Creates a new array internally.
 */
public static void reverseStringBuiltIn(char[] s) {
    int left = 0, right = s.length - 1;
    while (left < right) {
        char temp = s[left];
        s[left] = s[right];
        s[right] = temp;
        left++;
        right--;
    }
}
}
```

ReverseString.java

```

package programs.strings;

import org.testng.Assert;

/**
 * Reverse Each Word
 *
 * Problem Statement:
 * Given a string, reverse each word in the string while maintaining the original
 * order of words.
 *
 * Constraints:
 * - 1 <= str.length <= 10^4
 * - str contains English letters and spaces.
 */
public class ReverseString {

    public static void main(String[] args) {
        String str = "India is my country";
        String result = reverseEachWord(str);
        Assert.assertEquals(result, "aidnI si ym yrtnuoc");
    }

    /**
     * APPROACH: Split and Reverse (Optimal)
     *
     * Time Complexity: O(N * M)
     * - N words, M average word length.
     *
     * Space Complexity: O(N)
     * - Array for words and StringBuilder.
     */
    public static String reverseEachWord(String str) {
        String[] arr = str.split("\\s+");
        StringBuilder result = new StringBuilder();

        for (int i = arr.length - 1; i >= 0; i--) {
            char[] wordArray = arr[i].toCharArray();
            for (int j = wordArray.length - 1; j >= 0; j--) {
                result.append(wordArray[j]);
            }
            result.append(" ");
        }
        return result.toString().trim();
    }
}

```

ReverseStringPreservingSpaces.java

```
package programs.strings;

import org.testng.Assert;

/**
 * Reverse String Preserving Spaces
 *
 * Problem Statement:
 * Given a string, reverse it while maintaining the original positions of spaces.
 *
 * Constraints:
 * - 1 <= str.length <= 10^4
 * - str consists of printable ASCII characters and spaces.
 */
public class ReverseStringPreservingSpaces {

    public static void main(String[] args) {
        Assert.assertEquals(reverseString("this is a phone"), "enoh pa s isiht");
    }

    /**
     * APPROACH: Two Pointers (Optimal)
     *
     * Time Complexity: O(N)
     * - Single pass with two pointers.
     *
     * Space Complexity: O(N)
     * - Character array for in-place modification.
     */
    public static String reverseString(String str) {
        char[] arr = str.toCharArray();
        int left = 0;
        int right = arr.length - 1;

        while (left < right) {
            if (arr[left] == ' ') {
                left++;
            } else if (arr[right] == ' ') {
                right--;
            } else {
                char temp = arr[left];
                arr[left] = arr[right];
                arr[right] = temp;
                left++;
                right--;
            }
        }
        return new String(arr);
    }
}
```

ReverseStringWithoutSpecialChars.java

```
package programs.strings;

import org.testng.Assert;

/**
 * Reverse String Without Special Characters
 *
 * Problem Statement:
 * Given a string containing alphabetic characters and special characters, reverse
only
 * the alphabetic characters while keeping the special characters in their
original positions.
 *
 * Constraints:
 * - 1 <= str.length <= 10^4
 * - str consists of alphabetic characters and special characters.
 */
public class ReverseStringWithoutSpecialChars {

    public static void main(String[] args) {
        String str = "a!b?c";
        String reversedStr = reverseString(str);
        Assert.assertEquals(reversedStr, "c?b!a");
    }

    /**
     * APPROACH: Two Pointers (Optimal)
     *
     * Time Complexity: O(N)
     * - Single pass with two pointers.
     *
     * Space Complexity: O(N)
     * - Character array for in-place modification.
     */
    public static String reverseString(String str) {
        char[] charArray = str.toCharArray();
        int left = 0, right = charArray.length - 1;

        while (left < right) {
            if (!Character.isAlphabetic(charArray[left])) {
                left++;
            } else if (!Character.isAlphabetic(charArray[right])) {
                right--;
            } else {
                char temp = charArray[left];
                charArray[left] = charArray[right];
                charArray[right] = temp;
                left++;
                right--;
            }
        }
    }
}
```

```

        }
    }

    return new String(charArray);
}
}

```

RomanToInt.java

```

package programs.strings;

import org.testng.Assert;

import java.util.HashMap;
import java.util.Map;

/**
 * LeetCode 13: Roman to Integer
 *
 * Problem Statement:
 * Given a roman numeral, convert it to an integer.
 *
 * Constraints:
 * - 1 <= s.length <= 15
 * - s contains only the characters ('I', 'V', 'X', 'L', 'C', 'D', 'M').
 * - It is guaranteed that s is a valid roman numeral in the range [1, 3999].
 */
public class RomanToInt {

    public static void main(String[] args) {
        Assert.assertEquals(romanToInt("X"), 10);
        Assert.assertEquals(romanToInt("III"), 3);
        Assert.assertEquals(romanToInt("IV"), 4);
        Assert.assertEquals(romanToInt("IX"), 9);
        Assert.assertEquals(romanToInt("LVIII"), 58);
        Assert.assertEquals(romanToInt("MCMXCIV"), 1994);
    }

    /**
     * APPROACH: Right-to-Left Traversal (Optimal)
     *
     * Time Complexity: O(N)
     * - Single pass through the string.
     *
     * Space Complexity: O(1)
     * - Constant size map.
     */
    public static int romanToInt(String s) {
        Map<Character, Integer> romanMap = new HashMap<>();
        romanMap.put('I', 1);
    }
}

```

```

        romanMap.put('V', 5);
        romanMap.put('X', 10);
        romanMap.put('L', 50);
        romanMap.put('C', 100);
        romanMap.put('D', 500);
        romanMap.put('M', 1000);

        int result = 0;
        int prevValue = 0;

        for (int i = s.length() - 1; i >= 0; i--) {
            char currentChar = s.charAt(i);
            int currentValue = romanMap.get(currentChar);

            if (currentValue < prevValue) {
                result -= currentValue;
            } else {
                result += currentValue;
            }
            prevValue = currentValue;
        }
        return result;
    }
}

```

SeparateCharactersByType.java

```

package programs.strings;

import org.testng.Assert;

/**
 * Separate Characters By Type
 *
 * Problem Statement:
 * Given a string containing lowercase letters, uppercase letters, and digits,
 * print the lowercase characters, uppercase characters, and digits separately.
 *
 * Constraints:
 * - 1 <= input.length <= 10^4
 * - input consists of alphanumeric characters.
 */
public class SeparateCharactersByType {

    public static void main(String[] args) {
        String input = "sumitM28";
        String[] result = separateByType(input);
        Assert.assertEquals(result[0], "sumit");
        Assert.assertEquals(result[1], "M");
    }
}

```



```
        Assert.assertEquals(result[2], "28");
    }

    /**
     * APPROACH: Stream API (Optimal)
     *
     * Time Complexity: O(N)
     * - Single pass through the string.
     *
     * Space Complexity: O(N)
     * - StringBuilder for each character type.
     */
    public static String[] separateByType(String input) {
        StringBuilder lower = new StringBuilder();
        StringBuilder upper = new StringBuilder();
        StringBuilder digits = new StringBuilder();

        input.chars().filter(Character::isLowerCase).forEach(c ->
lower.append((char) c));
        input.chars().filter(Character::isUpperCase).forEach(c ->
upper.append((char) c));
        input.chars().filter(Character::isDigit).forEach(c -> digits.append((char)
c));

        return new String[]{lower.toString(), upper.toString(),
digits.toString()};
    }

    /**
     * APPROACH 2: Iterative (Alternative)
     *
     * Time Complexity: O(N)
     * - Single pass through the string.
     *
     * Space Complexity: O(N)
     * - StringBuilder for each character type.
     */
    public static String[] separateByTypeIterative(String input) {
        StringBuilder lower = new StringBuilder();
        StringBuilder upper = new StringBuilder();
        StringBuilder digits = new StringBuilder();

        for (char c : input.toCharArray()) {
            if (Character.isLowerCase(c)) {
                lower.append(c);
            } else if (Character.isUpperCase(c)) {
                upper.append(c);
            } else if (Character.isDigit(c)) {
                digits.append(c);
            }
        }

        return new String[]{lower.toString(), upper.toString(),
digits.toString()};
    }
}
```

```
    }
}
```

SortCharactersByFrequency.java

```
package programs.strings;

import org.testng.Assert;

import java.util.Arrays;
import java.util.HashMap;
import java.util.Map;
import java.util.PriorityQueue;

/**
 * LeetCode 451: Sort Characters By Frequency
 *
 * Problem Statement:
 * Given a string, sort its characters based on their frequency in descending
 * order.
 * If characters have the same frequency, maintain their original order.
 *
 * Constraints:
 * - 1 <= s.length <= 5 * 10^5
 * - s consists of uppercase and lowercase English letters and digits.
 */
public class SortCharactersByFrequency {

    public static void main(String[] args) {
        Assert.assertEquals(sortStrings("Aabb"), "bbAa");
        Assert.assertEquals(sortStrings("Aabb"), "bbAa");
        Assert.assertEquals(sortStrings("tree"), "eert");
    }

    /**
     * APPROACH 1: Frequency Array and Sort (Optimal)
     *
     * Time Complexity: O(N + K log K)
     * - N for counting, K log K for sorting (K = 256 for ASCII).
     *
     * Space Complexity: O(K)
     * - Fixed size array for ASCII.
     */
    public static String sortStrings(String s) {
        int[] freq = new int[256];
        for (char c : s.toCharArray()) {
            freq[c]++;
        }
    }
}
```

```

        Character[] chars = new Character[256];
        for (int i = 0; i < 256; i++) {
            chars[i] = (char) i;
        }

        Arrays.sort(chars, (a, b) -> freq[b] - freq[a]);

        StringBuilder result = new StringBuilder();
        for (char c : chars) {
            if (freq[c] > 0) {
                result.append(String.valueOf(c).repeat(freq[c]));
            }
        }
        return result.toString();
    }

    /**
     * APPROACH 2: HashMap and PriorityQueue (Optimal)
     *
     * Time Complexity: O(N log K)
     * - N for counting, K log K for heap operations.
     *
     * Space Complexity: O(K)
     * - Map and heap for unique characters.
     */
    public static String sortStringsWithMaps(String s) {
        Map<Character, Integer> frequencyMap = new HashMap<>();

        for (Character c : s.toCharArray()) {
            frequencyMap.put(c, frequencyMap.getOrDefault(c, 0) + 1);
        }

        PriorityQueue<Map.Entry<Character, Integer>> pq = new PriorityQueue<>(
            (a, b) -> b.getValue() - a.getValue()
        );
        pq.addAll(frequencyMap.entrySet());

        StringBuilder sb = new StringBuilder();
        while (!pq.isEmpty()) {
            Map.Entry<Character, Integer> entry = pq.poll();
            char c = entry.getKey();
            int freq = entry.getValue();
            sb.append(String.valueOf(c).repeat(freq));
        }

        return sb.toString();
    }
}

```

```
package programs.strings;

import org.testng.Assert;

/**
 * String Compression
 *
 * Problem Statement:
 * Given a string, compress it by replacing consecutive repeating characters with
 * the character followed by its count. If a character appears only once, don't
 * add a count.
 *
 * Constraints:
 * - 1 <= str.length <= 10^4
 * - str consists of lowercase English letters.
 */
public class StringCompression {

    public static void main(String[] args) {
        Assert.assertEquals(compressString("aaabbbacfwwww"), "a3b3acfw3");
        Assert.assertEquals(compressString("abc"), "abc");
    }

    /**
     * APPROACH: Single Pass (Optimal)
     *
     * Time Complexity: O(N)
     * - Single pass through the string.
     *
     * Space Complexity: O(N)
     * - StringBuilder for result.
     */
    public static String compressString(String str) {
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < str.length(); i++) {
            int count = 1;
            while (i < str.length() - 1 && str.charAt(i) == str.charAt(i + 1)) {
                count++;
                i++;
            }
            sb.append(str.charAt(i));
            if (count > 1) {
                sb.append(count);
            }
        }
        return sb.toString();
    }
}
```

StringPermutations.java

```
package programs.strings;

import org.testng.Assert;

import java.util.HashSet;
import java.util.Set;

/**
 * String Permutations
 *
 * Problem Statement:
 * Given a string, generate all possible permutations of its characters.
 *
 * Constraints:
 * - 1 <= str.length <= 8
 * - str consists of lowercase English letters.
 */
public class StringPermutations {

    public static void main(String[] args) {
        Set<String> result = permute("abc");
        Assert.assertTrue(result.contains("abc"));
        Assert.assertTrue(result.contains("acb"));
        Assert.assertTrue(result.contains("bac"));
        Assert.assertTrue(result.contains("bca"));
        Assert.assertTrue(result.contains("cab"));
        Assert.assertTrue(result.contains("cba"));
        Assert.assertEquals(result.size(), 6);
    }

    /**
     * APPROACH 1: Recursive with Backtracking (Optimal)
     *
     * Time Complexity: O(N! * N)
     * - N! permutations, each takes O(N) to build.
     *
     * Space Complexity: O(N!)
     * - Storing all permutations.
     */
    static void printPermutn(String str, String ans) {
        if (str.isEmpty()) {
            System.out.print(ans + " ");
            return;
        }

        boolean[] alpha = new boolean[26];
        for (int i = 0; i < str.length(); i++) {
            char ch = str.charAt(i);
            String ros = str.substring(0, i) + str.substring(i + 1);
            if (!alpha[ch - 'a']) printPermutn(ros, ans + ch);
        }
    }
}
```

```

        alpha[ch - 'a'] = true;
    }
}

/**
 * APPROACH 2: Recursive with Set (Handles duplicates)
 *
 * Time Complexity: O(N! * N)
 * - N! permutations, each takes O(N) to build.
 *
 * Space Complexity: O(N!)
 * - Storing all permutations.
 */
static Set<String> permute(String str) {
    if (str == null) {
        return null;
    } else if (str.isEmpty()) {
        Set<String> set = new HashSet<>();
        set.add("");
        return set;
    }
    char firstChar = str.charAt(0);
    String remainingStr = str.substring(1);
    Set<String> words = permute(remainingStr);
    Set<String> permutations = new HashSet<>();
    for (String word : words) {
        for (int j = 0; j <= word.length(); j++) {
            String s = word.substring(0, j) + firstChar + word.substring(j);
            permutations.add(s);
        }
    }
    return permutations;
}
}

```

Topic: twopointers

Code Solutions

ContainerWithMostWater.java

```

package programs.twopointers;

import org.testng.Assert;

import java.util.ArrayList;
import java.util.Arrays;

```

```
import java.util.List;

/**
 * LeetCode 11: Container With Most Water
 *
 * Problem Statement:
 * You are given an integer array height of length n. There are n vertical lines
 * drawn such
 * that the two endpoints of the ith line are (i, 0) and (i, height[i]). Find two
 * lines that together
 * with the x-axis form a container, such that the container contains the most
 * water. Return the
 * maximum amount of water a container can store.
 *
 * Constraints:
 * - n == height.length
 * - 2 <= n <= 10^5
 * - 0 <= height[i] <= 10^4
 */
public class ContainerWithMostWater {

    public static void main(String[] args) {
        List<Integer> height = new ArrayList<>(Arrays.asList(1, 8, 6, 2, 5, 4, 8,
3, 7));
        Assert.assertEquals(calculateMaxArea(height), 49);
    }

    /**
     * APPROACH: Two Pointers (Optimal)
     *
     * Time Complexity: O(N)
     * - Single pass with two pointers.
     *
     * Space Complexity: O(1)
     * - Constant extra space.
     */
    public static int calculateMaxArea(List<Integer> height) {
        int i = 0, j = height.size() - 1;
        int maxArea = Integer.MIN_VALUE;
        while (i <= j) {
            int width = j - i;
            int length = Math.min(height.get(i), height.get(j));
            maxArea = Math.max(maxArea, width * length);

            if (height.get(i) < height.get(j)) {
                i++;
            } else {
                j--;
            }
        }
        return maxArea;
    }

    /**
```

```

    * APPROACH 2: Brute Force
    *
    * Time Complexity: O(N^2)
    * - Nested loops checking all pairs.
    *
    * Space Complexity: O(1)
    * - Constant extra space.
    */
    public static int calculateMaxAreaBruteForce(List<Integer> height) {
        int maxArea = 0;
        int n = height.size();
        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                int width = j - i;
                int length = Math.min(height.get(i), height.get(j));
                maxArea = Math.max(maxArea, width * length);
            }
        }
        return maxArea;
    }
}

```

MoveNegatives.java

```

package programs.twopointers;

import org.testng.Assert;

/**
 * Move Negatives to Beginning
 *
 * Problem Statement:
 * Given an array of integers, move all negative numbers to the beginning of the
 * array
 * while maintaining the relative order among the positive numbers is not
 * required.
 *
 * Constraints:
 * - 1 <= nums.length <= 10^4
 * - -10^4 <= nums[i] <= 10^4
 */
public class MoveNegatives {

    public static void main(String[] args) {
        int[] nums1 = {1, 3, -1, -2, -3, 5, 6, -7};
        moveNegatives(nums1);
        Assert.assertEquals(nums1, new int[]{-1, -2, -3, -7, 1, 3, 5, 6});

        int[] nums2 = {-5, -3, -1, -2, -4};
        moveNegatives(nums2);
    }
}

```



```

        Assert.assertEquals(nums2, new int[]{-5, -3, -1, -2, -4});

        int[] nums3 = {1, 2, 3, 4, 5};
        moveNegatives(nums3);
        Assert.assertEquals(nums3, new int[]{1, 2, 3, 4, 5});
    }

    /**
     * APPROACH: Two Pointers (Optimal)
     *
     * Time Complexity: O(N)
     * - Single pass through the array.
     *
     * Space Complexity: O(1)
     * - In-place swapping.
     */
    private static void moveNegatives(int[] nums) {
        int j = 0;
        for (int i = 0; i < nums.length; i++) {
            if (nums[i] < 0) {
                int temp = nums[i];
                nums[i] = nums[j];
                nums[j] = temp;
                j++;
            }
        }
    }
}

```

MoveNegativesWithOrderMaintain.java

```

package programs.twopointers;

import org.testng.Assert;

import java.util.Arrays;

/**
 * Move Negatives With Order Maintain
 *
 * Problem Statement:
 * Given an array of integers, move all negative numbers to the beginning of the
array
 * while maintaining the relative order of both negative and positive numbers.
 *
 * Constraints:
 * - 1 <= nums.length <= 10^4
 * - -10^4 <= nums[i] <= 10^4
 */
public class MoveNegativesWithOrderMaintain {

```

```

public static void main(String[] args) {
    int[] arr = {-12, 11, -13, -5, 6, -7, 5, -3, -6};
    moveNegatives(arr);
    Assert.assertEquals(arr, new int[]{-12, -13, -5, -7, -3, -6, 11, 6, 5});
}

/**
 * APPROACH: Modified Insertion (Optimal for order maintenance)
 *
 * Time Complexity: O(N^2)
 * - Worst case shifting elements.
 *
 * Space Complexity: O(1)
 * - In-place operations.
 */
private static void moveNegatives(int[] nums) {
    if (nums == null || nums.length <= 1) {
        return;
    }
    int j = 0;
    for (int i = 0; i < nums.length; i++) {
        if (nums[i] < 0) {
            if (i != j) {
                int temp = nums[i];
                for (int k = i; k > j; k--) {
                    nums[k] = nums[k - 1];
                }
                nums[j] = temp;
            }
            j++;
        }
    }
}

```

SwapNumbers.java

```

package programs.twopointers;

import org.testng.Assert;

/**
 * Swap Numbers
 *
 * Problem Statement:
 * Swap two numbers using a temporary variable and also without using a temporary
 * variable.
 */

```

```
* Constraints:
* -  $-10^9 \leq x, y \leq 10^9$ 
*/
public class SwapNumbers {

    public static void main(String[] args) {
        int x = 10, y = 5;
        int[] result1 = swapWithTemp(x, y);
        Assert.assertEquals(result1[0], 5);
        Assert.assertEquals(result1[1], 10);

        int[] result2 = swapWithoutTemp(x, y);
        Assert.assertEquals(result2[0], 5);
        Assert.assertEquals(result2[1], 10);
    }

    /**
     * APPROACH 1: Using Temporary Variable (Optimal)
     *
     * Time Complexity:  $O(1)$ 
     * - Constant time operations.
     *
     * Space Complexity:  $O(1)$ 
     * - Single temporary variable.
     */
    public static int[] swapWithTemp(int x, int y) {
        int t = x;
        x = y;
        y = t;
        return new int[]{x, y};
    }

    /**
     * APPROACH 2: Arithmetic Operations (No Temp Variable)
     *
     * Time Complexity:  $O(1)$ 
     * - Constant time operations.
     *
     * Space Complexity:  $O(1)$ 
     * - No extra space.
     *
     * Note: May cause overflow for large numbers.
     */
    public static int[] swapWithoutTemp(int x, int y) {
        x = x + y;
        y = x - y;
        x = x - y;
        return new int[]{x, y};
    }

    /**
     * APPROACH 3: XOR Operation (No Temp Variable, No Overflow)
     *
     * Time Complexity:  $O(1)$ 
     */
}
```

```
* - Constant time operations.  
*  
* Space Complexity: O(1)  
* - No extra space.  
*/  
public static int[] swapWithXOR(int x, int y) {  
    x = x ^ y;  
    y = x ^ y;  
    x = x ^ y;  
    return new int[]{x, y};  
}  
}
```
